Contents

```r
##############################################################################
# Section 1: Setup and Dependencies
# Purpose: Load all required packages for data I/O, manipulation, and modelling.
# Coding & Reproducibility: Clearly separate package imports; ensures anyone
# can reproduce environment by installing these libraries. :contentReference[oaicite:0]{index=0}
##############################################################################
library(tidyverse)   # Includes dplyr (data wrangling), ggplot2 (plotting), readr (CSV I/O)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(readxl)      # For reading Excel files (FDWF data)
library(mgcv)        # For fitting Generalized Additive Models (GAMs)
```

```
## Loading required package: nlme
##
## Attaching package: 'nlme'
##
## The following object is masked from 'package:dplyr':
##
##     collapse
##
## This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.
```

```r
library(pracma)      # Provides numerical tools, e.g., trapezoidal integration trapz()
```

```
##
## Attaching package: 'pracma'
##
## The following object is masked from 'package:mgcv':
##
##     magic
##
## The following object is masked from 'package:purrr':
##
##     cross
```

```r
library(lubridate)   # Simplifies date-time parsing and extraction (ymd_hm, hour)
library(gratia)      # GAM diagnostics (draw(), appraise(), etc.)
```

```
##
```

```
## Attaching package: 'gratia'
##
## The following object is masked from 'package:pracma':
##
##     fderiv
##
## The following object is masked from 'package:stringr':
##
##     boundary
```

```r
################################################################################
# Section 2: Data Paths and Import
# Purpose: Define file locations and read raw PSD and weighting data.
# Style & Clarity: Variables clearly named; paths at top for easy configuration.
################################################################################
# File paths for PSD background, PSD operational, and frequency-weighting data
bg_csv  <- "Background_Frequency_Domain_WS12.csv"
op_csv  <- "Operational_Frequency_Domain_WS12.csv"
fdwf_xl <- "FDWF_Data.xlsx"

# Read background and operational PSD tables (CSV)
bg <- read_csv(bg_csv)   # Background noise spectra
```

```
## New names:
## Rows: 73728 Columns: 12
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (1): Hour dbl (10): ...1, Frequency..Hz., PSD.Displacement..m.2.Hz.,
## Frequency.Depend... date (1): Date
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```r
op <- read_csv(op_csv)   # Operational (turbine ON) spectra
```

```
## New names:
## Rows: 311296 Columns: 12
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (1): Hour dbl (10): ...1, Frequency..Hz., PSD.Displacement..m.2.Hz.,
## Frequency.Depend... date (1): Date
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```r
# Read frequency weighting function (Excel sheet 1) and rename columns for clarity
fdwf <- read_excel(fdwf_xl, sheet = 1) %>%
  rename(
    Frequency = Frequency_Hz,
    Weight    = Frequency_Distance_Weighting_Function
  )
```

```r
# Quick data inspection to verify import success and column names
glimpse(bg)
```

```
## Rows: 73,728
## Columns: 12
## $ ...1                                          <dbl> 1, 2, 3, 4, 5, 6, 7, 8, ~
## $ Frequency..Hz.                                <dbl> 0.006103516, 0.012207031~
## $ PSD.Displacement..m.2.Hz.                     <dbl> 6.830252e-11, 4.254498e-~
## $ Frequency.Dependent.PSD.Displacement..m.2.Hz. <dbl> 2.667622e-24, 1.646402e-~
## $ Date                                          <date> 2020-03-28, 2020-03-28,~
## $ Hour                                          <chr> "1510", "1510", "1510", ~
## $ Wind.speed                                    <dbl> 12.02112, 12.02112, 12.0~
## $ Rounded.Wind.speed                            <dbl> 12, 12, 12, 12, 12, 12, ~
## $ Wind.direction.degrees                        <dbl> 37.08, 37.08, 37.08, 37.~
## $ Wind.direction.radians                        <dbl> 0.6471681, 0.6471681, 0.~
## $ No.of.samples                                 <dbl> 100, 100, 100, 100, 100,~
## $ Operational                                   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, ~
```

```r
glimpse(op)
```

```
## Rows: 311,296
## Columns: 12
## $ ...1                                          <dbl> 1, 2, 3, 4, 5, 6, 7, 8, ~
## $ Frequency..Hz.                                <dbl> 0.006103516, 0.012207031~
## $ PSD.Displacement..m.2.Hz.                     <dbl> 8.324401e-11, 3.025114e-~
## $ Frequency.Dependent.PSD.Displacement..m.2.Hz. <dbl> 3.251176e-24, 1.170656e-~
## $ Date                                          <date> 2021-09-22, 2021-09-22,~
## $ Hour                                          <chr> "1130", "1130", "1130", ~
## $ Wind.speed                                    <dbl> 11.58977, 11.58977, 11.5~
## $ Rounded.Wind.speed                            <dbl> 12, 12, 12, 12, 12, 12, ~
## $ Wind.direction.degrees                        <dbl> 216.7858, 216.7858, 216.~
## $ Wind.direction.radians                        <dbl> 3.783626, 3.783626, 3.78~
## $ No.of.samples                                 <dbl> 100, 100, 100, 100, 100,~
## $ Operational                                   <dbl> 1, 1, 1, 1, 1, 1, 1, 1, ~
```

```r
glimpse(fdwf)
```

```
## Rows: 8,193
## Columns: 2
## $ Frequency <dbl> 0.000000000, 0.006103516, 0.012207031, 0.018310547, 0.024414~
## $ Weight    <dbl> 3.941719e-14, 3.905598e-14, 3.869792e-14, 3.834299e-14, 3.79~
```

```r
################################################################################
# Section 3: Data Cleaning Function
# Purpose: Standardise column names and tag dataset with operational flag.
# Style & Clarity: roxygen-style documentation aids reader comprehension.
################################################################################
#' Clean and standardise PSD dataset
#'
#' @param df A raw PSD data frame with original column names.
#' @param op_flag Integer flag: 0 for background, 1 for operational.
```

```r
#' @return A tibble with renamed columns and an 'Operational' indicator.
clean_psd <- function(df, op_flag) {
  df %>%
    rename(
      Frequency   = `Frequency..Hz.`,                 # rename to generic Frequency (Hz)
      PSD         = `PSD.Displacement..m.2.Hz.`,       # PSD in m²/Hz
      Date        = Date,                              # measurement date
      Hour        = Hour,                              # measurement hour
      Wind.speed  = Wind.speed,                        # wind speed (m/s)
      Wind.dir    = `Wind.direction.degrees`           # wind direction (°)
    ) %>%
    mutate(Operational = op_flag)                      # label dataset type
}

# Apply cleaning function to both datasets
bg <- clean_psd(bg, 0)  # background data
op <- clean_psd(op, 1)  # turbine operational data

##############################################################################
# Section 4: Merge and Weight PSD Data
# Purpose: Combine bg & op, join weighting function, convert to nm²/Hz.
# Computational Rationale: Weighting accounts for distance attenuation;
# conversion to nm²/Hz aligns units for seismic analysis.
##############################################################################
df <- bind_rows(bg, op) %>%                            # stack background + operational
  left_join(fdwf, by = "Frequency") %>%                # add distance weighting
  mutate(
    PSD_wtd_nm2 = PSD * Weight * 1e18                   # m²→nm² conversion × weighting
  ) %>%
  drop_na(PSD_wtd_nm2, Wind.speed, Wind.dir)            # drop incomplete records

##############################################################################
# Section 5: Create Time and Direction Variables
# Purpose: Build datetime, extract components, normalise wind direction.
# Reproducibility: Use lubridate for robust datetime parsing.
##############################################################################
freq_df <- df %>%
  mutate(
    Hour_str    = sprintf("%04d", as.numeric(Hour)),           # pad hour to HHMM
    Datetime    = ymd_hm(paste(Date, Hour_str), tz = "UTC"),  # combine date and hour
    Day         = factor(Date),                                # treat each day as factor
    HourOfDay   = hour(Datetime),                              # extract hour [0-23]
    WindDir     = Wind.dir %% 360,                             # wrap direction to [0,360)
    Operational = factor(Operational)                          # factor for modelling
  )

##############################################################################
# Section 6: Compute Integrated Energy per 10-min Block
# Purpose: Numerically integrate PSD over 0.5-8 Hz to derive seismic energy E.
# Scientific Rationale: Band-limited energy focuses on turbine noise band.
##############################################################################
energy_df <- freq_df %>%
  filter(between(Frequency, 0.5, 8)) %>%                     # select turbine band
```

```r
  arrange(Day, HourOfDay, Frequency) %>%                      # ensure monotonic order
  group_by(Day, HourOfDay, Wind.speed, WindDir, Operational) %>%
  summarise(
    E = trapz(Frequency, PSD_wtd_nm2),                        #   PSD df (nm²)
    .groups = "drop"
  ) %>%
  mutate(
    logE      = log10(pmax(E, 1e-20)),                        # stabilise variance
    HourOfDay = as.numeric(HourOfDay),                        # ensure numeric for GAM
    Day       = factor(Day)                                   # re-factor for random effect
  )

# End of initial data preparation pipeline - ready for modelling.
```

```r
##############################################################################
# Section 8: Exploratory Data Analysis Plots
# Purpose: Visualise how log10(Integrated Energy) varies with wind speed,
#          operational state, time of day, and over time to check model residual
#          patterns and remaining structure.
#################################################################################

# Load essential plotting and data-wrangling libraries
library(ggplot2)   # For plotting
library(dplyr)     # For data manipulation
library(lubridate) # For working with date-time objects

# Define a consistent color scheme for turbine state:
#   0 = Background (off), 1 = Operational (on)
custom_colors <- c("0" = "#E69F00", "1" = "#0072B2")  # Orange and Blue

# Apply default color and fill scales to enforce consistency across figures
scale_color_manual(values = custom_colors)
```

```
## <ggproto object: Class ScaleDiscrete, Scale, gg>
##     aesthetics: colour
##     axis_order: function
##     break_info: function
##     break_positions: function
##     breaks: waiver
##     call: call
##     clone: function
##     dimension: function
##     drop: TRUE
##     expand: waiver
##     get_breaks: function
##     get_breaks_minor: function
##     get_labels: function
##     get_limits: function
##     get_transformation: function
##     guide: legend
##     is_discrete: function
##     is_empty: function
##     labels: waiver
```

```
##     limits: function
##     make_sec_title: function
##     make_title: function
##     map: function
##     map_df: function
##     n.breaks.cache: NULL
##     na.translate: TRUE
##     na.value: grey50
##     name: waiver
##     palette: function
##     palette.cache: NULL
##     position: left
##     range: environment
##     rescale: function
##     reset: function
##     train: function
##     train_df: function
##     transform: function
##     transform_df: function
##     super:  <ggproto object: Class ScaleDiscrete, Scale, gg>
```

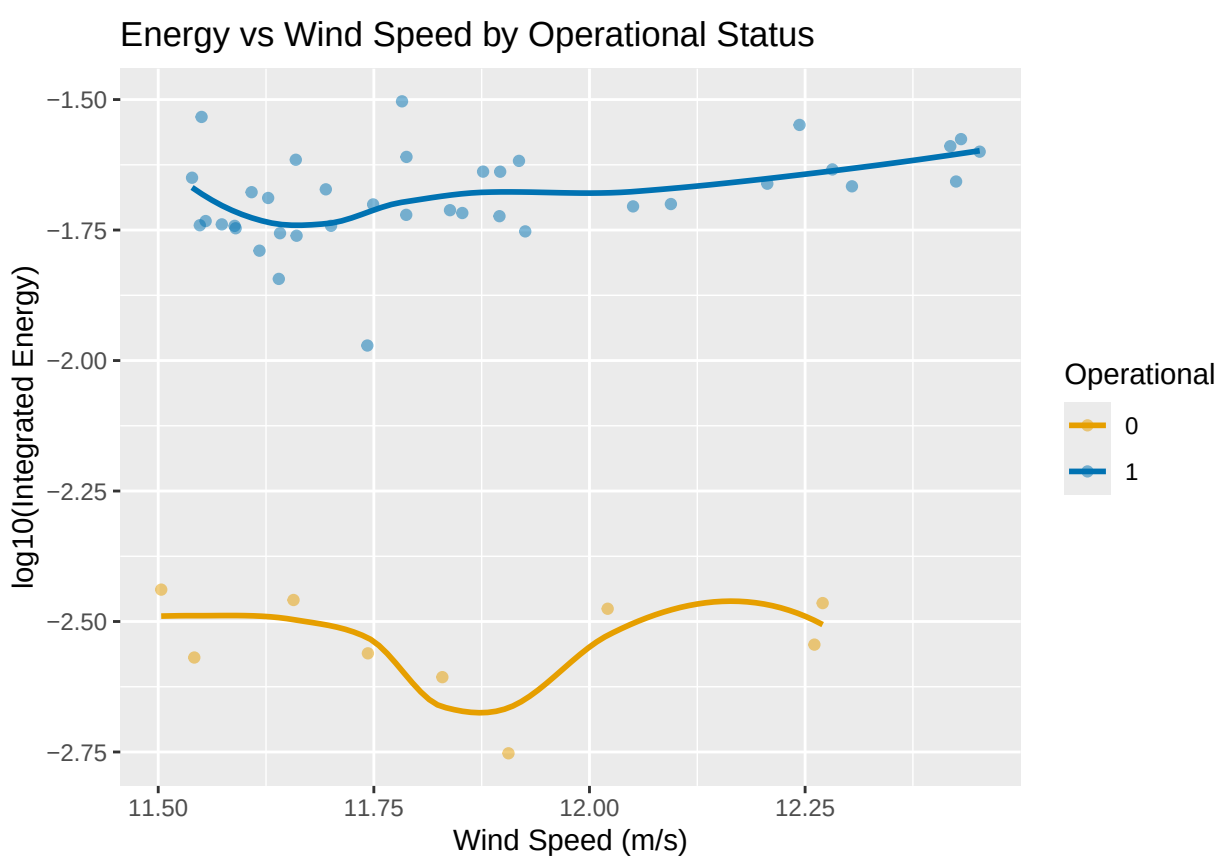
```r
scale_fill_manual(values = custom_colors)
```

```
## <ggproto object: Class ScaleDiscrete, Scale, gg>
##     aesthetics: fill
##     axis_order: function
##     break_info: function
##     break_positions: function
##     breaks: waiver
##     call: call
##     clone: function
##     dimension: function
##     drop: TRUE
##     expand: waiver
##     get_breaks: function
##     get_breaks_minor: function
##     get_labels: function
##     get_limits: function
##     get_transformation: function
##     guide: legend
##     is_discrete: function
##     is_empty: function
##     labels: waiver
##     limits: function
##     make_sec_title: function
##     make_title: function
##     map: function
##     map_df: function
##     n.breaks.cache: NULL
##     na.translate: TRUE
##     na.value: grey50
##     name: waiver
##     palette: function
##     palette.cache: NULL
```

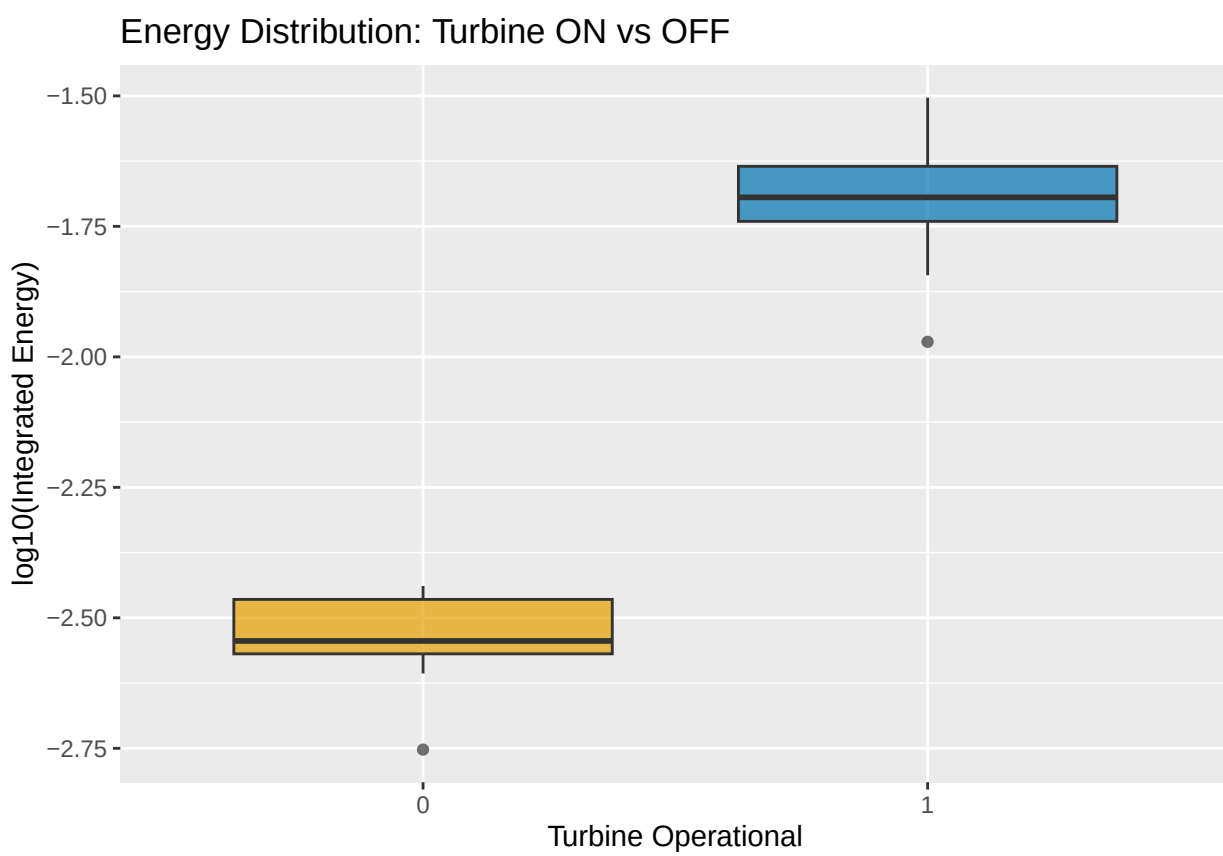
```
##     position: left
##     range: environment
##     rescale: function
##     reset: function
##     train: function
##     train_df: function
##     transform: function
##     transform_df: function
##     super:  <ggproto object: Class ScaleDiscrete, Scale, gg>
```

```r
# Plot 1: Scatterplot of log10(Integrated Energy) vs Wind Speed
#   - Each point is a 10-min block
#   - Color indicates turbine ON/OFF
#   - LOESS smoother highlights trend and nonlinearity
ggplot(energy_df, aes(x = Wind.speed, y = logE, color = Operational)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "loess", se = FALSE) +
  scale_color_manual(values = custom_colors) +
  labs(
    x     = "Wind Speed (m/s)",
    y     = "log10(Integrated Energy)",
    color = "Operational",
    title = "Energy vs Wind Speed by Operational Status"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



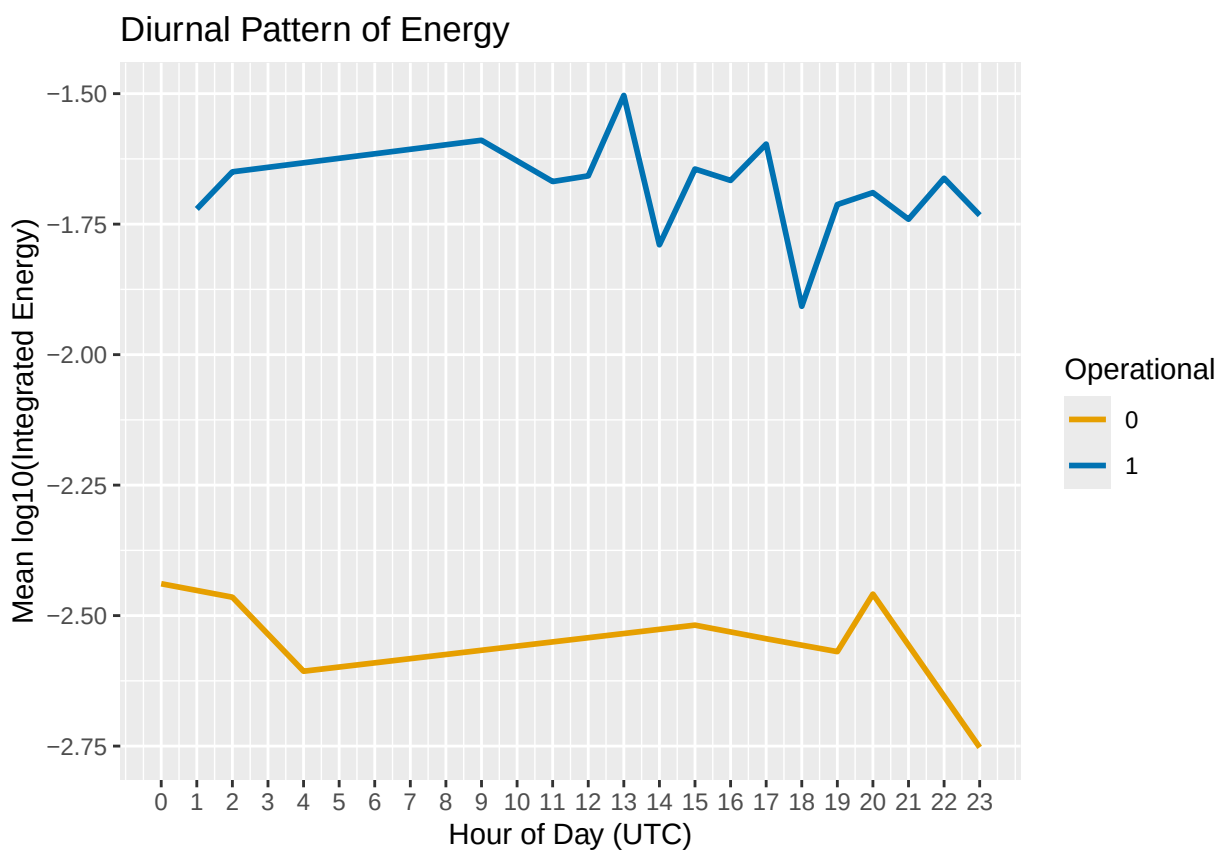
```r
# Plot 2: Boxplot of log10(Integrated Energy) by Operational State
#   - Summarises median, interquartile range, and outliers
#   - Useful to compare background noise vs operational noise distributions
ggplot(energy_df, aes(x = Operational, y = logE, fill = Operational)) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_manual(values = custom_colors) +
  labs(
    x     = "Turbine Operational",
    y     = "log10(Integrated Energy)",
    title = "Energy Distribution: Turbine ON vs OFF"
  ) +
  theme(legend.position = "none")  # Legend is redundant with x-axis labels
```




```r
# Plot 3: Diurnal cycle of mean log10(E) by hour and operational state
#   - Aggregates logE within each hour to reveal time-of-day patterns
#   - Checks for residual diurnal effects not captured by the GAM
energy_df %>%
  group_by(HourOfDay, Operational) %>%
  summarize(mean_logE = mean(logE, na.rm = TRUE), .groups = "drop") %>%
  ggplot(aes(x = HourOfDay, y = mean_logE, color = Operational)) +
    geom_line(size = 1) +
    scale_x_continuous(breaks = 0:23) +
    scale_color_manual(values = custom_colors) +
    labs(
      x     = "Hour of Day (UTC)",
```

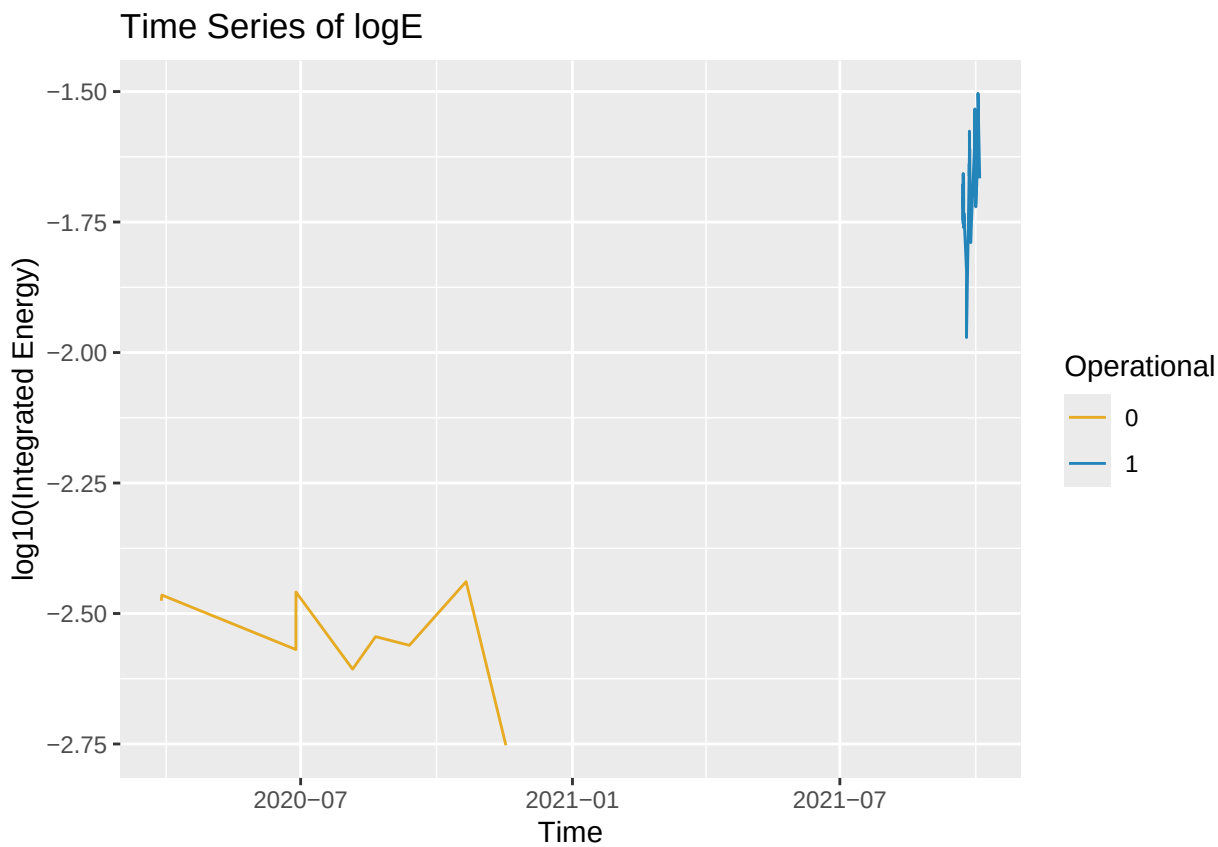
```
    y     = "Mean log10(Integrated Energy)",
    title = "Diurnal Pattern of Energy"
  )
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



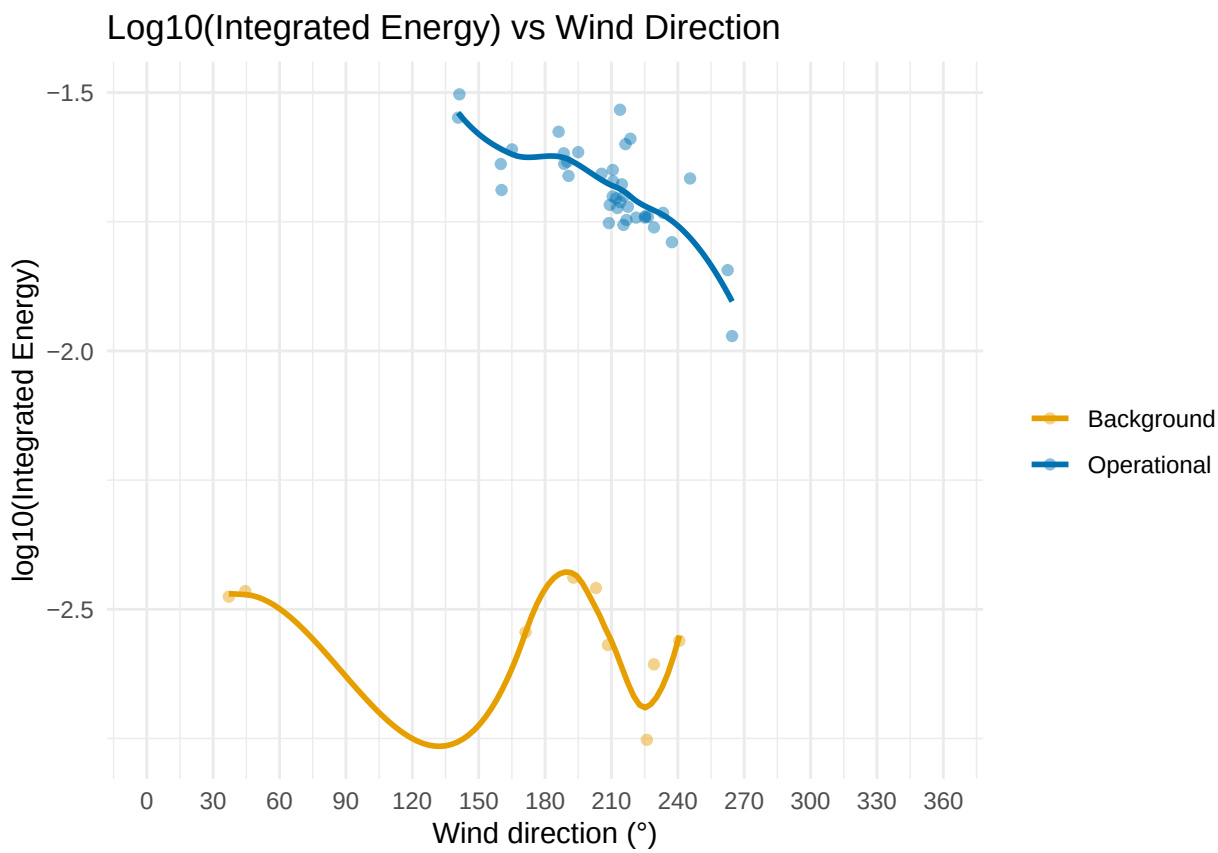

```
# Plot 4: Time series of log10(Integrated Energy) across the full record
#   - Reconstructs POSIXct datetime from Day and HourOfDay
#   - Identifies any long-term trends or anomalies over time
energy_df %>%
  arrange(Day, HourOfDay) %>%
  mutate(
    datetime = ymd_hm(paste(Day, sprintf("%02d:00", HourOfDay)), tz = "UTC")
  ) %>%
  ggplot(aes(x = datetime, y = logE, color = Operational)) +
    geom_line(alpha = 0.85) +
    scale_color_manual(values = custom_colors) +
    labs(
      x     = "Time",
      y     = "log10(Integrated Energy)",
```

```
      title = "Time Series of logE"
    )
```



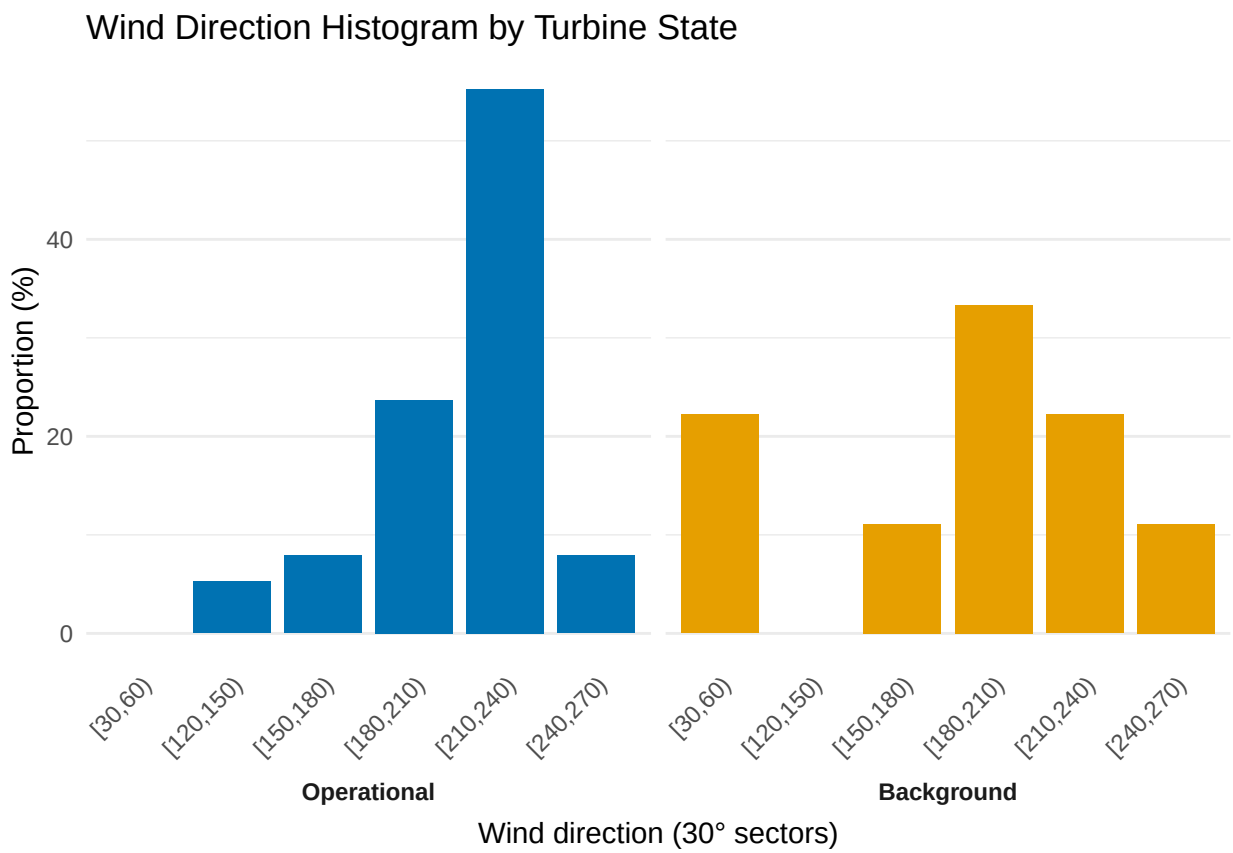

```
################################################################################
# Purpose: Explore how the fitted seismic energy (log10(E)) varies with wind
#          direction under Background vs Operational turbine states.
# Rationale: Highlights any directional bias in residual energy after modelling.
# Reproducibility: Assumes `energy_df` contains precomputed `WindDir`, `logE`,
#                 and `Operational` from the GAM pipeline above.
################################################################################

# Load essential libraries
library(dplyr)   # Data wrangling (mutate, %>%, etc.)
library(ggplot2) # Grammar of Graphics for plotting

# 1. Prepare the data by converting Operational flag (0/1) into a descriptive factor 'Type'
#    - Improves plot readability by using meaningful labels instead of numeric codes.
energy <- energy_df %>%
  mutate(
    Type = factor(
      Operational,
      levels = c("0", "1"),
      labels = c("Background", "Operational")
    )
  )
```

```r
# 2. Define custom colors for the two 'Type' levels
#    - Orange for Background, Blue for Operational ensure clear visual contrast.
cols <- c(
  "Background"  = "#E69F00",  # Orange shade
  "Operational" = "#0072B2"   # Blue shade
)

# 3. Build the plot: logE vs Wind Direction, colored by Type
#    - Points are semi-transparent to reduce overplotting.
#    - LOESS smoothing curve added without confidence intervals for clarity.
#    - Wind direction axis formatted from 0° to 360° in 30° increments.
p_dir <- ggplot(energy, aes(x = WindDir, y = logE, colour = Type)) +
  geom_point(alpha = 0.45) +
  geom_smooth(
    method    = "loess",
    formula   = y ~ x,
    se        = FALSE,
    linewidth = 1
  ) +
  scale_colour_manual(values = cols, name = "") +   # Apply custom colors, hide legend title
  scale_x_continuous(
    breaks = seq(0, 360, by = 30),
    limits = c(0, 360)
  ) +
  labs(
    title = "Log10(Integrated Energy) vs Wind Direction",
    x     = "Wind direction (°)",
    y     = "log10(Integrated Energy)"
  ) +
  theme_minimal()  # Clean, minimal theme for publication quality

# 4. Render the plot
print(p_dir)
```



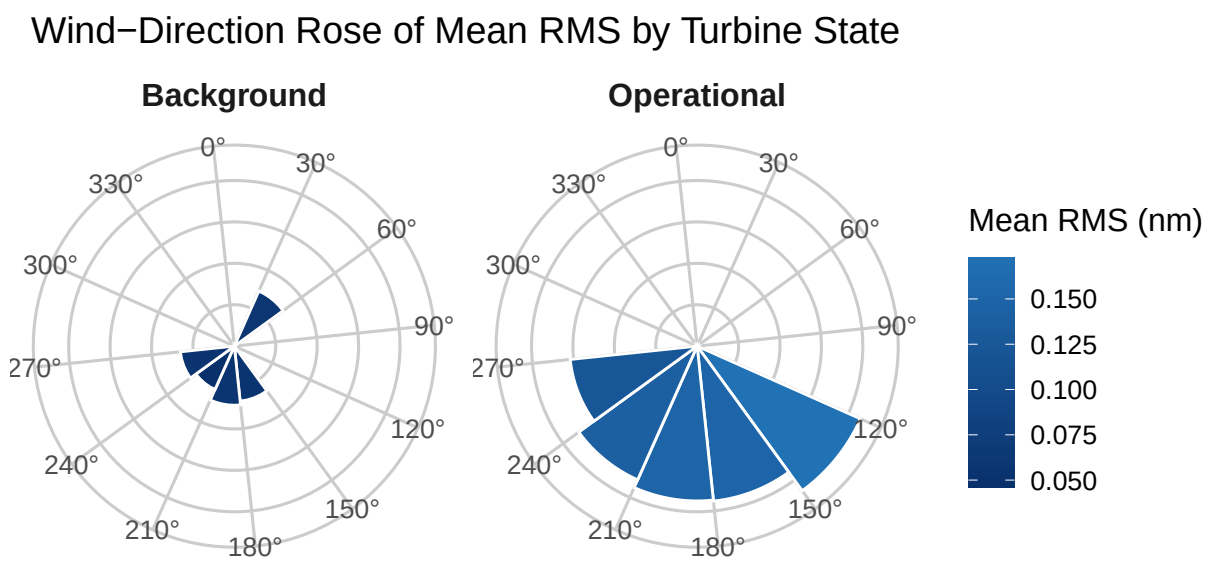

```
##############################################################################
# Side-by-Side Wind-Direction Histograms by Turbine State
# Purpose: Visualise how wind direction frequencies differ between
#          Background (turbine OFF) and Operational (turbine ON) periods.
# Rationale: Faceting by state aids direct comparison of directional distributions.
#############################################################################

# Assumes dplyr and ggplot2 are already loaded.

# 1. Bin WindDir into 30° sectors and compute percentage of observations per Type
dir_df <- energy %>%
  mutate(
    # Create 30°-wide bins from 0° to 360°
    WD_bin = cut(
      WindDir,
      breaks = seq(0, 360, 30),
      include.lowest = TRUE,
      right = FALSE
    ),
    # Ensure the factor levels order for consistent plotting (Operational first)
    Type = factor(Type, levels = c("Operational", "Background"))
  ) %>%
  count(Type, WD_bin) %>%              # Count observations within each bin for each Type
  group_by(Type) %>%                   # Group by Type to calculate proportions
  mutate(
    prop = n / sum(n) * 100            # Convert counts to percentage of total per Type
```

```r
  ) %>%
  ungroup()                              # Remove grouping for downstream operations

# 2. Define custom fill colors for each Type
#    - Operational: blue; Background: orange
cols <- c(
  "Background"  = "#E69F00",
  "Operational" = "#0072B2"
)

# 3. Build the side-by-side bar plot of wind-direction proportions
p_dir_hist_side <- ggplot(dir_df, aes(x = WD_bin, y = prop, fill = Type)) +
  geom_col(
    width = 0.85,        # Narrow bars leave small gaps for readability
    show.legend = FALSE  # Legend omitted because facets indicate Type
  ) +
  facet_grid(. ~ Type, switch = "x") +  # Horizontal facets: left = Background, right = Operational
  scale_fill_manual(values = cols) +    # Apply custom color mapping
  labs(
    title = "Wind Direction Histogram by Turbine State",
    x     = "Wind direction (30° sectors)",
    y     = "Proportion (%)"
  ) +
  theme_minimal(base_size = 11) +
  theme(
    strip.placement    = "outside",               # Move facet labels outside for clarity
    strip.text.x       = element_text(face = "bold"),
    axis.text.x        = element_text(angle = 45, hjust = 1),  # Tilt labels for readability
    panel.grid.major.x = element_blank()          # Remove vertical grid lines
  )

# 4. Render the plot
print(p_dir_hist_side)
```



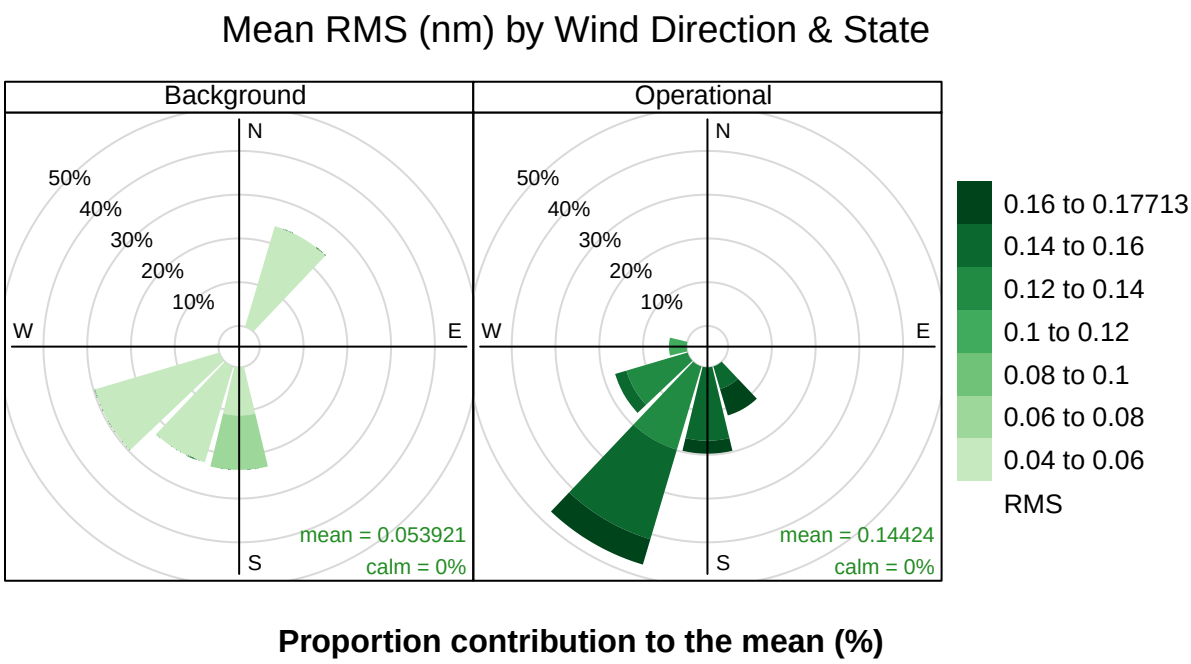

```
##############################################################################
# Wind-Direction Rose of Mean RMS by Turbine State
# Purpose: Display how mean RMS displacement varies with wind direction
#          for Background vs Operational periods in a polar "rose" format.
# Rationale: Highlights directional dependence in seismic noise under different
#            turbine states, directly addressing H : wind-direction effect.
##############################################################################

# 1. Load required libraries for data manipulation and plotting
library(dplyr)   # Data wrangling: mutate, group_by, summarise
library(ggplot2) # Grammar of Graphics plotting system

# 2. Prepare the summary data frame for the wind rose plot from energy_df
rose_df <- energy_df %>%
  mutate(
    # Compute RMS displacement in nm from energy (nm²)
    RMS = sqrt(E),
    # Convert numeric Operational flag into descriptive factor for clarity
    Type = factor(
      Operational,
      levels = c("0", "1"),
      labels = c("Background", "Operational")
    )
  ) %>%
  mutate(
    # Bin wind directions into 30° sectors labelled by their midpoints
```

```r
    wd_bin = cut(
      WindDir,
      breaks = seq(0, 360, by = 30),
      include.lowest = TRUE,
      right = FALSE,
      labels = seq(15, 345, by = 30)
    ),
    # Convert factor labels to numeric mid-point values for angular plotting
    wd_bin = as.numeric(as.character(wd_bin))
  ) %>%
  group_by(Type, wd_bin) %>%
  summarise(
    # Compute mean RMS within each bin and turbine state
    mean_RMS = mean(RMS, na.rm = TRUE),
    .groups   = "drop"
  )

# 3. Create the polar bar chart ("wind rose")
ggplot(rose_df, aes(x = wd_bin, y = mean_RMS, fill = mean_RMS)) +
  # Bars represent average RMS per directional sector
  geom_col(width = 30, color = "white") +
  # Transform to polar coordinates for rose appearance
  coord_polar(start = -pi/30) +
  # Label x-axis at every 30° increment
  scale_x_continuous(
    breaks = seq(0, 330, by = 30),
    labels = paste0(seq(0, 330, by = 30), "°"),
    limits = c(0, 360)
  ) +
  # Facet side-by-side for Background vs Operational
  facet_wrap(~ Type, nrow = 1) +
  # Blue gradient fill encoding magnitude of mean RMS
  scale_fill_gradient(
    low  = "#08306b",      # Dark blue for low RMS
    high = "#2171b5",      # Lighter blue for high RMS
    name = expression("Mean RMS (nm)")
  ) +
  # Titles and labels
  labs(
    title = "Wind-Direction Rose of Mean RMS by Turbine State",
    x     = NULL,  # No x-label (compass orientation implicit)
    y     = NULL   # No y-label in polar context
  ) +
  # Clean theme and styling adjustments
  theme_minimal(base_size = 12) +
  theme(
    strip.text        = element_text(face = "bold", size = 12),
    axis.text.y       = element_blank(),         # Remove radial grid labels
    panel.grid.minor  = element_blank(),         # Remove minor grid lines
    panel.grid.major  = element_line(color = "grey80"),
    legend.position   = "right",
    plot.title        = element_text(hjust = 0.5)
  )
```




```
##############################################################################
# Wind Directional Dependence of RMS Displacement (Green Palette)
# Purpose: Create a polar-rose of mean RMS displacement by wind direction,
#          split by turbine state (Background vs Operational), using a
#          5-shade green gradient.
#############################################################################

# 0) Load required libraries (skip if already loaded in your session)
library(openair)      # pollutionRose function for polar-rose plots
library(RColorBrewer) # ColorBrewer palettes (e.g. "Greens")
library(dplyr)        # Data manipulation (mutate, transmute, etc.)
library(lubridate)    # Date-time parsing (ymd_hm)

# 1) Prepare the 'energy' table with datetime, RMS, and a descriptive Type factor
energy <- energy_df %>%
  mutate(
    # Reconstruct a POSIXct timestamp at the top of each hour
    Datetime = ymd_hm(
      paste0(as.character(Day), " ", sprintf("%02d:00", HourOfDay)),
      tz = "UTC"
    ),
    # Compute RMS displacement in nanometers from integrated energy (E in nm²)
    RMS_nm = sqrt(E),
    # Create a factor for turbine state with clear labels
    Type   = factor(
      Operational,
```

```r
      levels = c(0, 1),
      labels = c("Background", "Operational")
    )
  )

# 2) Define a custom 5-shade green gradient (from light to dark)
green_grad <- colorRampPalette(
  brewer.pal(9, "Greens")[3:9]
)(5)

# 3) Build the data frame in the format expected by pollutionRose()
rose_df <- energy %>%
  transmute(
    date = Datetime,   # Timestamp
    ws   = Wind.speed, # Wind speed (m/s)
    wd   = WindDir,    # Wind direction (°)
    RMS  = RMS_nm,     # RMS displacement (nm)
    Type = Type        # Turbine state
  )

# 4) Generate the polar-rose plot
pollutionRose(
  mydata       = rose_df,
  pollutant    = "RMS",         # Variable to summarise
  statistic    = "prop.mean",   # Use the mean proportion method
  type         = "Type",        # Facet by Background vs Operational
  cols         = green_grad,    # Apply the custom green palette
  paddle       = FALSE,         # Use rectangular legend boxes
  key.position = "right",       # Position legend on the right
  main         = "Mean RMS (nm) by Wind Direction & State"
)
```

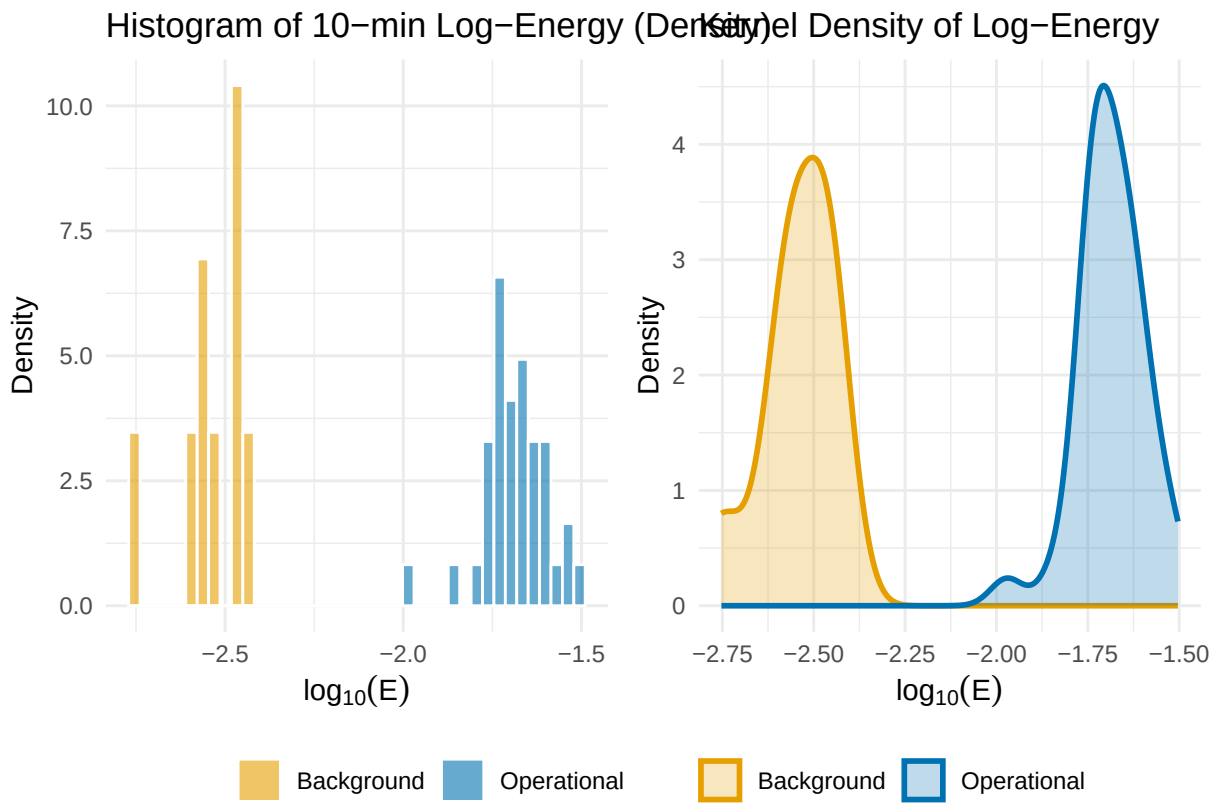



```
##############################################################################
# Distributional Comparison of log10(E) by Turbine State
# Purpose: Visually compare the distribution (histogram) and density estimates
#          of 10-minute integrated seismic energy (log  scale) for Background
#          versus Operational periods, as a post-modeling check.
# Rationale: Ensures the GAM has captured key differences in energy distributions
#            between turbine ON/OFF states; aids interpretation of residual structure.
##############################################################################

# 1. Load required libraries (skip if already loaded)
library(ggplot2)   # Grammar of Graphics for plotting
library(dplyr)     # Data manipulation (used below for data preparation)
library(patchwork) # Combine multiple ggplots with shared legends

# 2. Data prerequisites:
#    - `energy` must be a data frame with:
#        logE : numeric vector of log10-transformed integrated energy values
#        Type : factor with levels "Background" and "Operational"
#    Ensures consistent mapping between data and aesthetics in the plots.

# 3. Density-scaled histogram of logE by Type
#    - y = ..density.. maps histogram heights to density for direct comparison
#    - bins = 40 chosen to balance granularity and noise
#    - alpha and white borders improve overplotting visibility
p_hist <- ggplot(energy, aes(x = logE, fill = Type)) +
  geom_histogram(
```

```r
    aes(y = ..density..),   # Plot density instead of counts
    bins     = 40,          # Number of histogram bins
    alpha    = 0.6,         # Semi-transparent bars to show overlap
    position = "identity",  # Overlay bars for both levels of Type
    colour   = "white"      # White borders to delineate bins
  ) +
  scale_fill_manual(
    values = c(
      "Background"  = "#E69F00",  # Orange for turbine OFF
      "Operational" = "#0072B2"   # Blue for turbine ON
    ),
    name = ""                      # No legend title for cleaner appearance
  ) +
  labs(
    title = "Histogram of 10-min Log-Energy (Density)",
    x     = expression(log[10](E)),  # Mathematical notation for log (E)
    y     = "Density"
  ) +
  theme_minimal() +
  theme(legend.position = "none")     # Legend hidden; will be shared later

# 4. Kernel density curves of logE by Type
#    - Smooth density estimate with bandwidth adjust = 1.3 for moderate smoothing
#    - alpha fill improves perception of overlapping areas
p_dens <- ggplot(energy, aes(x = logE, colour = Type, fill = Type)) +
  geom_density(
    aes(y = ..density..),  # Explicit density mapping
    alpha  = 0.25,         # Semi-transparent fill
    adjust = 1.3,          # Controls smoothness of kernel
    linewidth = 1          # Line width of density curves
  ) +
  scale_colour_manual(
    values = c(
      "Background"  = "#E69F00",
      "Operational" = "#0072B2"
    ),
    name = ""              # No legend title
  ) +
  scale_fill_manual(
    values = c(
      "Background"  = "#E69F00",
      "Operational" = "#0072B2"
    ),
    name = ""
  ) +
  labs(
    title = "Kernel Density of Log-Energy",
    x     = expression(log[10](E)),
    y     = "Density"
  ) +
  theme_minimal() +
  theme(legend.position = "none")     # Legend hidden for combined display
```

```r
# 5. Combine the two plots side-by-side with a shared legend at the bottom
#    - patchwork::plot_layout(guides = "collect") gathers legends from both plots
#    - theme(legend.position = "bottom") places the shared legend below
p_combo <- (p_hist | p_dens) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom")

# 6. Render the combined figure
print(p_combo)
```

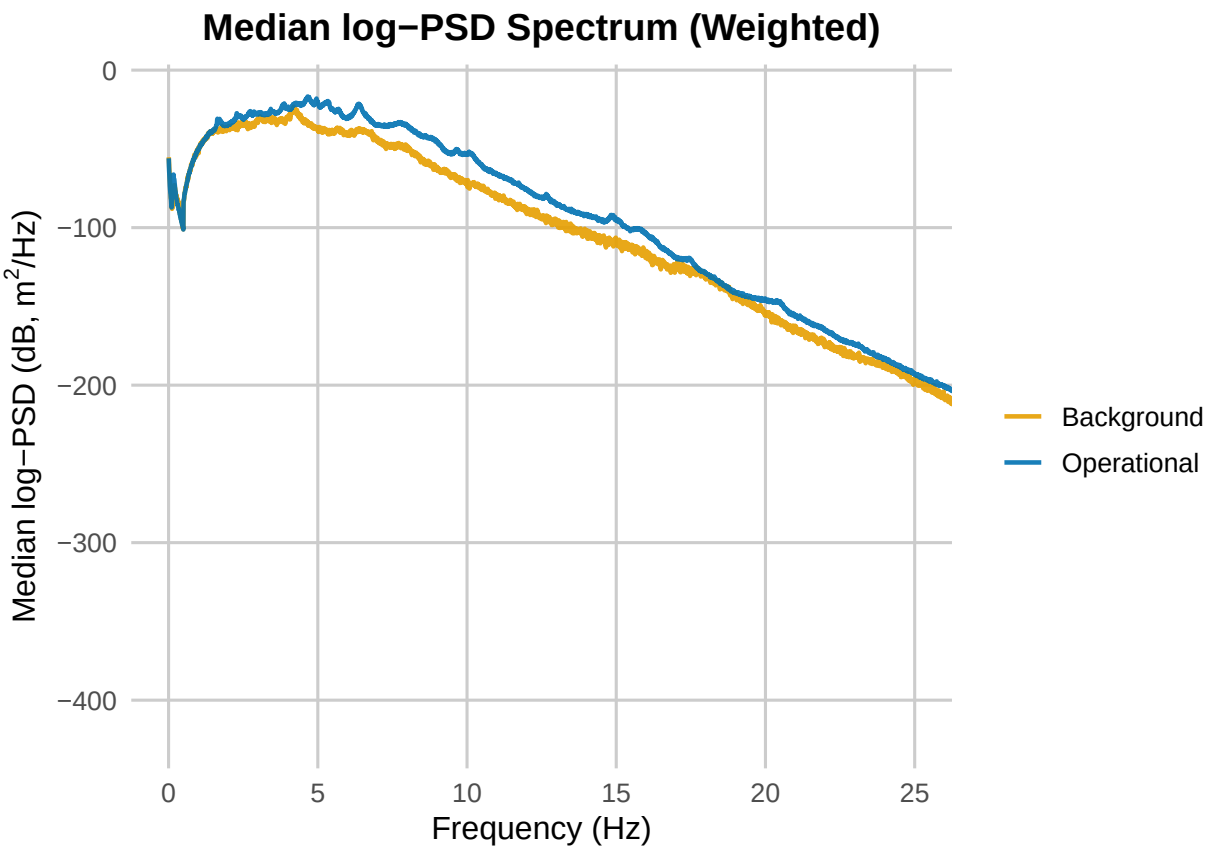
```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
##############################################################################
# Median log-PSD Spectrum by Turbine State
# Purpose: Compare the frequency-domain seismic energy (PSD) between
#          background and operational periods by plotting the median
#          log-scaled, distance-weighted PSD across frequency.
# Context: Addresses research objective to quantify how turbine operation
#          shifts the seismic spectrum (0.5-25 Hz focus).
##############################################################################
```

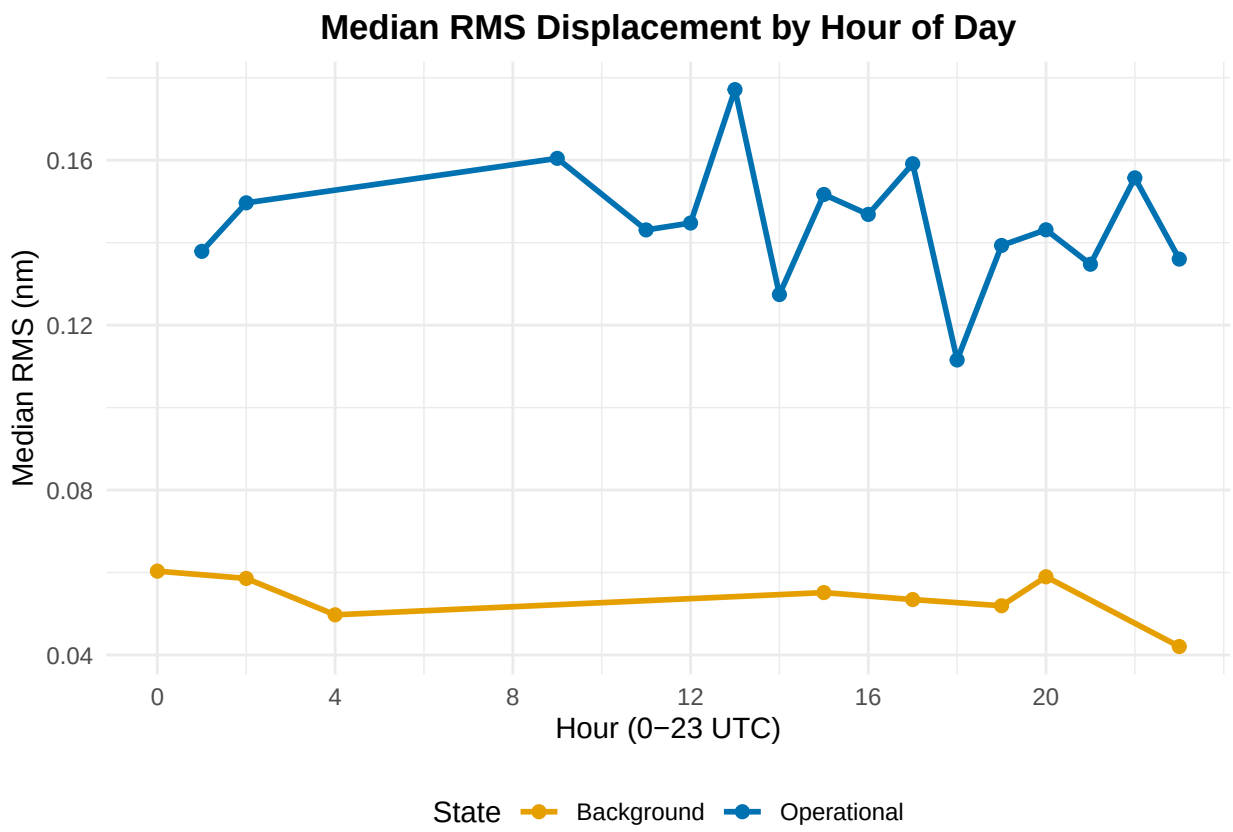
```r
# 1. Load required libraries (skip if already loaded in previous sections)
#    dplyr for data manipulation; ggplot2 for plotting
library(dplyr)
library(ggplot2)

# 2. Summarise the PSD data:
#    - Convert PSD (nm²/Hz) to decibels: 10·log10()
#    - Group by Operational state and Frequency to capture spectral differences
spec_df <- freq_df %>%
  mutate(
    logPSD = 10 * log10(PSD_wtd_nm2)   # Transform to dB scale for interpretability
  ) %>%
  group_by(Operational, Frequency) %>% # Separate background vs operational
  summarise(
    med_logPSD = median(logPSD, na.rm = TRUE), # Median reduces influence of outliers
    .groups     = "drop"                        # Ungroup to avoid carrying grouping downstream
  )

# 3. Define custom colours for clarity:
#    - Orange for Background (Operational == 0)
#    - Blue for Operational (Operational == 1)
cols <- c(
  "0" = "#E69F00",  # Background
  "1" = "#0072B2"   # Operational
)

# 4. Create the line plot of the median log-PSD spectrum:
#    - Lines coloured by turbine state
#    - Zoom into 0-25 Hz where turbine noise is most relevant
p_spec <- ggplot(spec_df, aes(
    x      = Frequency,
    y      = med_logPSD,
    colour = factor(Operational)             # Factor to ensure proper legend labels
  )) +
  geom_line(size = 0.8, alpha = 0.9) +        # Clear lines with slight transparency
  scale_colour_manual(
    values = cols,
    labels = c("Background", "Operational"), # Human-readable legend entries
    name   = ""                              # No title for a cleaner legend box
  ) +
  coord_cartesian(xlim = c(0, 25)) +          # Focus on the 0-25 Hz band
  labs(
    title = "Median log-PSD Spectrum (Weighted)",
    x     = "Frequency (Hz)",
    y     = expression("Median log-PSD (dB, m"^2*"/Hz)")  # Math notation for units
  ) +
  theme_minimal(base_size = 12) +             # Minimal theme for publication quality
  theme(
    legend.position   = "right",              # Legend on the right for space efficiency
    panel.grid.minor  = element_blank(),      # Remove minor grid lines
    panel.grid.major  = element_line(color = "grey80"), # Light major grid lines
    plot.title        = element_text(face = "bold", hjust = 0.5) # Bold, centered title
  )
```

```r
# 5. Render the PSD spectrum figure
print(p_spec)
```




```r
##############################################################################
# Median RMS by Hour of Day
# Purpose: Examine diurnal variation in RMS displacement, comparing
#          Background vs Operational states to check residual time-of-day structure.
##############################################################################

# Load required libraries for data manipulation and plotting
library(dplyr)   # Data wrangling functions: mutate, group_by, summarise
library(ggplot2) # Plotting: ggplot, geom_line, geom_point, etc.

# 1. Prepare the summary data frame 'hour_df'
#    - Compute RMS displacement (nm) from integrated energy E (nm²)
#    - Create a descriptive factor 'Type' for turbine state
#    - Group by turbine state and hour of day, then compute median RMS
hour_df <- energy_df %>%
  mutate(
    RMS = sqrt(E),  # Convert energy variance (nm²) to RMS displacement (nm)
    Type = factor(
      Operational,
      levels = c(0, 1),
      labels = c("Background", "Operational")
    )  # Convert 0/1 flag into human-readable levels
```

```r
  ) %>%
  group_by(Type, HourOfDay) %>%
  summarize(
    med_RMS = median(RMS, na.rm = TRUE),  # Median reduces influence of outliers
    .groups = "drop"                       # Ungroup after summarisation
  )

# 2. Define custom colors for plotting by turbine state
cols <- c(
  "Background"  = "#E69F00",
  "Operational" = "#0072B2"
)

# 3. Create the line + point plot of median RMS vs HourOfDay
#    - Lines connect hourly medians; points mark exact values
#    - Custom colors, axis breaks, and theme for clarity
ggplot(hour_df, aes(x = HourOfDay, y = med_RMS, color = Type)) +
  geom_line(size = 1) +                           # Draw connecting lines
  geom_point(size = 2) +                          # Highlight each median point
  scale_color_manual(values = cols, name = "State") +  # Apply custom palette
  scale_x_continuous(breaks = seq(0, 23, by = 4)) +    # Tick every 4 hours
  labs(
    title = "Median RMS Displacement by Hour of Day",
    x     = "Hour (0-23 UTC)",
    y     = "Median RMS (nm)"
  ) +
  theme_minimal() +                                 # Clean background
  theme(
    legend.position = "bottom",                   # Place legend below plot
    plot.title       = element_text(face = "bold", hjust = 0.5)  # Center bold title
  )
```

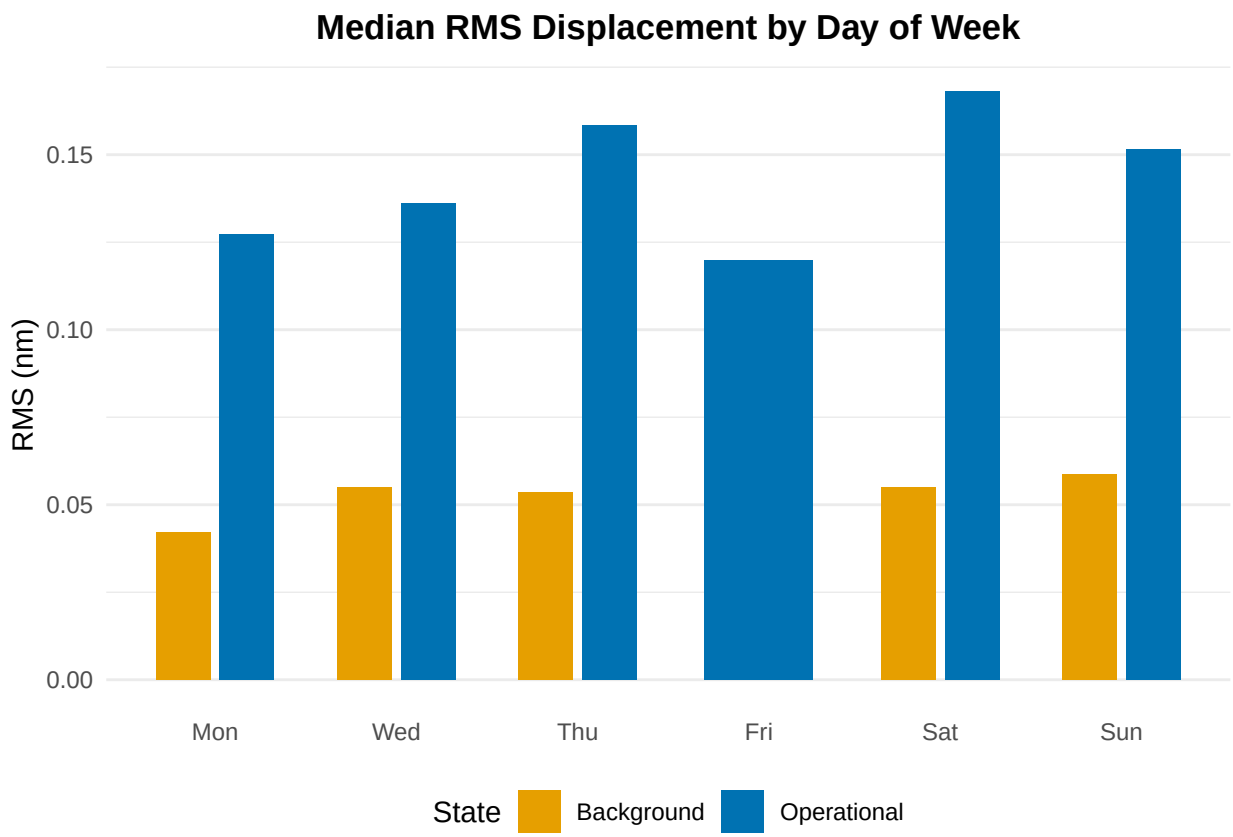



```
##############################################################################
# Median RMS by Day of Week
# Purpose: Assess how RMS displacement varies across weekdays for Background
#          vs Operational turbine states, checking for systematic weekly cycles.
#############################################################################

# 1. Load required libraries (skip if already loaded earlier in the script)
library(dplyr)     # Data manipulation: mutate, group_by, summarize
library(ggplot2)   # Plotting: ggplot, geom_col, etc.
library(lubridate) # Date-time utilities: wday to extract weekday factor

# 2. Prepare summary data frame 'day_df'
day_df <- energy_df %>%
  mutate(
    # Convert integrated energy (E in nm²) to RMS displacement (nm)
    RMS = sqrt(E),
    # Create descriptive factor for turbine state instead of numeric codes
    Type = factor(
      Operational,
      levels = c(0, 1),
      labels = c("Background", "Operational")
    ),
    # Extract weekday from 'Day' factor: Mon-Sun abbreviated, week starts on Monday
    Weekday = wday(Day, label = TRUE, abbr = TRUE, week_start = 1)
  ) %>%
  # Group by turbine state and weekday to compute central tendency
```

```r
  group_by(Type, Weekday) %>%
  summarise(
    # Median is robust to outliers, summarizing typical RMS per day
    med_RMS = median(RMS, na.rm = TRUE),
    .groups = "drop"      # Remove grouping after summarisation for downstream operations
  )
```

```
## Warning: There was 1 warning in `mutate()`.
## i In argument: `Weekday = wday(Day, label = TRUE, abbr = TRUE, week_start =
##   1)`.
## Caused by warning:
## ! tz(): Don't know how to compute timezone for object of class factor; returning "UTC".
```

```r
# 3. Define custom colors for the two turbine states
cols <- c(
  "Background"  = "#E69F00",
  "Operational" = "#0072B2"
)

# 4. Create bar chart of median RMS by weekday and state
ggplot(day_df, aes(x = Weekday, y = med_RMS, fill = Type)) +
  geom_col(
    position = position_dodge(width = 0.7),  # Side-by-side bars for Background vs Operational
    width    = 0.6                           # Bar width covers 60% of bin width
  ) +
  scale_fill_manual(
    values = cols,    # Apply custom color mapping
    name   = "State"  # Legend title for clarity
  ) +
  labs(
    title = "Median RMS Displacement by Day of Week",
    x     = NULL,      # No x-axis title; weekday labels are self-explanatory
    y     = "RMS (nm)" # Y-axis title with units
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",             # Place legend below the plot
    panel.grid.major.x = element_blank(),   # Remove vertical grid lines for cleaner look
    plot.title = element_text(face = "bold", hjust = 0.5)  # Center and bold the title
  )
```




```
##############################################################################
# Statistical Modelling Phase - Simple GAM for Operational Effect
# Purpose: Quantify the shift in log10-integrated seismic energy when the
#          turbine is ON vs OFF, before adding smooth terms for covariates.
##############################################################################

# Center and scale wind speed to have mean = 0 and SD = 1
# This standardization ensures the smooth term for wind speed (when added later)
# uses a dimensionless scale, improving numerical stability and interpretability.
energy_df <- energy_df %>%
  mutate(
    ws_cs = as.numeric(scale(Wind.speed))  # Standardize wind speed
  )

# Determine the number of unique values in each predictor to set knot limits
# Having too many knots risks overfitting; too few risks underfitting.
n_ws   <- n_distinct(energy_df$ws_cs)     # Unique scaled wind-speed levels
n_dir  <- n_distinct(energy_df$WindDir)   # Unique wind-direction angles
n_hour <- n_distinct(energy_df$HourOfDay) # Unique hours of day

# Choose basis dimension (k) for future smooth terms:
# - Up to 10 for wind speed and hour-of-day
# - Up to 12 for wind direction (cyclic)
k_ws   <- min(10, n_ws)
k_dir  <- min(12, n_dir)
k_hour <- min(10, n_hour)
```

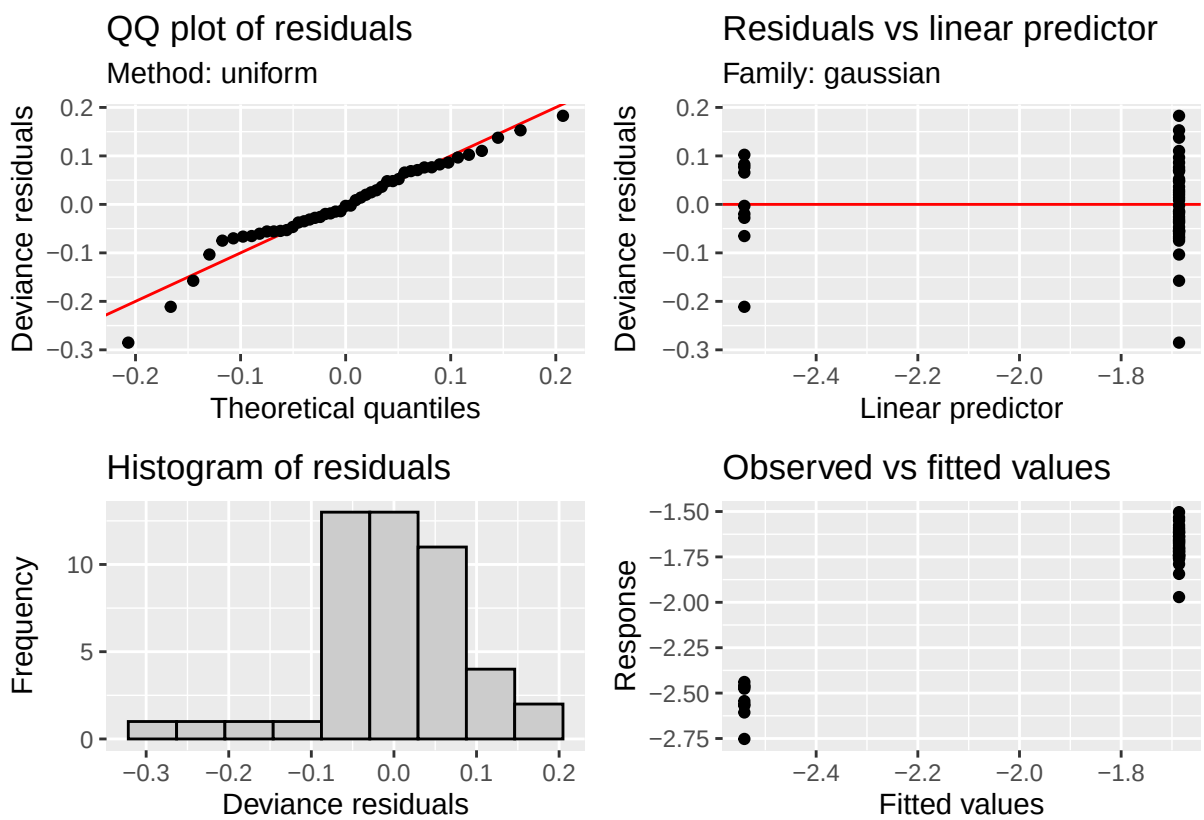
```r
# Model 1: Fit a baseline GAM with only turbine Operational status
# logE ~ Operational tests the main hypothesis H : no shift in mean log-energy
# when the turbine is turned ON, without adjusting for other covariates.
gam_op <- gam(
  logE ~ Operational,    # Predictor: 0 = Background, 1 = Operational
  data   = energy_df,
  method = "REML"        # Use restricted maximum likelihood for smoothness selection
)

# Summarize the fitted model:
# - Parametric coefficients: estimate the difference in logE for Operational vs Background
# - R-squared and deviance explained: gauge model fit quality
summary(gam_op)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## logE ~ Operational
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.54126    0.02994  -84.88   <2e-16 ***
## Operational1  0.85515    0.03330   25.68   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## R-sq.(adj) =  0.935   Deviance explained = 93.6%
## -REML = -41.678  Scale est. = 0.0080677  n = 47
```

```r
# Visualize model components:
# - draw() shows any smooth terms (none here) and the intercept shift
# - Useful for checking if the operational effect is estimated correctly
draw(gam_op)
```

```
## Unable to draw any of the model terms.
```

```r
# Perform residual diagnostics:
# - appraise() runs standard checks (QQ-plot, residuals vs fitted, etc.)
# - Ensures model assumptions (normality, homoscedasticity) hold
appraise(gam_op)
```

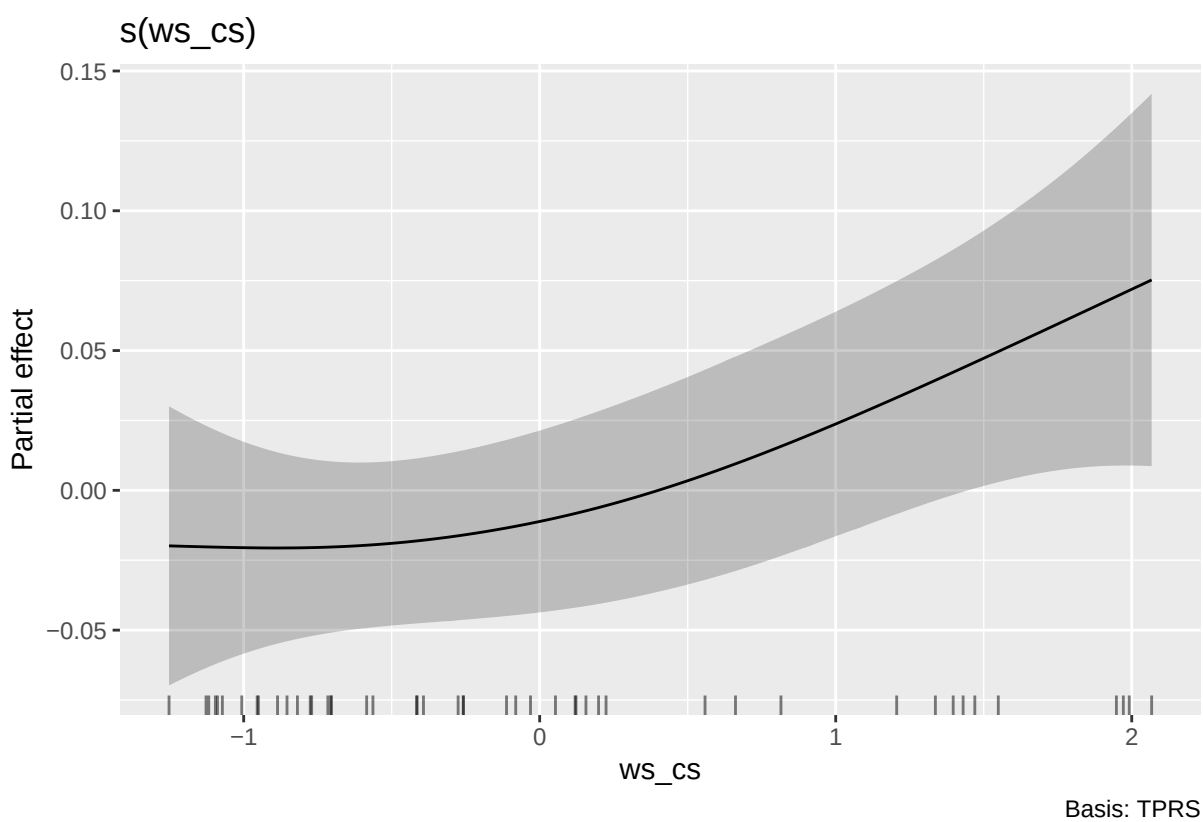



```
##############################################################################
# Model 2: GAM with Wind-Speed Smooth and Operational Status
# Purpose: Model log -transformed seismic energy as a nonlinear function of
#          wind speed while accounting for turbine ON/OFF state.
# Rationale: The smooth term s(ws_cs) captures the continuous effect of wind
#            speed (hypothesis H ), and the parametric Operational term tests
#            the shift in mean log-energy when the turbine is ON (H ).
##############################################################################

# 1. Fit the GAM: include a smooth for standardized wind speed and a parametric
#    shift for turbine operational status. Use REML for reliable smoothness selection.
gam_wsop <- gam(
  logE ~
    s(ws_cs, k = k_ws) +     # Nonparametric spline of wind speed with k_ws basis functions
    Operational,             # Parametric effect: 0 = Background, 1 = Operational
  data   = energy_df,
  method = "REML"            # Restricted Maximum Likelihood for smoothing parameter
)

# 2. Summarize the fitted model:
#    - Check significance of the wind-speed smooth (edf and F-statistic)
#    - Evaluate the parametric Operational coefficient (t-value, p-value)
#    - Inspect overall fit: adjusted R² and percentage deviance explained
summary(gam_wsop)
```

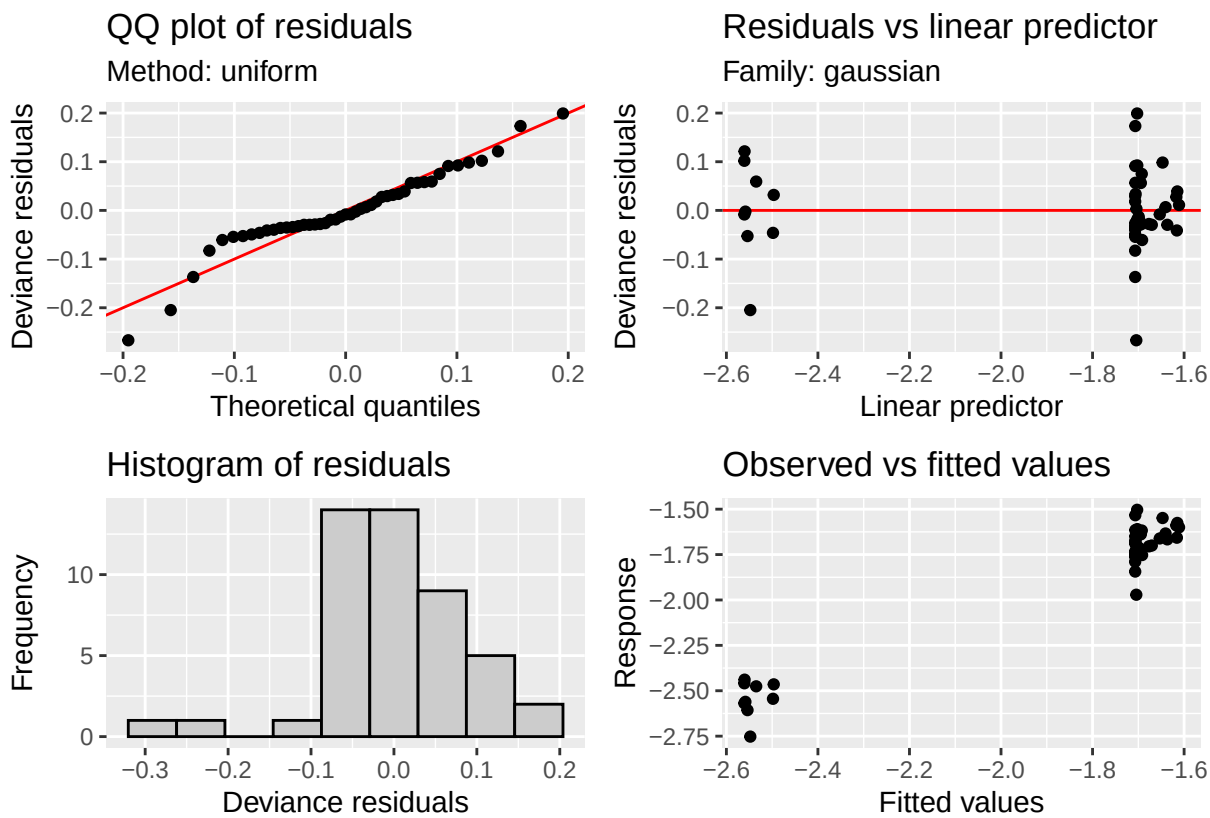
```
##
```

```
## Family: gaussian
## Link function: identity
##
## Formula:
## logE ~ s(ws_cs, k = k_ws) + Operational
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.54041    0.02834  -89.65   <2e-16 ***
## Operational1  0.85410    0.03154   27.08   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##            edf Ref.df     F p-value
## s(ws_cs) 1.622  2.002 3.218  0.0501 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.942   Deviance explained = 94.5%
## -REML = -40.897  Scale est. = 0.007189  n = 47
```

```r
# 3. Visualise model components:
#    - draw() plots the estimated smooth function of wind speed with confidence bands
draw(gam_wsop)
```



```r
# 4. Perform residual diagnostics:
#    - appraise() provides QQ-plot, residuals vs fitted, and other checks
#    - Confirms assumptions of normality, homogeneity, and lack of pattern in residuals
appraise(gam_wsop)
```

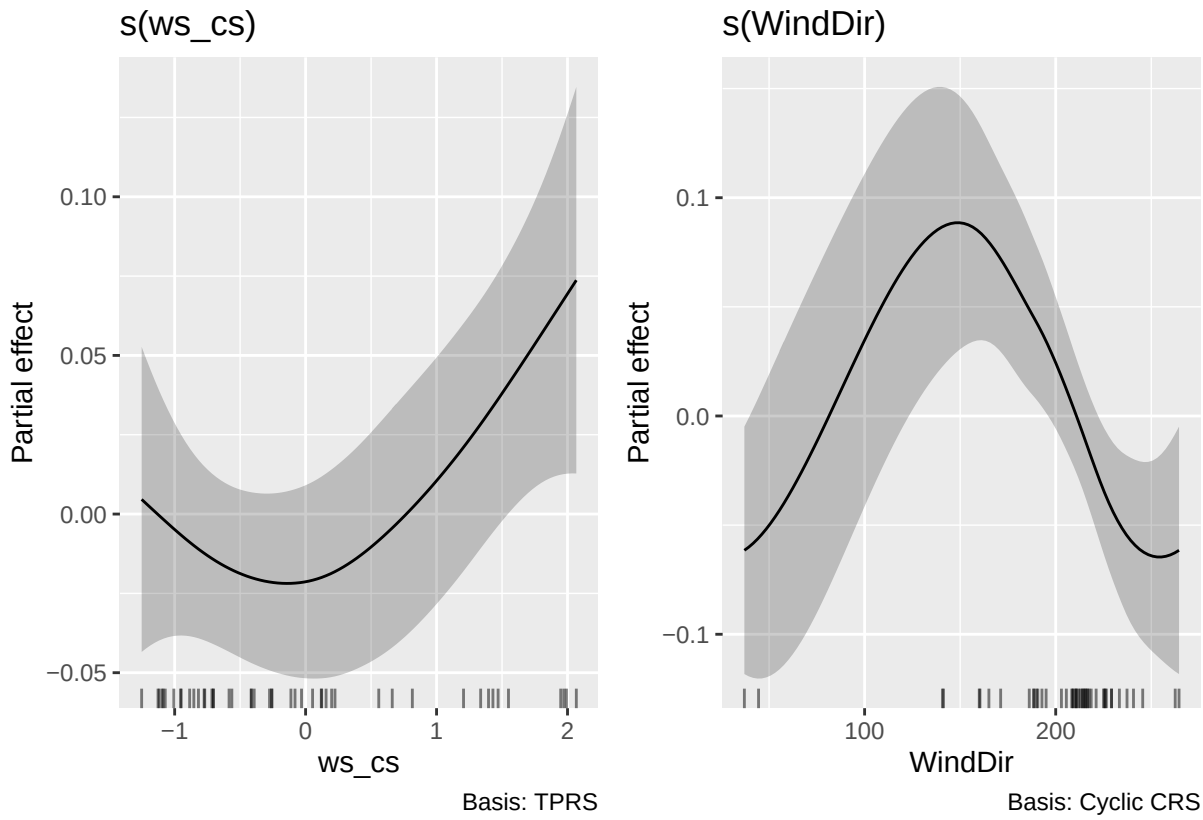



```r
##############################################################################
# Model 3: GAM with Wind Speed & Cyclic Wind Direction Smooths
# Purpose: Extend the GAM to capture nonlinear effects of wind direction
#          (with continuity at 0°/360°) alongside wind speed, and assess
#          the remaining operational shift in seismic energy.
# Rationale: Tests H  that wind direction influences seismic energy, and
#            ensures the model respects the circular nature of directional data.
#############################################################################

# Model 3: GAM adding cyclic smooth for wind direction
gam_wop <- gam(
  logE ~
    s(ws_cs,    k = k_ws)       +  # Smooth term for centered & scaled wind speed
    s(WindDir,  bs = "cc", k = k_dir) +  # Cyclic smooth for wind direction (0° = 360° continuity)
    Operational,                    # Parametric effect of turbine ON/OFF status
  data   = energy_df,              # Use the preprocessed energy data frame
  method = "REML"                  # REML estimation for smoothing parameter selection
)

# 1. Model summary: term significance, estimated degrees of freedom,
```

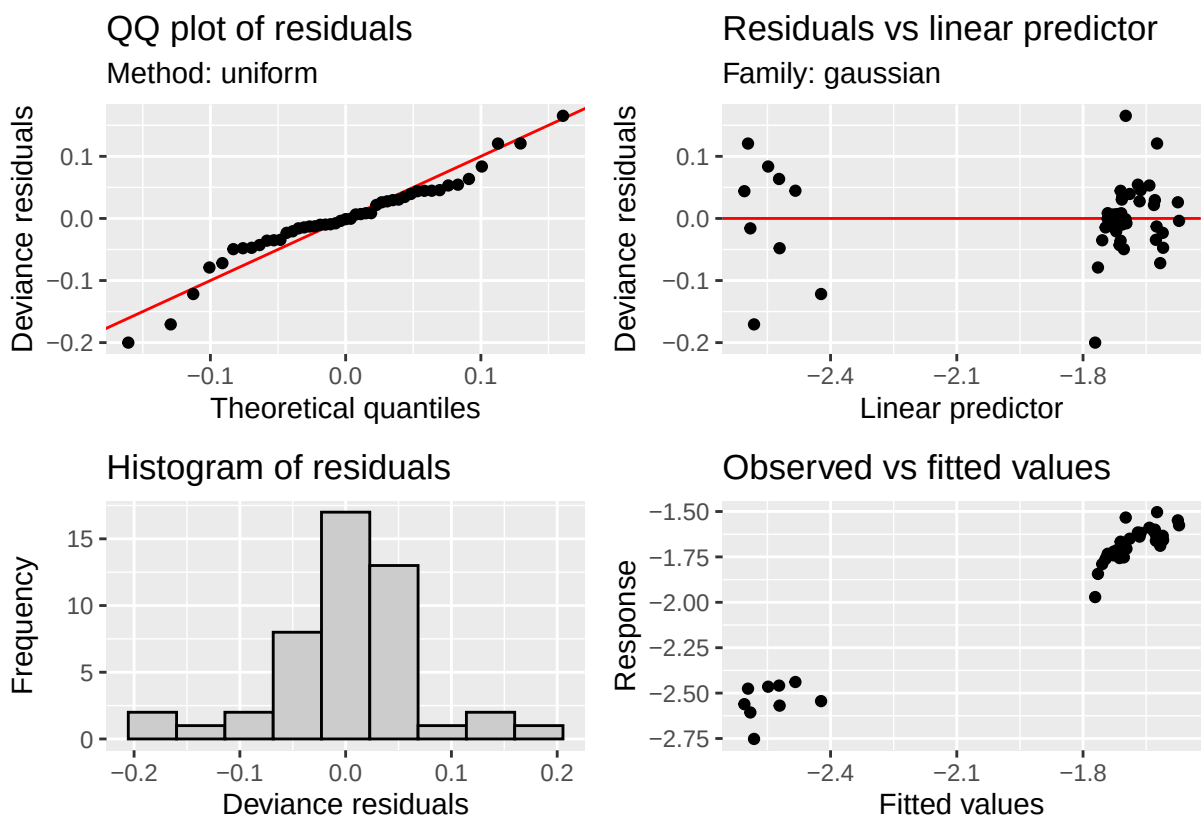
```r
#    adjusted R², and deviance explained
summary(gam_wop)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## logE ~ s(ws_cs, k = k_ws) + s(WindDir, bs = "cc", k = k_dir) +
##     Operational
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.52632    0.02400 -105.28   <2e-16 ***
## Operational1  0.83668    0.02688   31.13   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df     F  p-value
## s(ws_cs)   2.093  2.585 3.165 0.047058 *
## s(WindDir) 2.265 10.000 1.994 0.000142 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.961   Deviance explained = 96.5%
## -REML = -46.661  Scale est. = 0.0048699  n = 47
```

```r
# 2. Visualize the smooth terms:
#    draw() displays the estimated spline for ws_cs and the cyclic spline for WindDir,
#    including approximate confidence intervals to assess uncertainty.
draw(gam_wop)
```



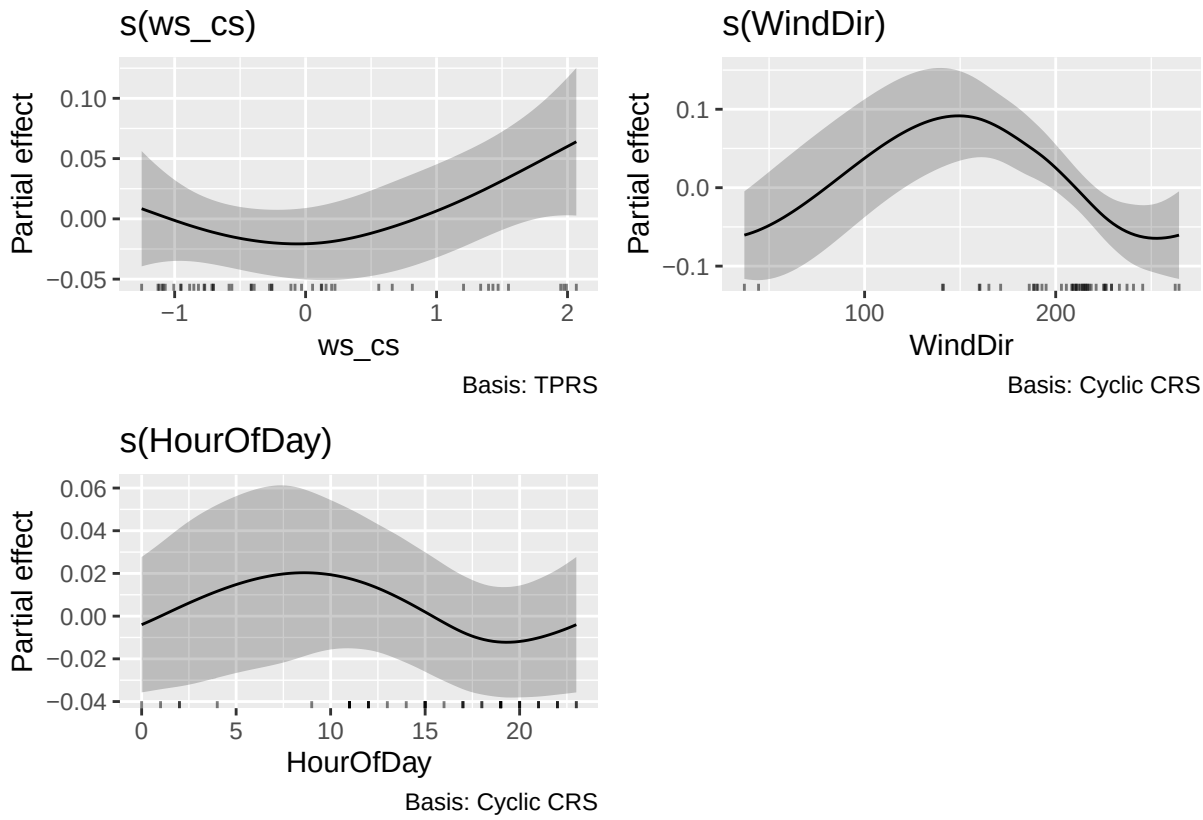

```
# 3. Residual diagnostics:
#    appraise() generates standard GAM diagnostic plots (QQ-plot, residuals vs fitted, etc.)
#    to check normality, homoscedasticity, and any remaining patterns in residuals.
appraise(gam_wop)
```



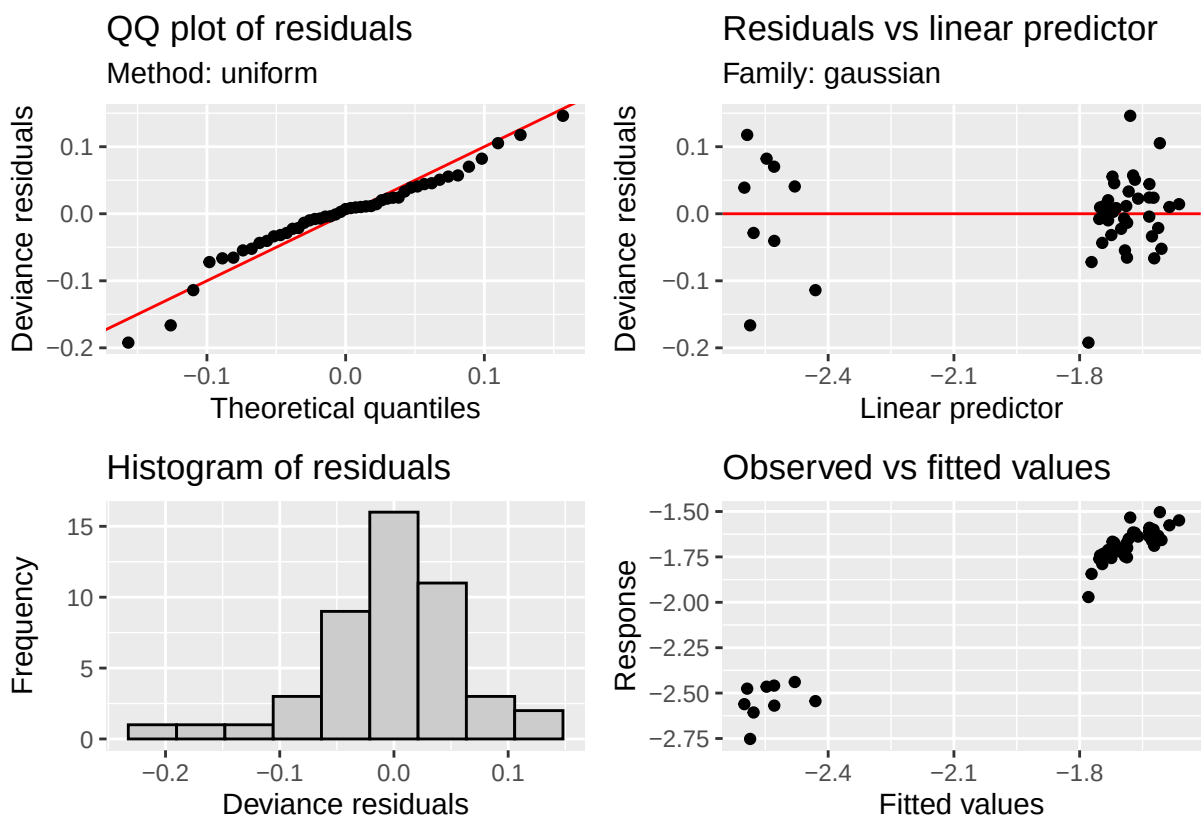

```
##############################################################################
# Model 4: GAM with Wind Speed, Wind Direction, and Diurnal Smooths
# Purpose: Fit a comprehensive GAM that captures nonlinear effects of wind speed,
#          cyclic effects of wind direction and hour of day, and quantifies the
#          remaining shift in log10-energy due to turbine operational status.
# Rationale: This model tests hypotheses H  (wind-speed), H  (wind-direction), and
#            H  (diurnal cycle) simultaneously, while also estimating H
#            (operational shift) in a single unified framework.
#############################################################################

# Model 4: GAM with smooth terms for wind speed, cyclic wind direction, cyclic hour of day, plus turbin
gam_whop <- gam(
  logE ~ s(ws_cs, k = k_ws) +              # Smooth for centered & scaled wind speed
    s(WindDir, bs = "cc", k = k_dir) +    # Cyclic smooth for wind direction (0° = 360° continuity)
    s(HourOfDay, bs = "cc", k = k_hour) + # Cyclic smooth for diurnal (hour of day) pattern
    Operational,                           # Parametric turbine ON/OFF effect
  data   = energy_df,
  method = "REML"                          # REML smoothing parameter estimation
)

# Summarize the fitted model:
# - Parametric coefficient for Operational shows the mean log-energy shift when ON vs OFF
# - Each smooth term's estimated degrees of freedom (edf) indicates complexity
# - Adjusted R² and deviance explained assess overall fit quality
summary(gam_whop)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## logE ~ s(ws_cs, k = k_ws) + s(WindDir, bs = "cc", k = k_dir) +
##     s(HourOfDay, bs = "cc", k = k_hour) + Operational
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.52450    0.02376 -106.24   <2e-16 ***
## Operational1  0.83442    0.02670   31.25   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##                edf Ref.df     F  p-value
## s(ws_cs)     2.077  2.562 2.512   0.0906 .
## s(WindDir)   2.284 10.000 2.195 7.16e-05 ***
## s(HourOfDay) 1.081  8.000 0.268   0.1301
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.963   Deviance explained = 96.8%
## -REML = -46.972  Scale est. = 0.0046288  n = 47
```

```r
# Visualize the estimated smooth functions and intercept:
# - draw() plots s(ws_cs), s(WindDir), and s(HourOfDay) with approximate CIs
# - Helps interpret the shape and uncertainty of each smooth effect
draw(gam_whop)
```



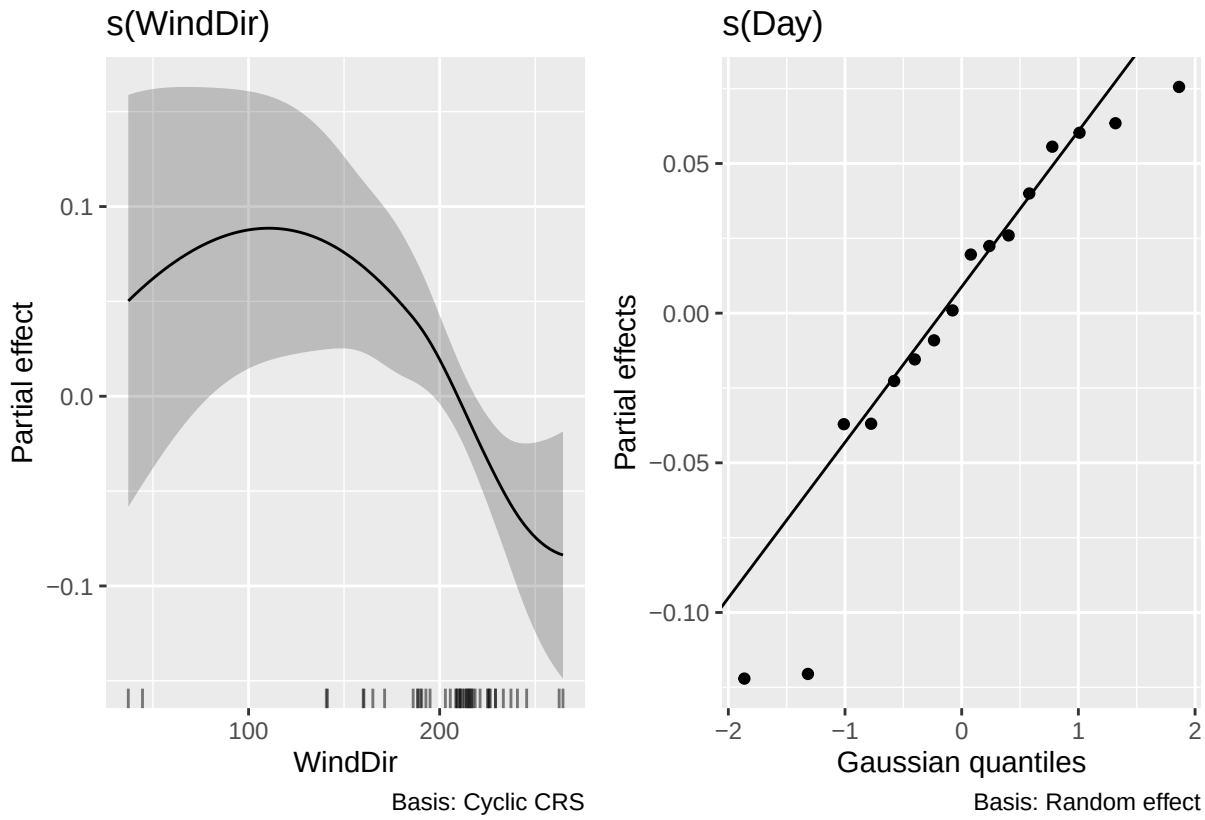

```r
# Perform residual diagnostics:
# - appraise() checks QQ-plot, residuals vs fitted, and other diagnostic plots
# - Ensures no remaining structure or violation of modelling assumptions
appraise(gam_whop)
```



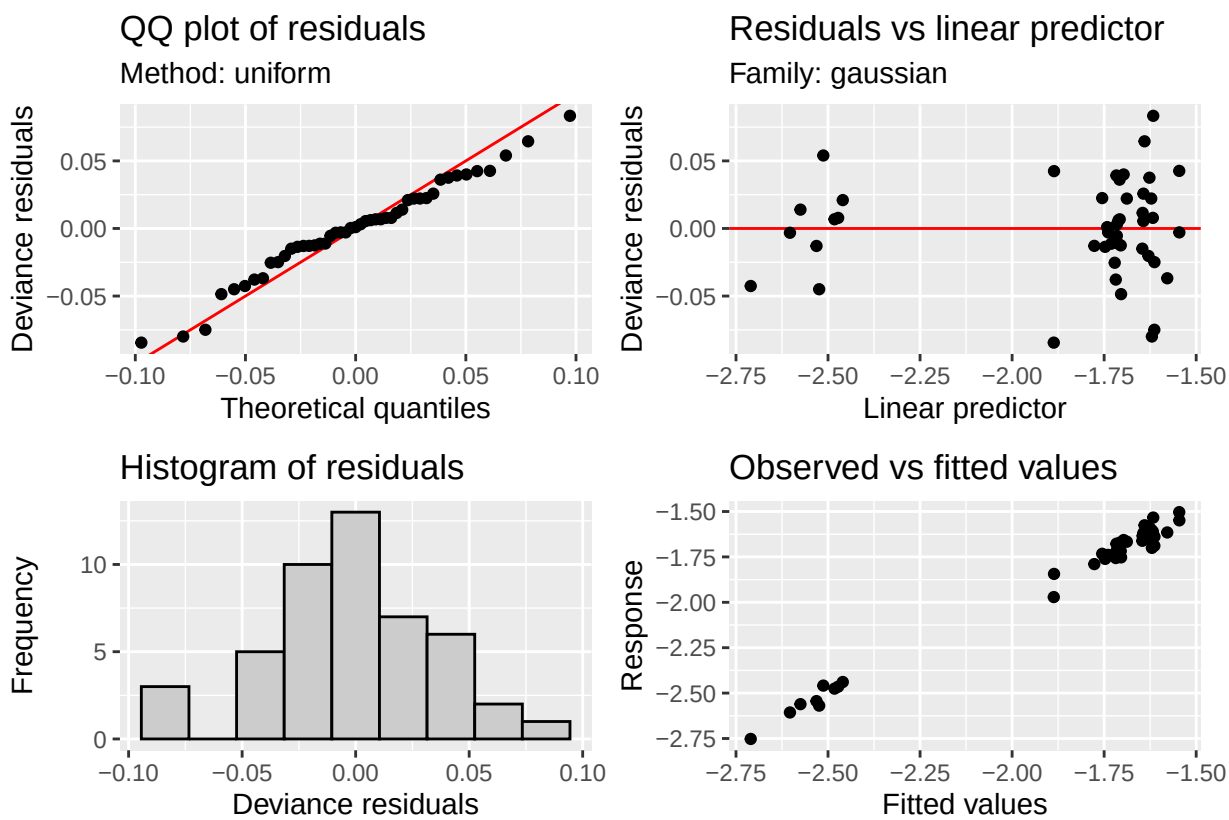

```
##############################################################################
# Model 5: Simplified GAM - Wind Direction, Day Random Effect, and Operational
# Purpose: Fit a reduced GAM excluding wind-speed smooth to test if wind direction,
#          day-to-day variation, and turbine operational status alone capture
#          the key patterns in log -energy.
# Rationale: Compare via AIC to assess parsimony vs. fit (balances complexity and explanatory power).
##############################################################################

# Model 5: Fit a simplified GAM with cyclic wind-direction smooth,
#          random intercept for Day, and Operational status
gam_simpler <- gam(
  logE ~
    s(WindDir, bs = "cc", k = k_dir) +  # Cyclic smooth for wind direction (ensures continuity at 0°=36
    s(Day,       bs = "re") +           # Random intercept per day to capture unobserved daily effects
    Operational,                        # Parametric effect of turbine ON/OFF (tests H )
  data   = energy_df,
  method = "REML",                     # REML for smoothing parameter estimation
  knots  = list(WindDir = c(0, 360))   # Specify knots for cyclic smooth boundary
)


# Summarize the simplified model:
# - Parametric coefficient for Operational indicates logE shift when turbine is ON
# - Smooth term edf and F-statistic show wind-direction effect significance
# - Adjusted R² and deviance explained gauge overall model fit
summary(gam_simpler)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## logE ~ s(WindDir, bs = "cc", k = k_dir) + s(Day, bs = "re") +
##     Operational
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.55223    0.03116  -81.91   <2e-16 ***
## Operational1  0.86969    0.04185   20.78   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##               edf Ref.df      F  p-value
## s(WindDir)  2.355     10 53.480 0.000597 ***
## s(Day)     10.290     14  4.202  1.6e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.986   Deviance explained =   99%
## -REML = -60.673  Scale est. = 0.0017833  n = 47
```

```r
# Visualize smooth and parametric components for interpretation
draw(gam_simpler)
```




```
# Perform residual diagnostics to check model assumptions:
# - QQ-plot for normality
# - Residuals vs fitted for homoscedasticity
# - Other standard GAM checks
appraise(gam_simpler)
```

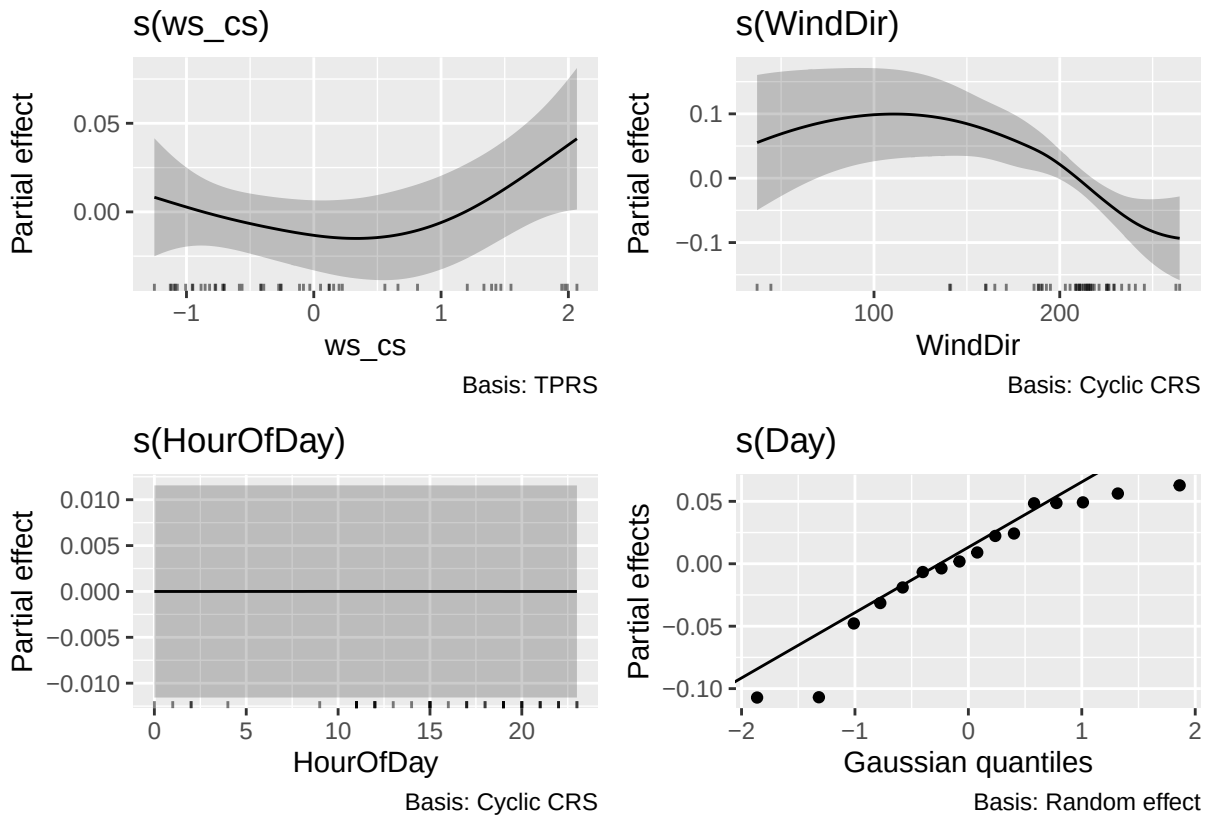



```
##############################################################################
# Model 6: Full GAM with All Smooths, Random Day Effects, and Operational
# Purpose: Fit the most comprehensive GAM capturing wind-speed, wind-direction,
#          diurnal cycle, day-to-day variability, and turbine operational status.
# Rationale: Tests all hypotheses (H -H ) in a single model, providing the final
#             quantitative assessment of operational noise impacts on seismic energy.
##############################################################################

# Model 6: Fit the full GAM
gam_energy <- gam(
  logE ~
    s(ws_cs,       k = k_ws)        +  # Smooth for centered & scaled wind speed (nonlinear effect)
    s(WindDir,     bs = "cc", k = k_dir) +  # Cyclic smooth for wind direction (0° and 360° continuous)
    s(HourOfDay,   bs = "cc", k = k_hour) + # Cyclic smooth for hour of day (captures diurnal pattern)
    s(Day,         bs = "re")       +  # Random intercept per day to account for daily shifts
    Operational,                         # Parametric shift for turbine ON vs OFF status
  data   = energy_df,                   # Use preprocessed energy data
  method = "REML",                      # Restricted Maximum Likelihood for smoothing parameter estimation
  knots  = list(                        # Ensure cyclic continuity at the boundaries
    WindDir   = c(0, 360),
    HourOfDay = c(0, 24)
  )
)

# 1. Model summary
#    - Parametric coefficient (Operational) tests H : shift in log-energy when ON
```

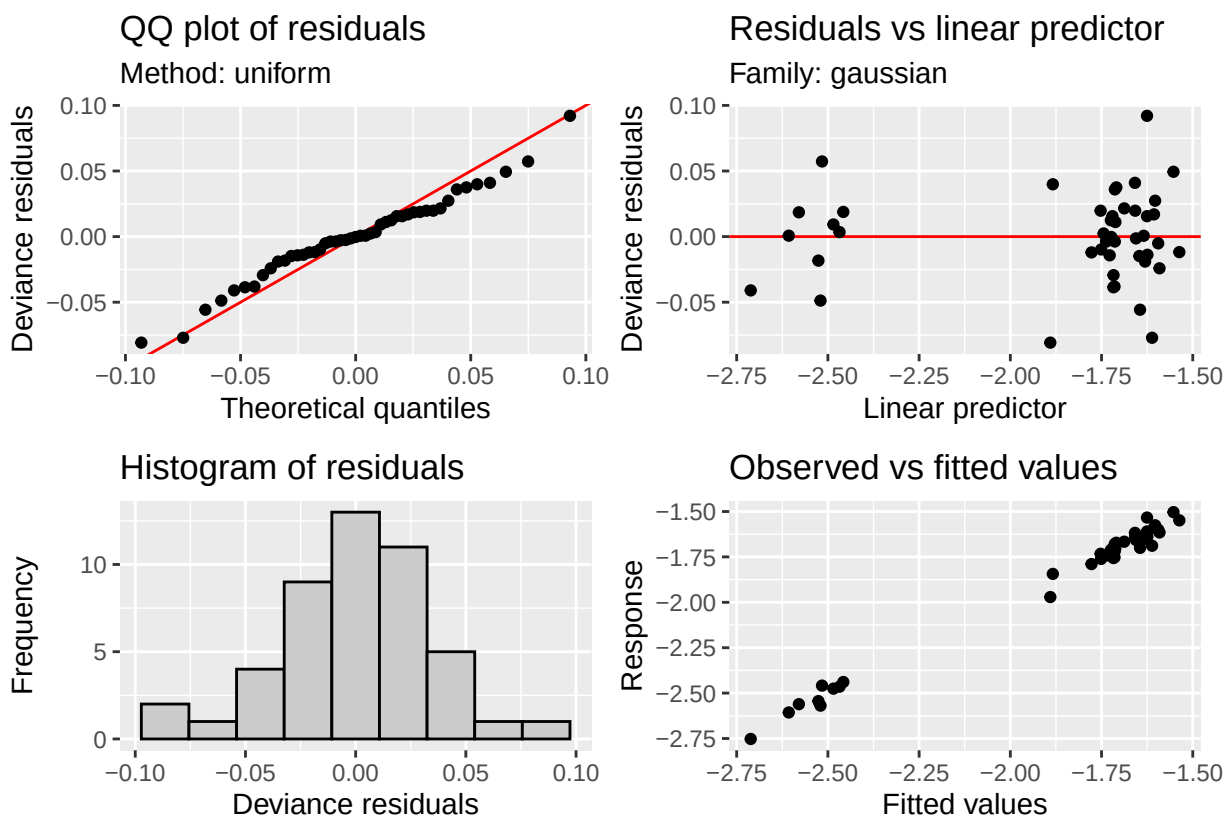
```r
#    - edf for each smooth term indicates complexity of the estimated relationship
#    - Adjusted R² and deviance explained assess overall fit quality
summary(gam_energy)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## logE ~ s(ws_cs, k = k_ws) + s(WindDir, bs = "cc", k = k_dir) +
##     s(HourOfDay, bs = "cc", k = k_hour) + s(Day, bs = "re") +
##     Operational
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.55036    0.02899  -87.98   <2e-16 ***
## Operational1  0.86814    0.03888   22.33   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##                    edf Ref.df      F  p-value
## s(ws_cs)     2.1761078  2.641  1.595 0.149954
## s(WindDir)   2.5116211 10.000 31.679 0.000465 ***
## s(HourOfDay) 0.0003332  8.000  0.000 0.422436
## s(Day)       9.8364681 14.000  3.365 0.000102 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.987   Deviance explained = 99.1%
## -REML = -58.375  Scale est. = 0.0016342  n = 47
```

```r
# 2. Visualise smooth terms and intercept
#    - draw() plots each s(...) term with approximate confidence intervals
#    - Helps interpret shape and uncertainty of each effect
draw(gam_energy)
```



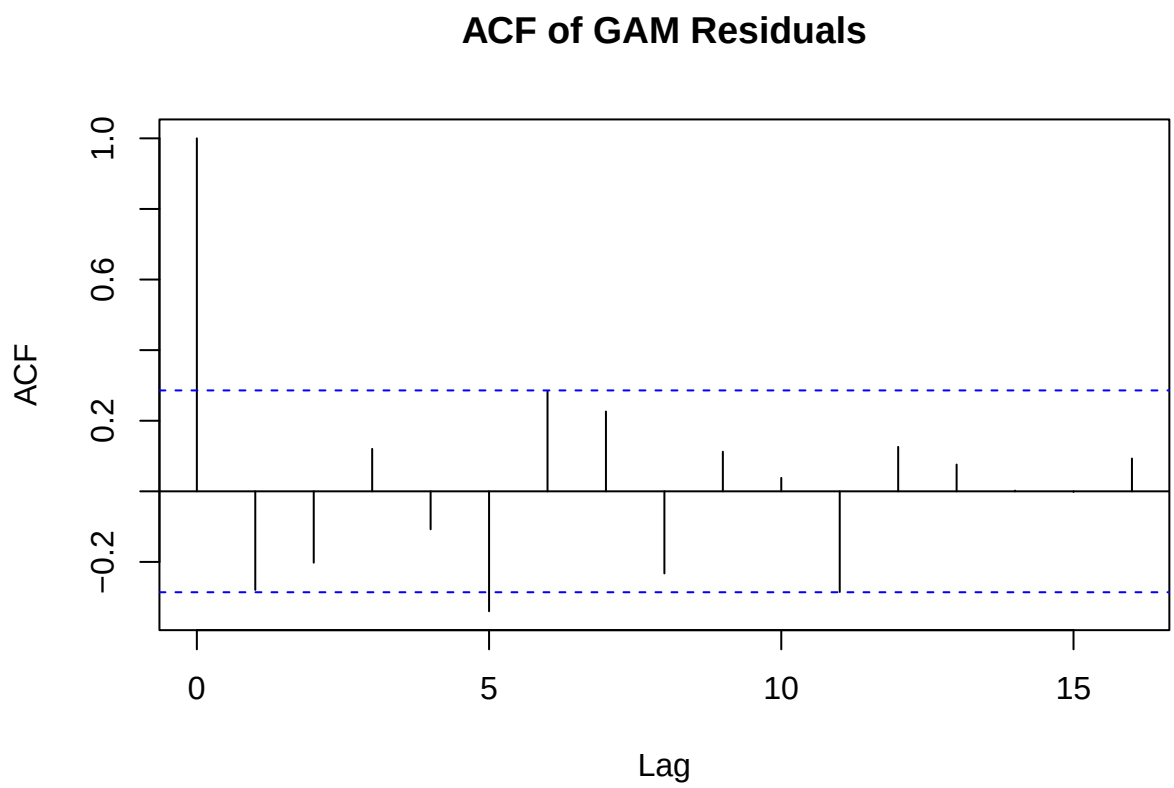

```r
# 3. Residual diagnostics
#    - appraise() provides QQ-plot, residuals vs fitted, and other checks
#    - Ensures no violation of normality, homoscedasticity, or remaining structure
appraise(gam_energy)
```




```r
# 4. Model comparison via AIC
#    - Compare across simpler and fuller models to judge parsimony vs. fit
AIC(gam_op, gam_wsop, gam_wop, gam_whop, gam_simpler, gam_energy)
```

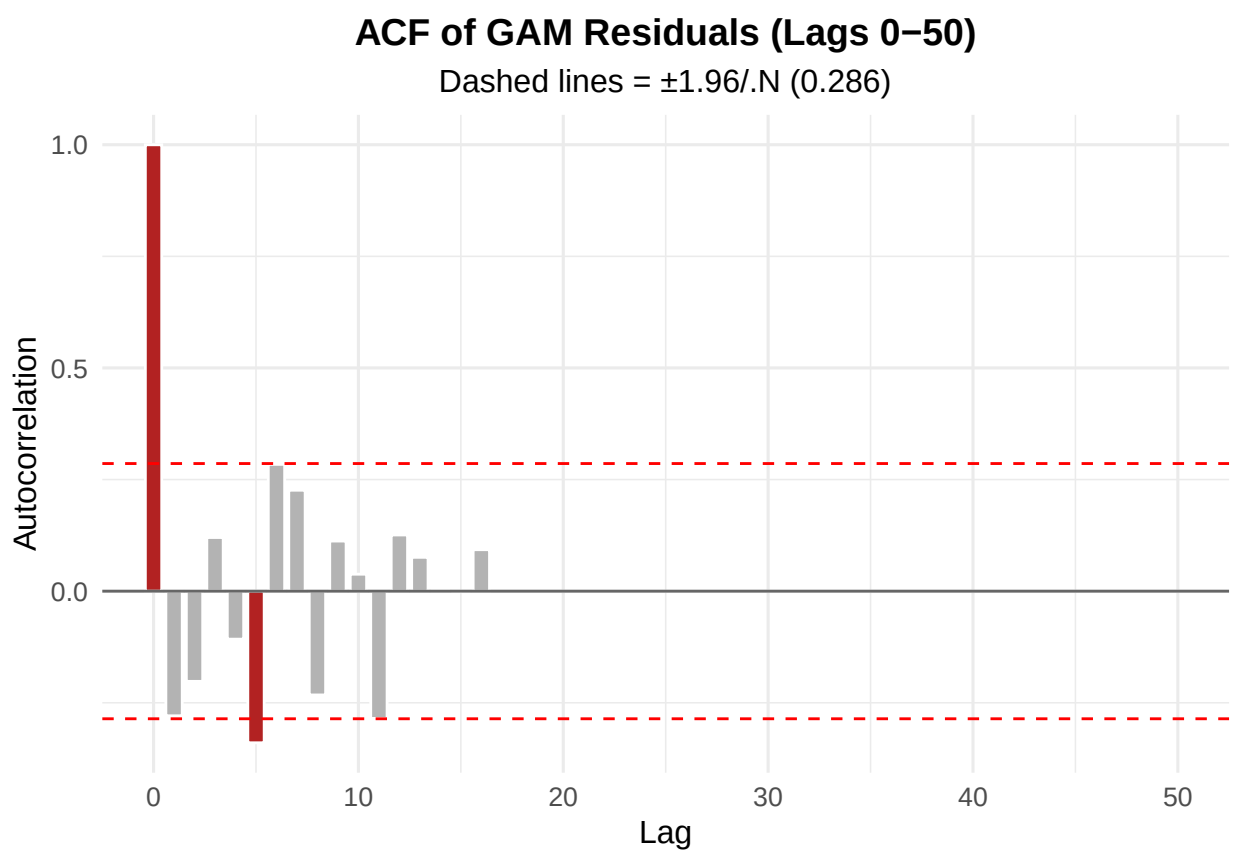
```
##                    df        AIC
## gam_op       3.000000  -89.19834
## gam_wsop     5.002343  -92.33859
## gam_wop      8.488180 -106.73493
## gam_whop    10.117240 -107.13338
## gam_simpler 16.952264 -147.74094
## gam_energy  19.653625 -149.25578
```

```r
# 5. Autocorrelation of residuals
#    - ACF plot checks for temporal dependence left in residuals (should be minimal)
acf(residuals(gam_energy), main = "ACF of GAM Residuals")
```



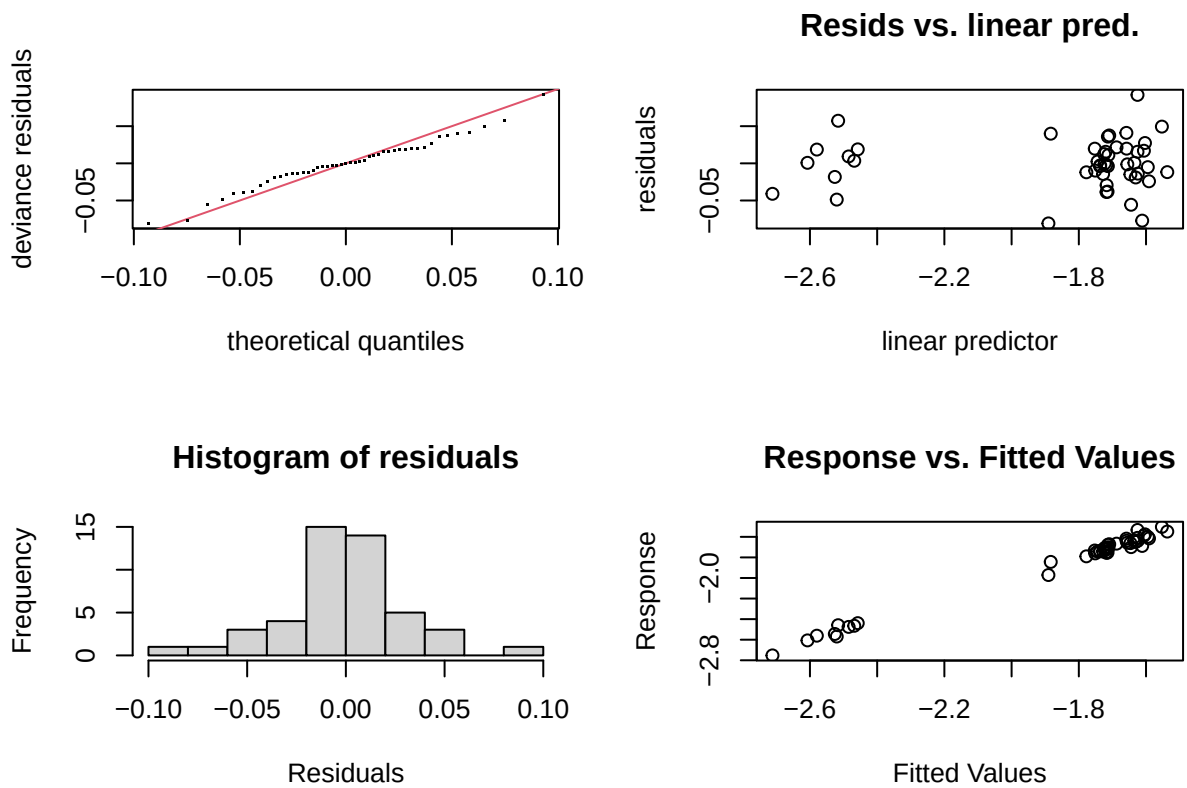

```
#
# Extended ACF: increase lag range to 50 time units
# Purpose: Verify residual autocorrelation remains low across multiple lags.
# Insight: Significant autocorrelation would indicate missing temporal structure.
#
acf_out <- acf(residuals(gam_energy), plot = FALSE)

# 2. Convert to a data frame for plotting
#    lag: the lag number
#    acf: the autocorrelation at that lag
#    bound: whether the acf exceeds the 95% confidence interval
N      <- length(residuals(gam_energy))         # Number of residuals
sig_lim <- qnorm((1 + 0.95) / 2) / sqrt(N)      # 95% bounds = ±1.96/√N

acf_df <- with(acf_out, data.frame(
  lag    = lag[,1,1],
  acf    = acf[,1,1]
)) %>%
  mutate(
    significant = abs(acf) > sig_lim           # Flag significant lags
  )

# 3. Plot using ggplot2
ggplot(acf_df, aes(x = lag, y = acf, fill = significant)) +
  geom_col(width = 0.8, color = "white") +     # Vertical bars for each lag
  geom_hline(yintercept = 0, colour = "grey40") +
```

```r
  geom_hline(yintercept =  sig_lim, linetype = "dashed", colour = "red") +
  geom_hline(yintercept = -sig_lim, linetype = "dashed", colour = "red") +
  scale_fill_manual(
    values = c(`FALSE` = "grey70", `TRUE` = "firebrick"),
    guide  = "none"
  ) +
  coord_cartesian(xlim = c(0, 50)) +            # Show lags 0-50
  labs(
    title = "ACF of GAM Residuals (Lags 0-50)",
    subtitle = sprintf("Dashed lines = ±1.96/√N (%.3f)", sig_lim),
    x = "Lag",
    y = "Autocorrelation"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title    = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5)
)
```




```r
##############################################################################
# GAM Residual and Smooth Diagnostics
# Purpose: Run standard diagnostic checks on the full GAM (gam_energy) to assess
#          model fit, identify any violations of assumptions, and visualize
#          how well the smooth terms capture the data.
##############################################################################
```

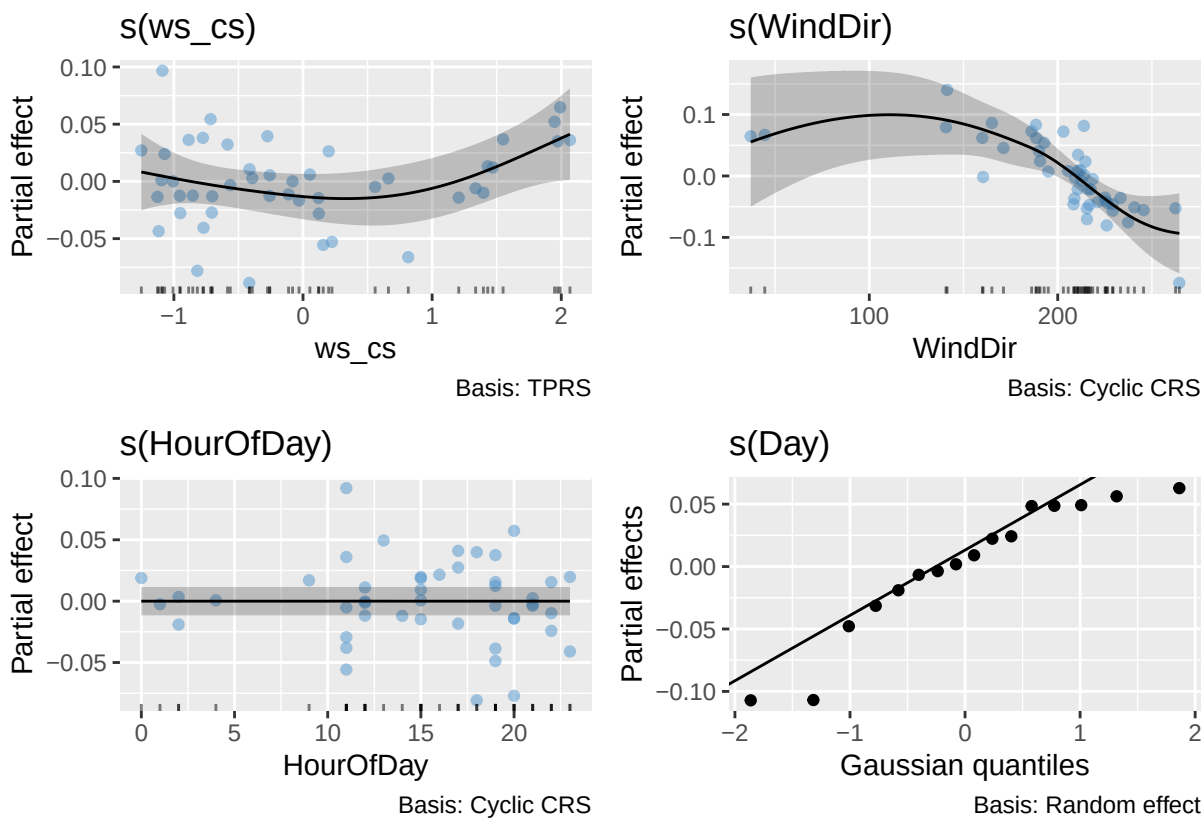
```r
# 1) Base-R GAM diagnostic checks
#    - gam.check() produces:
#        • QQ-plot of normalized residuals vs theoretical normal quantiles
#        • Residuals vs fitted values to reveal heteroscedasticity or patterns
#        • Histogram of residuals to assess normality
#        • Response vs fitted values to inspect overall fit
#    - Use these plots to confirm that the Gaussian assumption and smoothness
#      selection are appropriate.
gam.check(gam_energy)
```




```
##
## Method: REML   Optimizer: outer newton
## full convergence after 11 iterations.
## Gradient range [-3.184645e-05,0.0001336563]
## (score -58.37538 & scale 0.001634153).
## Hessian positive definite, eigenvalue range [3.187416e-05,23.31475].
## Model rank =  45 / 45
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##                     k'      edf k-index p-value
## s(ws_cs)      9.00e+00 2.18e+00    1.22    0.92
## s(WindDir)    1.00e+01 2.51e+00    0.97    0.40
## s(HourOfDay) 8.00e+00 3.33e-04    1.18    0.88
```

```
## s(Day)       1.60e+01 9.84e+00      NA      NA
```

```r
# 2) gratia::draw() with residuals overlay
#    - draw() visualizes each smooth term with its confidence band.
#    - Setting residuals = TRUE adds the raw residuals as points below each curve,
#      helping to check whether any structure remains unmodeled.
library(gratia)
draw(gam_energy, residuals = TRUE)
```



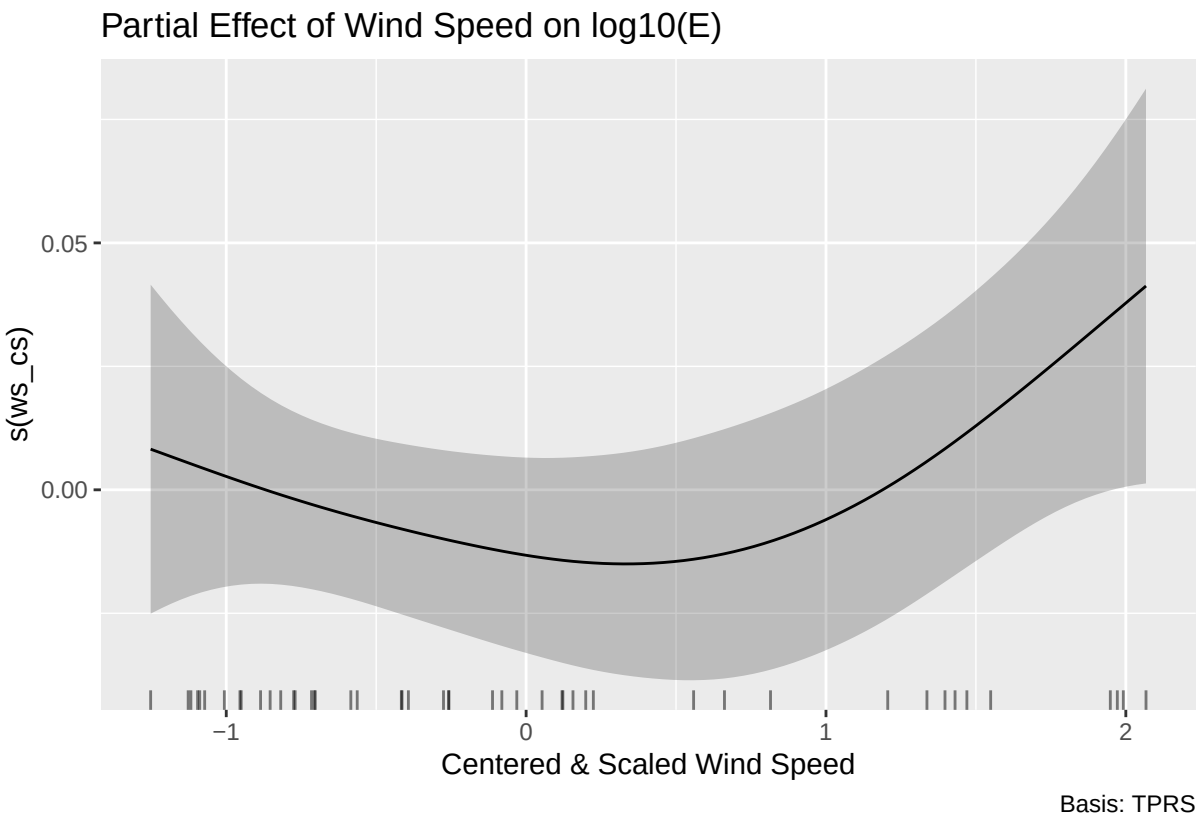

```r
##############################################################################
# Extract and Summarise RMS Displacement from Simplified GAM
# Purpose: Derive and back-transform model coefficients to obtain RMS noise levels
#          for Background and Operational states, then compare against a regulatory
#          threshold in a clear, formatted summary.
##############################################################################

# 1) Extract fitted model coefficients from the simplified GAM (gam_simpler)
coefs <- coef(gam_simpler)                      # Retrieve all parametric coefficients
logE0 <- coefs["(Intercept)"]                   # Baseline: log10(E) when turbine is OFF
logE1 <- logE0 + coefs["Operational1"]          # Add the ON/OFF shift to get log10(E) when ON

# 2) Back-transform from log10 scale to original energy units (nm²)
E_bg <- 10^logE0                                # Energy (variance) under Background conditions
E_op <- 10^logE1                                # Energy under Operational conditions
```

```r
# 3) Convert energy (variance) into RMS displacement (nm)
#    RMS = sqrt(variance) gives the displacement amplitude
RMS_bg <- sqrt(E_bg)                            # RMS noise floor when turbine is OFF
RMS_op <- sqrt(E_op)                            # RMS noise when turbine is ON

# 4) Define the regulatory RMS noise limit (nm)
limit <- 0.336                                  # Maximum allowable RMS displacement

# 5) Define a helper function to print a formatted summary
#    - Uses fixed-width formatting for alignment
#    - Clearly states whether each condition is above/below the limit
print_rms_summary <- function(RMS_bg, RMS_op, limit) {
  cat("\n==================== RMS Noise Summary ====================\n\n")
  cat(sprintf("%-25s : %8.5f nm\n", "Background RMS noise", RMS_bg))
  cat(sprintf("%-25s : %8.5f nm\n", "Operational RMS noise", RMS_op))
  cat(sprintf("%-25s : %8.5f nm\n\n", "Regulatory limit", limit))

  cat(sprintf("Background noise is %-5s the regulatory limit.\n",
              ifelse(RMS_bg < limit, "below", "above")))
  cat(sprintf("Operational noise is %-5s the regulatory limit.\n",
              ifelse(RMS_op < limit, "below", "above")))
  cat("============================================================\n\n")
}

# 6) Call the summary function to display results
print_rms_summary(RMS_bg, RMS_op, limit)
```

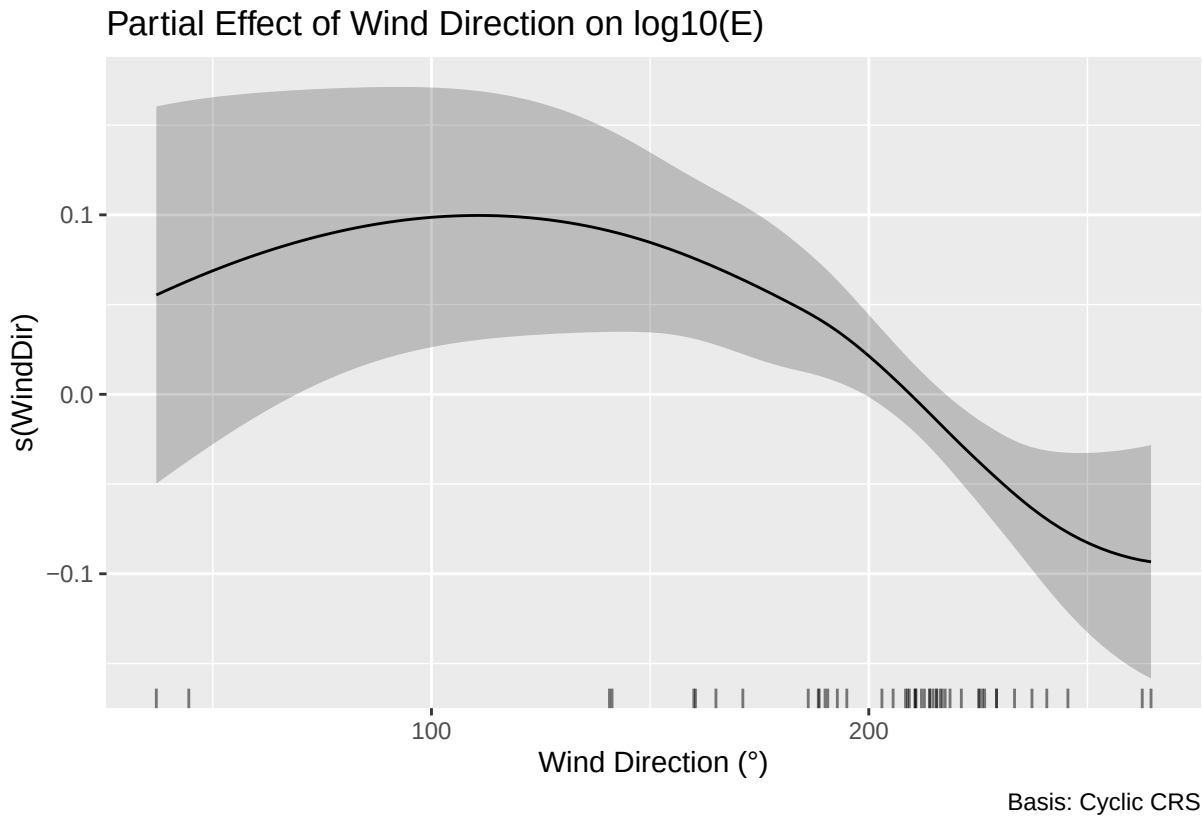
```
##
## ==================== RMS Noise Summary ====================
##
## Background RMS noise      :  0.05295 nm
## Operational RMS noise     :  0.14412 nm
## Regulatory limit          :  0.33600 nm
##
## Background noise is below the regulatory limit.
## Operational noise is below the regulatory limit.
## ============================================================
```

```r
##############################################################################
# Visualize Smooth Effects from Full GAM (gam_energy)
# Purpose: Display the estimated nonlinear relationships between predictors
#          and log -energy via GAM smooths, aiding interpretation of each term.
# Notes: Uses `gratia::draw()` to extract and plot individual smooth components.
##############################################################################

# Load 'gratia' library for GAM diagnostic and visualization tools
library(gratia)

# 1) Smooth effect of centered & scaled wind speed on log (E)
#    - Selects the first smooth term, s(ws_cs)
#    - Shows how deviations in wind speed modulate seismic energy
draw(gam_energy, select = 1) +
  labs(
```

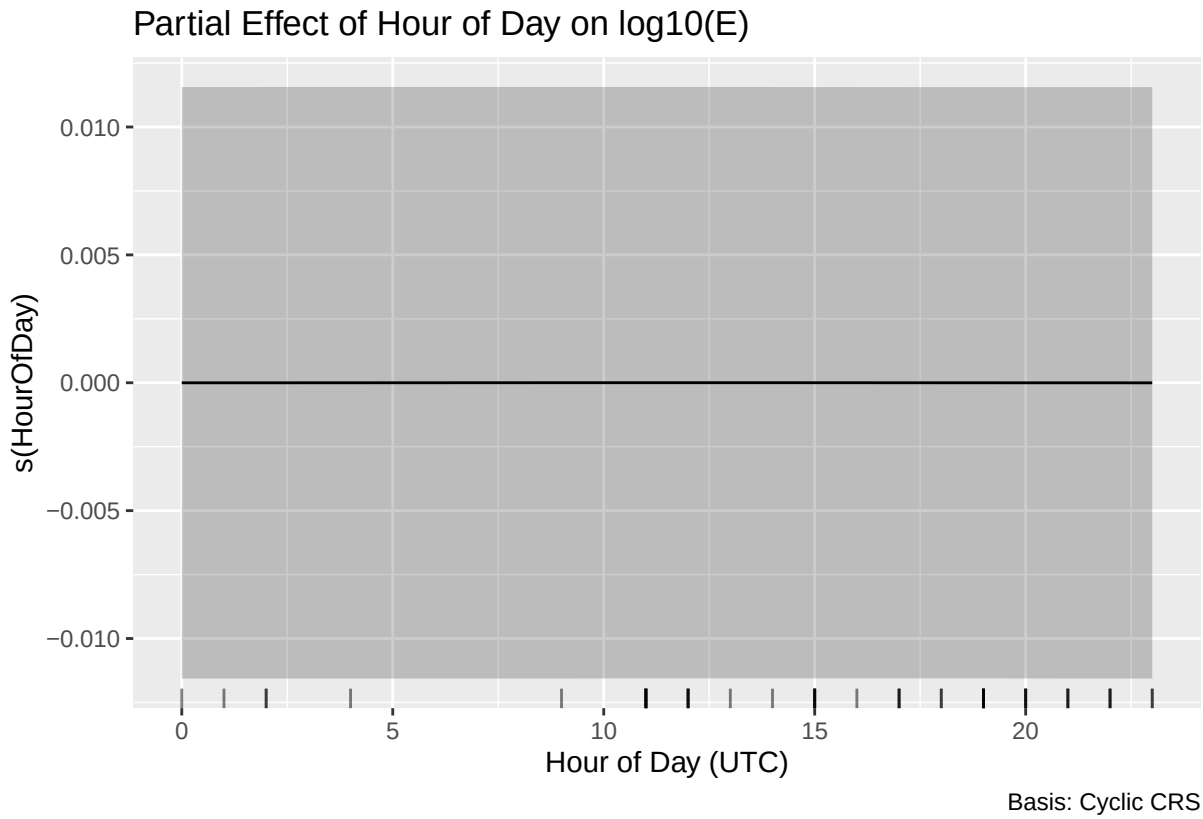
```r
    title = "Partial Effect of Wind Speed on log10(E)",  # Describes plot content
    x     = "Centered & Scaled Wind Speed",             # Predictor axis
    y     = "s(ws_cs)"                                  # Response: smooth function estimate
  )
```




```r
# 2) Cyclic smooth effect of wind direction on log (E)
#    - Selects the second smooth term, s(WindDir)
#    - Accounts for circular continuity at 0°/360°
draw(gam_energy, select = 2) +
  labs(
    title = "Partial Effect of Wind Direction on log10(E)",
    x     = "Wind Direction (°)",     # Compass degrees
    y     = "s(WindDir)"              # Smooth function label
  )
```



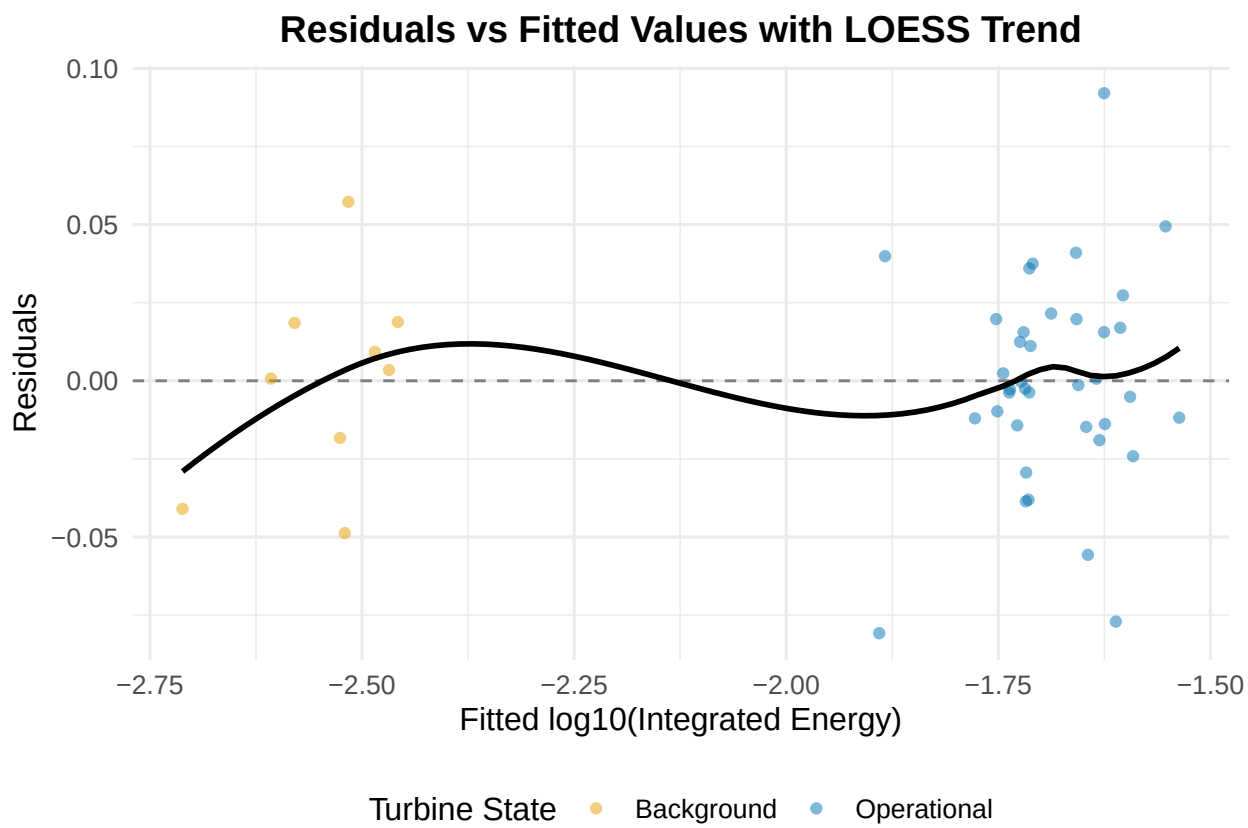

```r
# 3) Cyclic smooth effect of hour of day on log (E) (diurnal cycle)
#    - Selects the third smooth term, s(HourOfDay)
#    - Captures within-day variation in seismic energy
draw(gam_energy, select = 3) +
  labs(
    title = "Partial Effect of Hour of Day on log10(E)",
    x     = "Hour of Day (UTC)",      # Time-of-day in hours
    y     = "s(HourOfDay)"            # Smooth term label
  )
```




```
##############################################################################
# Enhanced Residuals vs. Fitted Plot
# Purpose: Improve presentation of GAM residual diagnostics by
#          • Adding a LOESS trend line to highlight systematic deviations
#          • Coloring points by turbine state to detect state-specific patterns
##############################################################################

# Prepare data frame with fitted values, residuals, and turbine state
resid_df <- tibble(
  fitted      = fitted(gam_energy),                # Predicted log10(E)
  resid       = residuals(gam_energy),             # Observed minus predicted
  Type        = factor(energy_df$Operational,      # Bring in turbine state for coloring
                       levels = c(0,1),
                       labels = c("Background","Operational"))
)

# Build the plot
ggplot(resid_df, aes(x = fitted, y = resid, color = Type)) +
  # Reference line at zero residual for visual baseline
  geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50") +
  # Scatter points, semi-transparent to reduce overplotting
  geom_point(alpha = 0.5, size = 1.5) +
  # LOESS smooth curve (no CI) to reveal any systematic trend
  geom_smooth(method = "loess", se = FALSE, colour = "black", linetype = "solid") +
  # Color palette matching earlier figures for consistency
  scale_color_manual(
```

```r
    values = c("Background" = "#E69F00", "Operational" = "#0072B2"),
    name   = "Turbine State"
  ) +
  # Labels and title
  labs(
    title = "Residuals vs Fitted Values with LOESS Trend",
    x     = "Fitted log10(Integrated Energy)",
    y     = "Residuals"
  ) +
  # Clean, publication-ready theme
  theme_minimal(base_size = 12) +
  theme(
    plot.title    = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

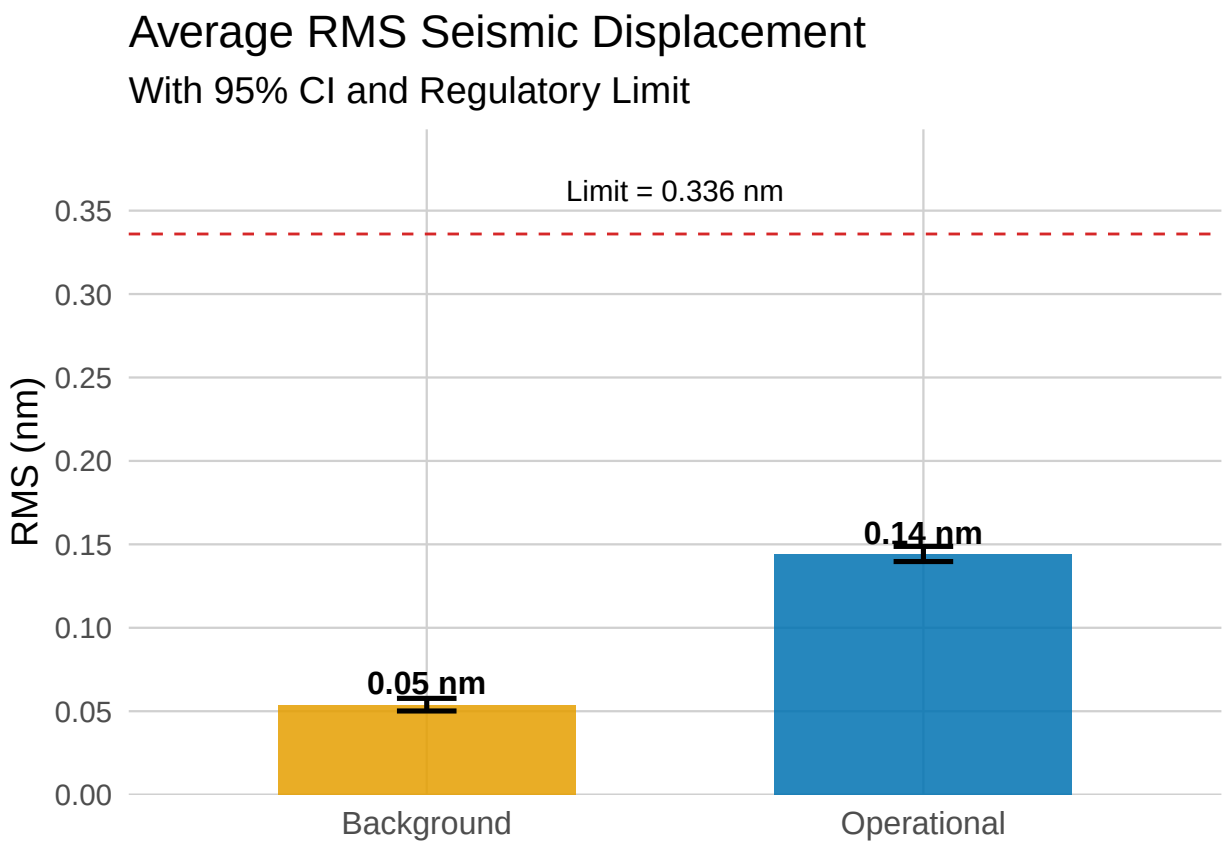



```r
##############################################################################
# Average RMS Seismic Displacement with 95% CI & Regulatory Limit
# Purpose: Calculate and visualise mean RMS displacement for Background vs
#          Operational states, including 95% confidence intervals and the
#          regulatory threshold, to succinctly communicate key noise metrics.
##############################################################################
```

```r
# 1) Derive RMS per 10-min block and classify by operational state
rms_df <- energy_df %>%
  mutate(
    RMS   = sqrt(10^logE),                          # Convert log10(E) back to E (nm²), then sqrt → RM
    Group = if_else(Operational == "1",             # Label each block as Background or Operational
                    "Operational",
                    "Background")
  )

# 2) Compute group-level summary statistics (mean ± 95% CI)
rms_summary <- rms_df %>%
  group_by(Group) %>%                               # Summarise separately for Background and Operatio
  summarise(
    meanRMS = mean(RMS, na.rm = TRUE),              # Average RMS displacement
    sdRMS   = sd(RMS,   na.rm = TRUE),              # Sample standard deviation
    n       = n(),                                  # Number of observations
    seRMS   = sdRMS / sqrt(n),                      # Standard error of the mean
    lo      = meanRMS - 1.96 * seRMS,               # Lower bound of the 95% confidence interval
    hi      = meanRMS + 1.96 * seRMS,               # Upper bound of the 95% confidence interval
    .groups = "drop"                                # Remove grouping afterwards
  )

# 3) Plot: bars for mean RMS, error bars for 95% CI, dashed limit line, and numeric labels
limit <- 0.336                                     # Regulatory RMS limit in nm

ggplot(rms_summary, aes(x = Group, y = meanRMS, fill = Group)) +
  geom_col(
    width = 0.60,      # Bar width
    alpha = 0.85,      # Semi-transparent fill
    show.legend = FALSE
  ) +
  geom_errorbar(
    aes(ymin = lo, ymax = hi),  # 95% CI error bars
    width = 0.12,
    size = 0.9,
    colour = "black"
  ) +
  geom_text(
    aes(label = sprintf("%.2f nm", meanRMS)),  # Annotate each bar with its mean value
    vjust = -0.45,
    size = 4.2,
    fontface = "bold"
  ) +
  geom_hline(
    yintercept = limit,          # Regulatory limit line
    linetype = "dashed",
    colour = "#d62728",
    linewidth = 0.5
  ) +
  annotate(
    "text",
    x     = 1.5,                 # Centered between the two bars
    y     = limit + 0.02,        # Just above the limit line
```

```r
    label = paste0("Limit = ", limit, " nm"),
    colour = "black",
    size   = 4,
    vjust  = 0
  ) +
  scale_fill_manual(
    values = c(
      Background  = "#E69F00",   # Orange for Background
      Operational = "#0072B2"    # Blue for Operational
    )
  ) +
  scale_y_continuous(
    breaks = seq(0, 0.40, 0.05),            # Y-axis ticks every 0.05 nm
    expand = expansion(mult = c(0, 0.12))   # Add space above bars for labels
  ) +
  labs(
    title    = "Average RMS Seismic Displacement",
    subtitle = "With 95% CI and Regulatory Limit",
    x        = NULL,                        # No x-axis title needed
    y        = "RMS (nm)"                   # Y-axis label with units
  ) +
  theme_minimal(base_size = 14) +
  theme(
    panel.grid.major = element_line(
      colour = "#d0d0d0", size = 0.4       # Light grey major grid lines
    ),
    panel.grid.minor = element_blank(),    # Remove minor grid lines
    axis.text.x      = element_text(size = 12),
    axis.ticks.x     = element_blank()     # Clean up x-axis ticks
  )
```

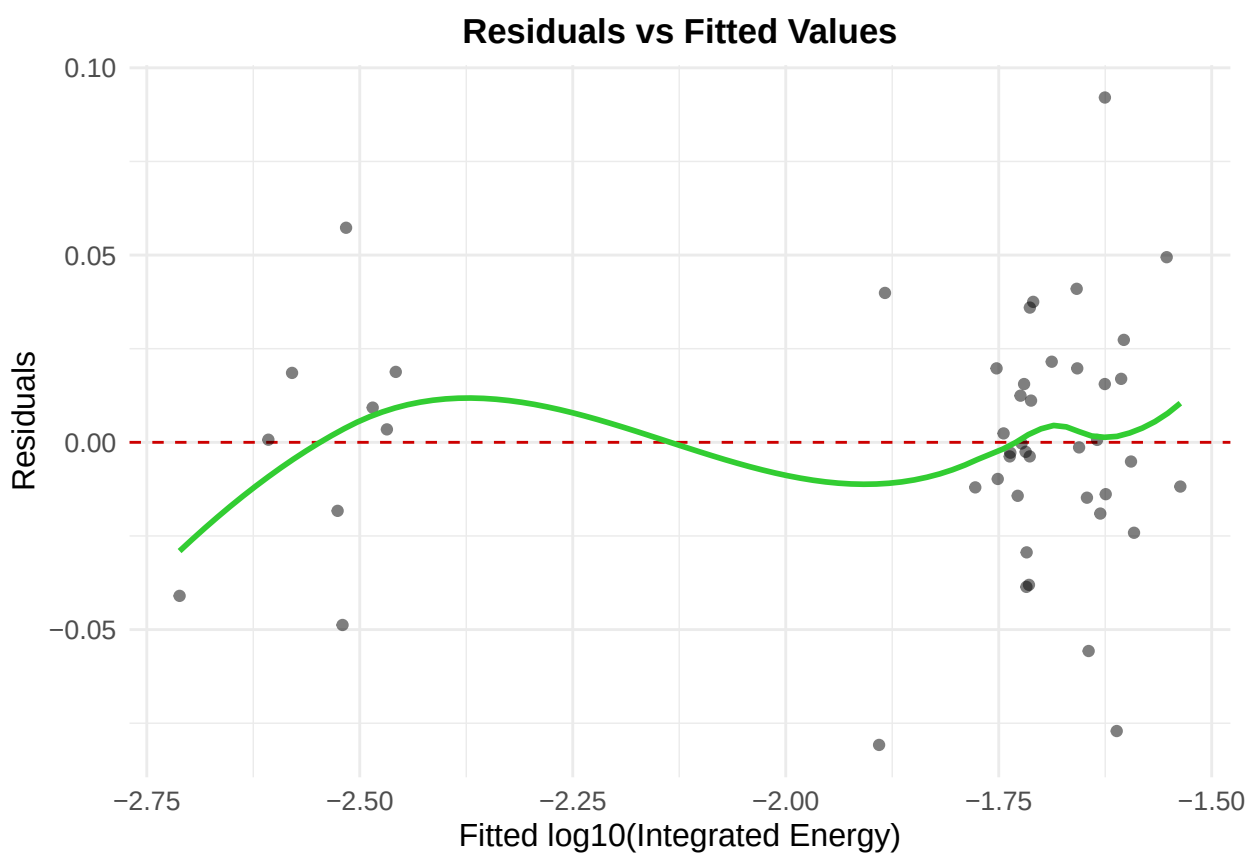
```
## Warning: The `size` argument of `element_line()` is deprecated as of ggplot2 3.4.0.
## i Please use the `linewidth` argument instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```
##############################################################################
# Residuals vs Fitted Diagnostic Plot
# Purpose: Check for model misspecification by plotting residuals against
#          fitted values, with a trend line to reveal systematic patterns.
# Rationale: Ensures homoscedasticity and unbiasedness of the GAM fit.
##############################################################################

# Prepare a data frame of fitted values and raw residuals from the full GAM
resid_df <- data.frame(
  fitted = fitted(gam_energy),    # Predicted log10(E) from the model
  resid  = residuals(gam_energy)  # Observed minus predicted values
)

# Generate the diagnostic plot using ggplot2
ggplot(resid_df, aes(x = fitted, y = resid)) +
  # Reference line at zero residual highlights bias if residuals systematically deviate
  geom_hline(
    yintercept = 0,
    linetype    = "dashed",
    color       = "red3"
  ) +
  # Scatter of residuals to inspect spread and detect any funnel shapes
  geom_point(
    alpha = 0.5                # Semi-transparent points reduce overplotting
  ) +
  # LOESS smooth line (no confidence interval) to reveal any non-random trend
```

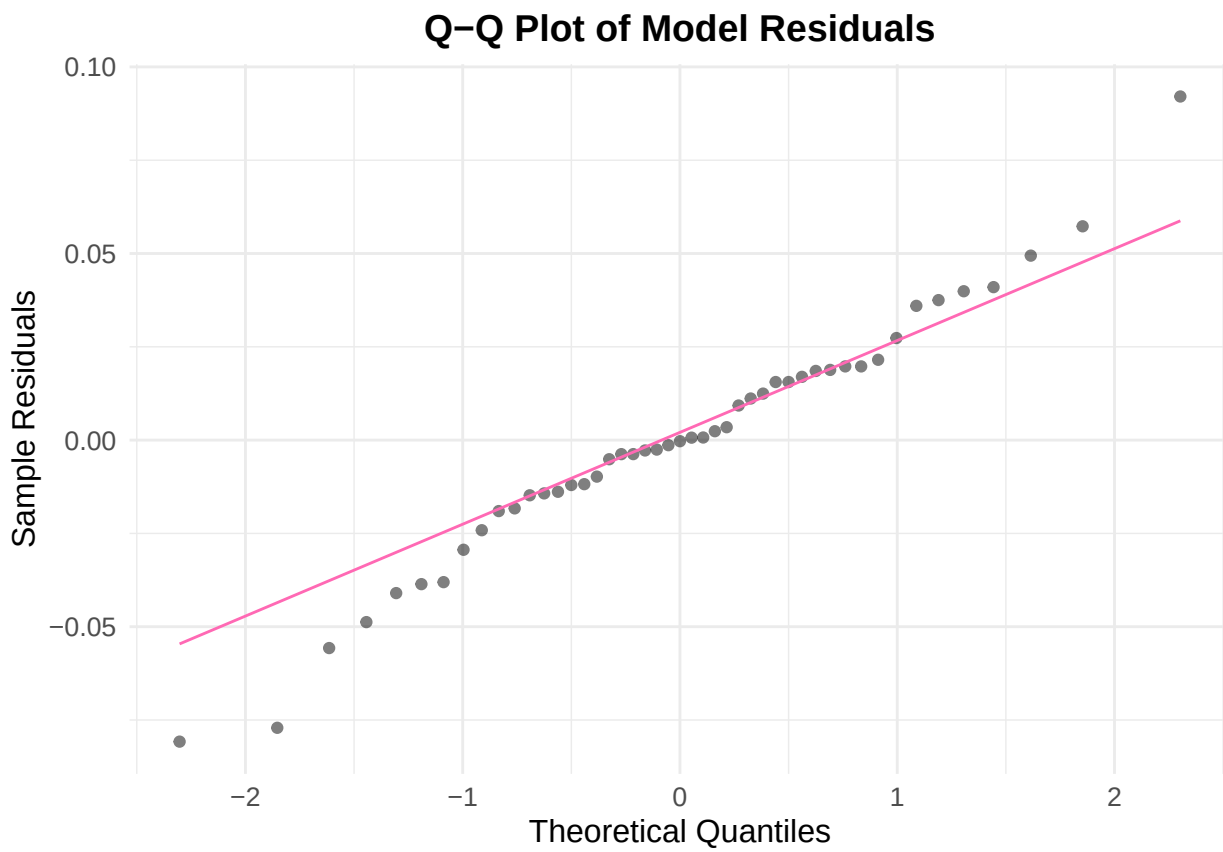
```r
  geom_smooth(
    se      = FALSE,
    colour  = "limegreen",
    linewidth = 1
  ) +
  # Descriptive labels for clarity
  labs(
    title = "Residuals vs Fitted Values",
    x     = "Fitted log10(Integrated Energy)",
    y     = "Residuals"
  ) +
  # Minimal theme for a clean, publication-ready appearance
  theme_minimal() +
  theme(
    plot.title    = element_text(face = "bold", hjust = 0.5),  # Center and bold title
    axis.title    = element_text(size = 12),                   # Readable axis titles
    axis.text     = element_text(size = 10)                    # Readable axis text
  )
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```r
##############################################################################
# Q-Q Plot of Model Residuals for Normality Check
# Purpose: Assess whether the residuals from the GAM are approximately
#          normally distributed, validating the Gaussian assumption.
```

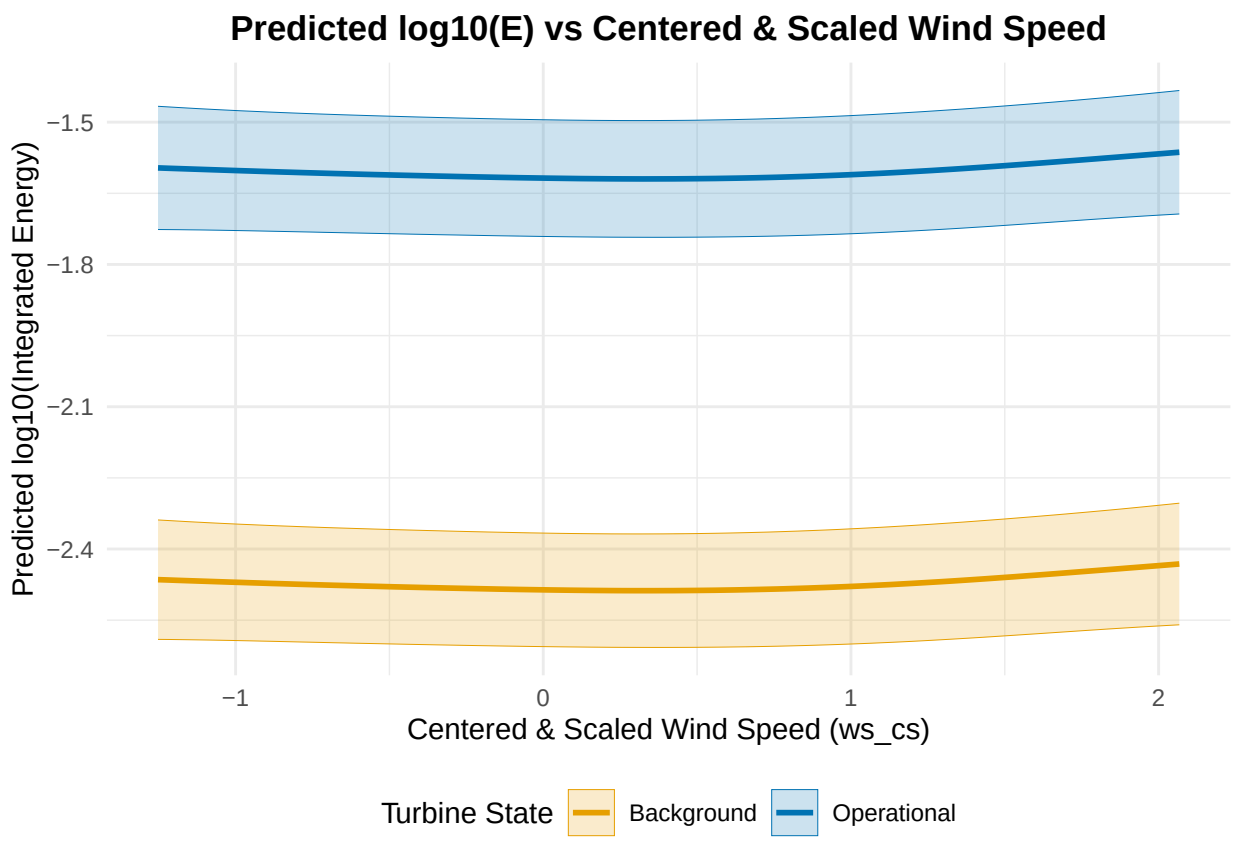
```r
################################################################################

# Assumes `resid_df` exists with a column `resid` of model residuals.

ggplot(resid_df, aes(sample = resid)) +
  # Plot sample quantiles of residuals against theoretical normal quantiles
  stat_qq(alpha = 0.5) +

  # Add reference line for perfect normality (passes through quartiles)
  stat_qq_line(color = "hotpink") +

  # Descriptive title and axis labels for clarity
  labs(
    title = "Q-Q Plot of Model Residuals",
    x     = "Theoretical Quantiles",
    y     = "Sample Residuals"
  ) +

  # Clean minimal theme for focus on data
  theme_minimal() +

  # Enhance presentation: center and bold title, adjust text sizes
  theme(
    plot.title  = element_text(face = "bold", hjust = 0.5, size = 14),
    axis.title  = element_text(size = 12),
    axis.text   = element_text(size = 10)
  )
```




```
##############################################################################
# Model-Based Prediction of log (E) vs Wind Speed
# Purpose: Generate and visualise the GAM's predicted log -energy as a function
#          of wind speed, holding other covariates constant, for Background vs
#          Operational states to illustrate the modeled wind-speed effect.
##############################################################################

# Load required libraries (skip if already loaded)
library(dplyr)     # Data wrangling (mutate, group_by, etc.)
library(ggplot2)   # Plotting system

# 1) Build a prediction grid that varies only wind speed
#    - ws_cs     : 100 equally spaced values of centered & scaled wind speed
#    - WindDir   : fixed at 180° (southerly flow) to isolate wind-speed effect
#    - HourOfDay : fixed at 12 (noon UTC) to remove diurnal variation
#    - Day       : choose the first observed day as a reference factor level
#    - Operational: include both turbine OFF (0) and ON (1) for comparison
ws_seq <- seq(
  from = min(energy_df$ws_cs),     # minimum scaled wind speed in data
  to   = max(energy_df$ws_cs),     # maximum scaled wind speed in data
  length.out = 100                 # number of points in the grid
)

newdata <- expand.grid(
  ws_cs       = ws_seq,
  WindDir     = 180,
```

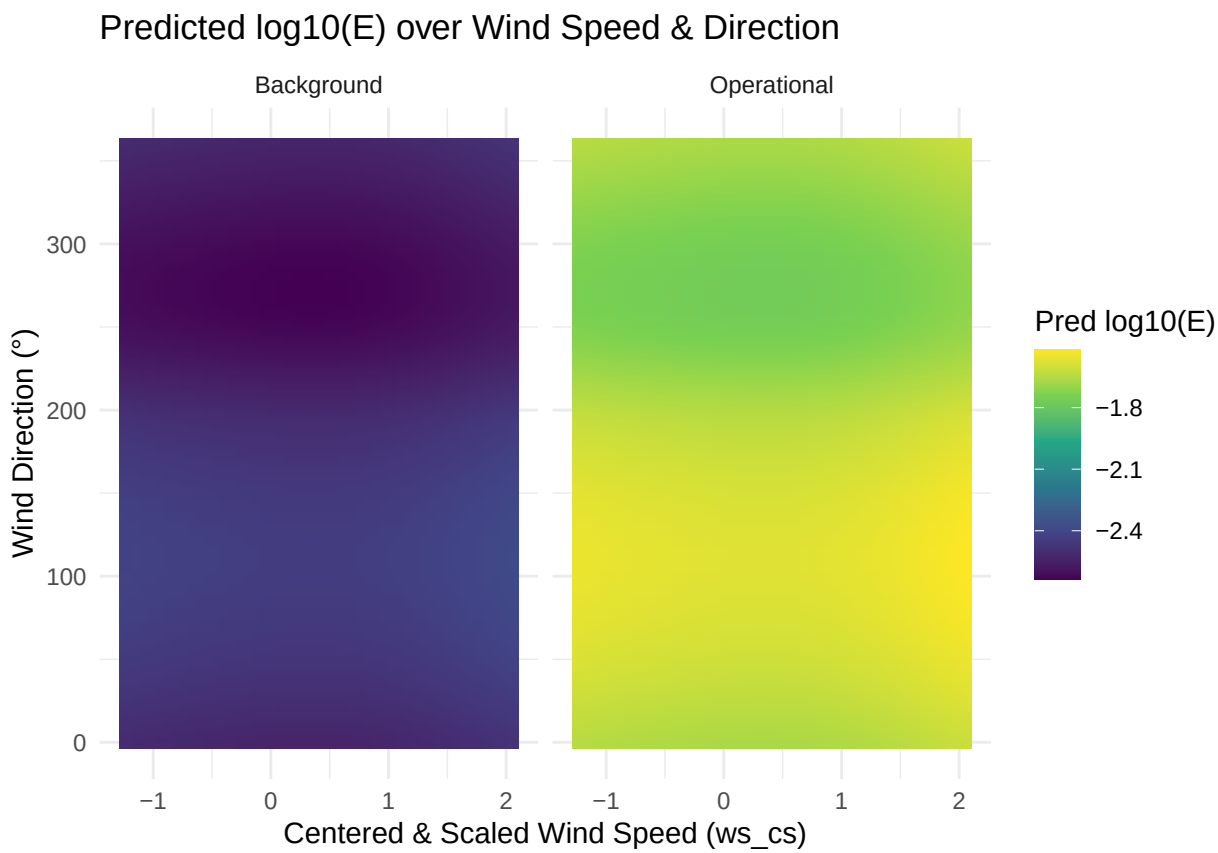
```r
  HourOfDay   = 12,
  Day          = factor(
                  levels(energy_df$Day)[1],            # reference level
                  levels = levels(energy_df$Day)
                ),
  Operational = c(0, 1)            # two scenarios: Background vs Operational
)

# 2) Obtain model predictions and standard errors on the grid
#    - predict(..., se.fit = TRUE) returns fitted values and their SEs
pred <- predict(gam_energy, newdata, se.fit = TRUE)

# Assemble a tidy data frame with fit and 95% confidence bands
plot_df <- newdata %>%
  mutate(
    fit  = pred$fit,               # predicted log10(E)
    lo   = fit - 2 * pred$se.fit,  # lower bound of 95% CI
    hi   = fit + 2 * pred$se.fit,  # upper bound of 95% CI
    Type = factor(
      Operational,
      levels = c(0, 1),
      labels = c("Background", "Operational")
    )  # descriptive labels for plotting
  )

# 3) Plot predicted log10(E) vs wind speed with ribbons for 95% CI
#    - Separate curves and ribbons for Background vs Operational
#    - Consistent line and fill colors for clarity
ggplot(plot_df, aes(x = ws_cs, y = fit, colour = Type, fill = Type)) +
  geom_line(size = 1) +                           # mean prediction line
  geom_ribbon(
    aes(ymin = lo, ymax = hi),                    # CI ribbon
    alpha = 0.20,                                 # semi-transparent fill
    linewidth = 0                                # no border on ribbon
  ) +
  labs(
    title = "Predicted log10(E) vs Centered & Scaled Wind Speed",
    x     = "Centered & Scaled Wind Speed (ws_cs)",
    y     = "Predicted log10(Integrated Energy)",
    colour = "Turbine State",
    fill   = "Turbine State"
  ) +
  scale_color_manual(
    values = c("Background"  = "#E69F00",
               "Operational" = "#0072B2")
  ) +
  scale_fill_manual(
    values = c("Background"  = "#E69F00",
               "Operational" = "#0072B2")
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",  # place legend below the plot
```

```
    plot.title      = element_text(face = "bold", hjust = 0.5)
  )
```



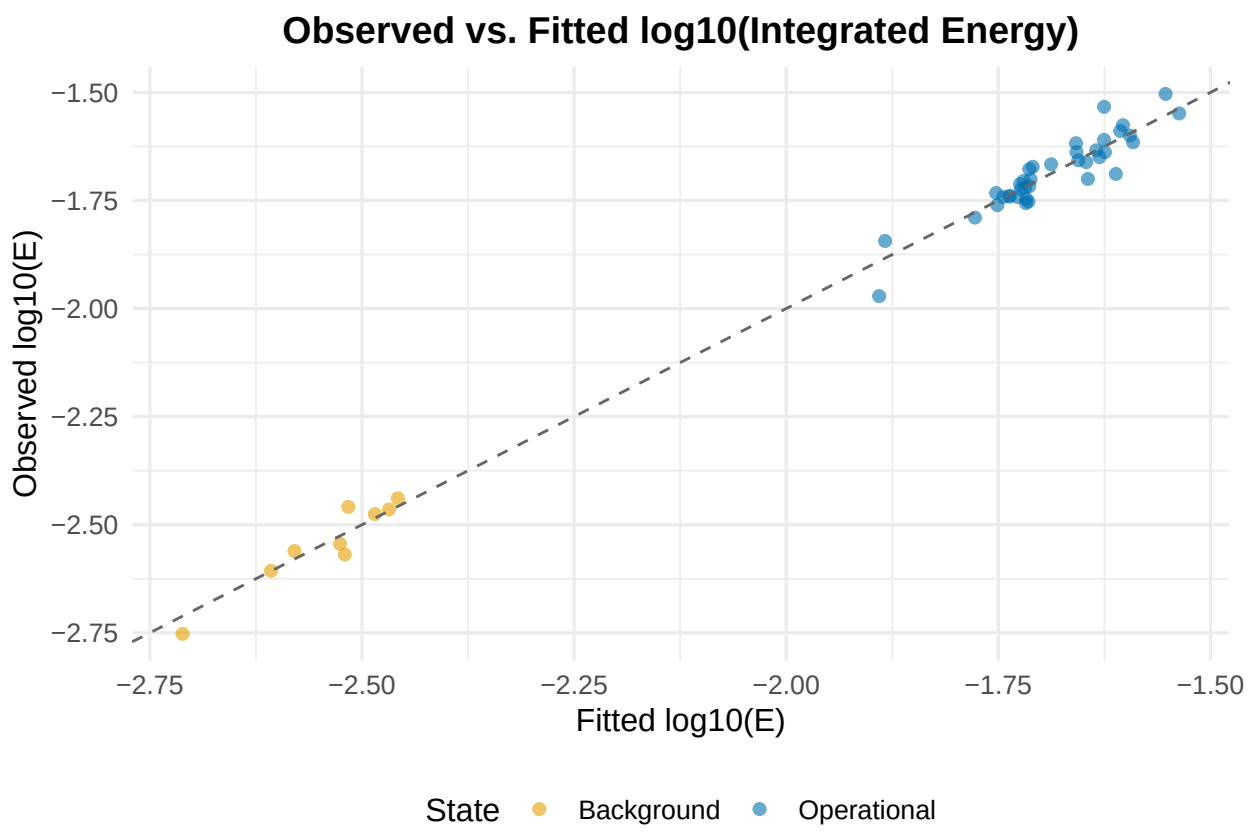

```
##############################################################################
# Predicted log10(E) Heat-Map over Wind Speed & Direction
# Purpose: Display the GAM's predicted log -energy as a 2D raster (heat-map)
#          to assess the joint influence of wind speed and wind direction,
#          with separate panels for Background vs Operational states.
##############################################################################

# Load ggplot2 for heat-map visualisation (skip if already loaded)
library(ggplot2)

#
# 1) Build a prediction grid spanning:
#    • ws_cs      : centred-&-scaled wind speed (50 evenly spaced points)
#    • WindDir    : full 0-360° compass (50 points)
#  Fixed conditions:
#    • HourOfDay  = 12 (mid-day, removes diurnal variation)
#    • Day        = first observed day (preserves factor structure)
#    • Operational= 0 & 1 → separate panels for each state
#
ws_seq <- seq(
  min(energy_df$ws_cs),
  max(energy_df$ws_cs),
```

```r
  length.out = 50
)
wd_seq <- seq(0, 360, length.out = 50)

grid <- expand.grid(
  ws_cs       = ws_seq,
  WindDir     = wd_seq,
  HourOfDay   = 12,
  Day         = factor(
                  levels(energy_df$Day)[1],  # reference level
                  levels = levels(energy_df$Day)
                ),
  Operational = c(0, 1)                       # Background vs Operational
)

#
# 2) Add model predictions to the grid
#    - fit: predicted log10(E) from the full GAM
#    - Type: human-readable factor for plotting facets
#
grid$fit  <- predict(gam_energy, grid)         # Model-predicted log10(E)
grid$Type <- factor(
  grid$Operational,
  levels = c(0, 1),
  labels = c("Background", "Operational")
)

#
# 3) Heat-map: predicted log10(E) over wind speed & direction
#    - geom_raster with interpolate = TRUE for smooth gradient
#    - scale_fill_viridis_c for perceptually uniform color scale
#    - facet_wrap(~ Type) to compare states side-by-side
#
ggplot(grid, aes(x = ws_cs, y = WindDir, fill = fit)) +
  geom_raster(interpolate = TRUE) +               # Smoothed color tiles
  scale_fill_viridis_c(name = "Pred log10(E)") +  # Uniform perceptual scale
  facet_wrap(~ Type, nrow = 1) +                  # Separate panels: Background vs Operational
  labs(
    title = "Predicted log10(E) over Wind Speed & Direction",
    x     = "Centered & Scaled Wind Speed (ws_cs)",
    y     = "Wind Direction (°)"
  ) +
  theme_minimal()                                 # Clean, minimal theme for presentation
```



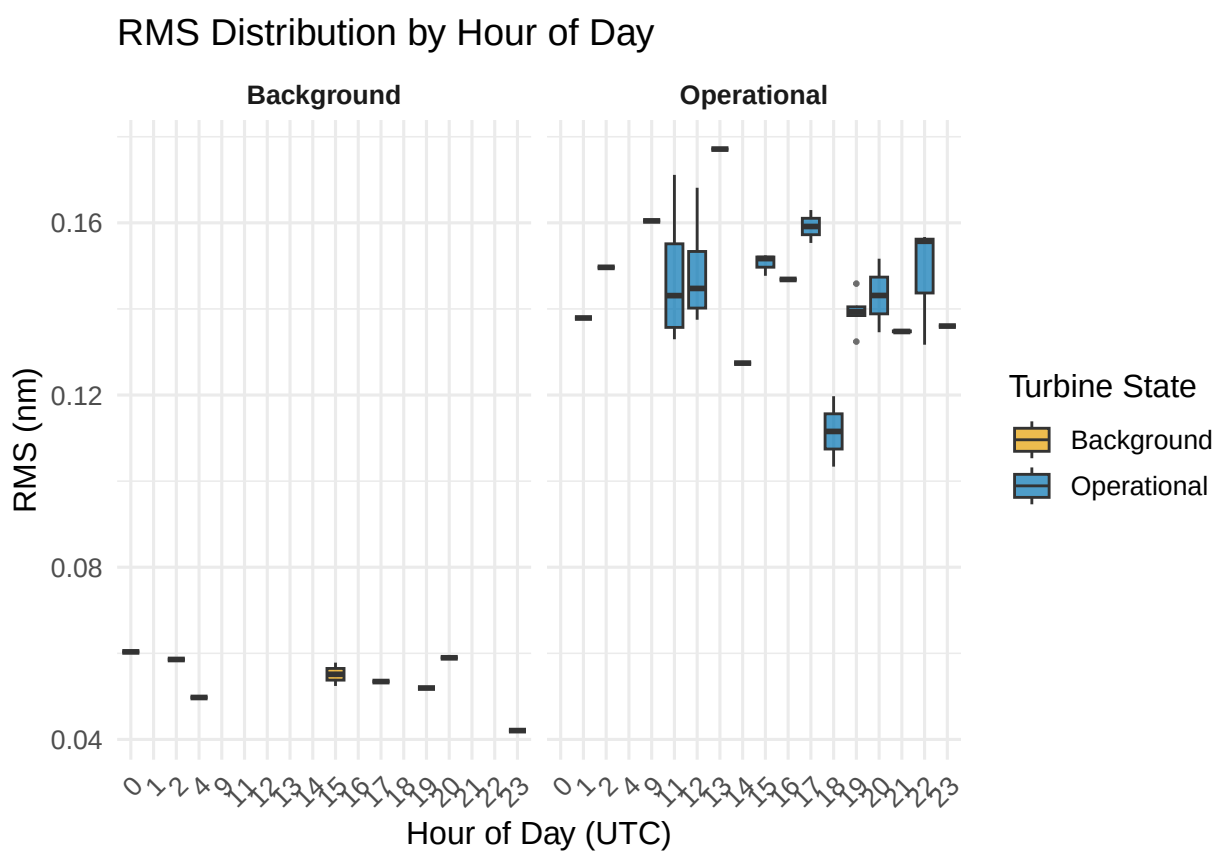

```
##############################################################################
# Observed vs Fitted Values from the Full GAM
# Purpose: Compare the GAM's predictions to the observed log -energy values
#          to assess overall model accuracy and identify any systematic bias.
##############################################################################

# 1) Load required libraries for data manipulation and plotting
library(dplyr)   # Data wrangling functions (mutate, %>%, etc.)
library(ggplot2) # Grammar of Graphics for plotting

# 2) Compute fitted values from the GAM and add a descriptive state factor
#    - predict(..., type = "response") returns the model's fitted log -energy
#    - State factor uses human-readable labels for turbine status
obs_fit <- energy_df %>%
  mutate(
    fitted = predict(
      gam_energy,
      newdata = .,
      type    = "response"
    ),
    State = factor(
      Operational,
      levels = c(0, 1),
      labels = c("Background", "Operational")
    )
  )
```

```r
# 3) Define custom colors for each turbine state to match earlier figures
cols <- c(
  "Background"  = "#E69F00",  # Blue for turbine OFF
  "Operational" = "#0072B2"   # Orange for turbine ON
)

# 4) Create scatter plot of observed vs fitted values with a 1:1 reference line
#    - geom_point(): plot each observation (semi-transparent to reduce overplotting)
#    - geom_abline(): dashed line at slope = 1 to indicate perfect prediction
#    - coord_equal() could be added for equal scales on both axes (optional)
ggplot(obs_fit, aes(x = fitted, y = logE, color = State)) +
  geom_point(alpha = 0.6, size = 1.8) +
  geom_abline(
    slope     = 1,
    intercept = 0,
    linetype  = "dashed",
    color     = "grey40"
  ) +
  scale_color_manual(values = cols, name = "State") +
  labs(
    title = "Observed vs. Fitted log10(Integrated Energy)",
    x     = "Fitted log10(E)",
    y     = "Observed log10(E)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "bottom",                        # Place legend below plot
    plot.title      = element_text(face = "bold", hjust = 0.5)
  )
```




```
##############################################################################
# RMS Distribution by Hour of Day and Turbine State (Colored)
# Purpose: Visualise the hourly distribution of RMS displacement (nm) separately
#          for Background vs Operational periods, using the approved orange/blue
#          palette to maintain consistency across figures.
#############################################################################

# Assumes dplyr and ggplot2 are already loaded.

energy_df %>%
  # 1) Compute RMS and create descriptive state factor
  mutate(
    RMS  = sqrt(E),                                        # Convert energy (nm²) to RMS (nm)
    Type = factor(
      Operational,                                         # Convert 0/1 flag to factor
      levels = c("0", "1"),
      labels = c("Background", "Operational")             # Human-readable state labels
    )
  ) %>%
  # 2) Plot boxplots of RMS by each hour, facetted by state
  ggplot(aes(
    x    = as.factor(HourOfDay),                           # Hour of day as categorical axis
    y    = RMS,                                            # RMS displacement in nm
    fill = Type                                            # Fill color by turbine state
  )) +
    geom_boxplot(
```

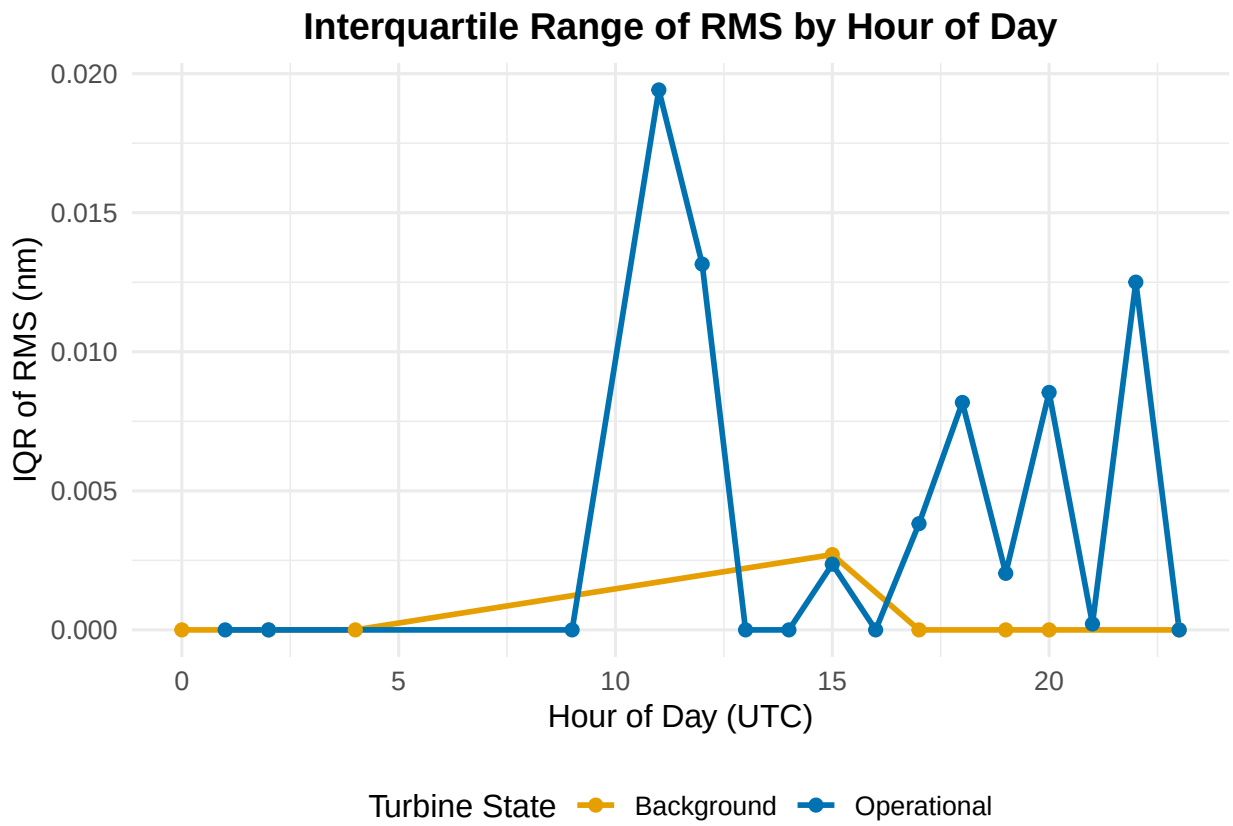
```r
    outlier.size = 0.5,                                  # Smaller points for outliers
    alpha        = 0.7                                   # Semi-transparent boxes
  ) +
  # 3) Apply the custom orange/blue palette for turbine state
  scale_fill_manual(
    values = c(
      "Background"  = "#E69F00",                        # Orange for turbine OFF
      "Operational" = "#0072B2"                         # Blue for turbine ON
    ),
    name = "Turbine State"                               # Legend title for clarity
  ) +
  facet_wrap(~ Type) +                                   # Separate panels for each state
  labs(
    title = "RMS Distribution by Hour of Day",
    x     = "Hour of Day (UTC)",                         # Specify time zone for reproducibility
    y     = "RMS (nm)"                                   # Y-axis label with units
  ) +
  theme_minimal(base_size = 12) +                        # Clean, legible theme
  theme(
    strip.text        = element_text(face = "bold"),     # Emphasise facet labels
    axis.text.x       = element_text(angle = 45, hjust = 1)# Tilt x labels for readability
  )
```




```r
##############################################################################
# Hourly Dispersion of RMS Displacement (Interquartile Range)
```

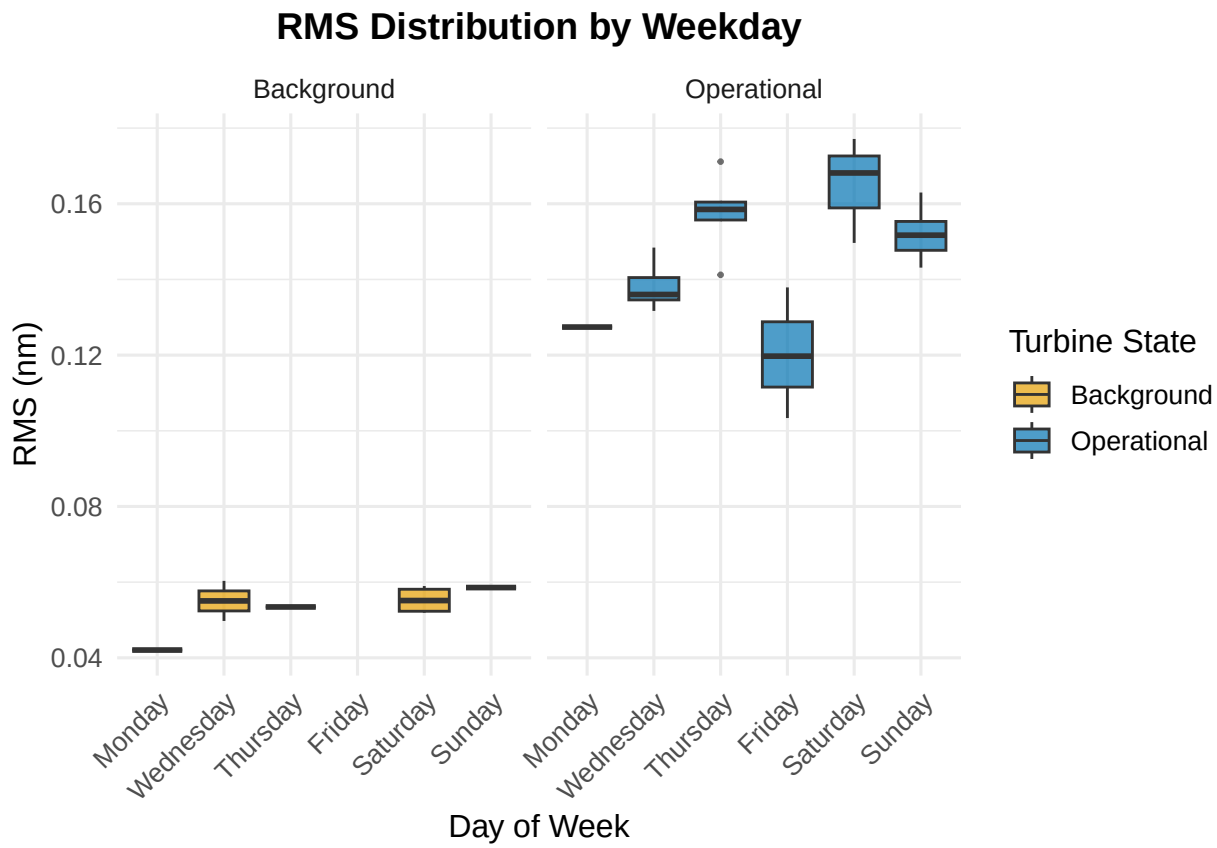
```r
# Purpose: Assess the variability of RMS displacement across each hour of the day
#          for Background vs Operational periods by plotting the IQR.
# Rationale: Highlights hours with greater spread in noise, indicating potential
#            periods of increased fluctuation not captured by mean trends.
##############################################################################

# Load tidyr for any future data reshaping (though IQR computation uses base R)
library(tidyr)

# 1) Compute the IQR of RMS by hour and turbine state
iqr_hourly <- energy_df %>%
  mutate(
    # Convert integrated energy (nm²) to RMS displacement (nm)
    RMS = sqrt(E),
    # Human-readable factor for turbine state
    Type = factor(
      Operational,
      levels = c("0", "1"),
      labels = c("Background", "Operational")
    )
  ) %>%
  group_by(Type, HourOfDay) %>%
  summarise(
    # Compute the interquartile range of RMS for each group
    IQR = IQR(RMS, na.rm = TRUE),
    .groups = "drop"  # Ungroup after summarising
  )

# 2) Plot IQR of RMS by HourOfDay with custom colors for consistency
ggplot(iqr_hourly, aes(x = HourOfDay, y = IQR, color = Type)) +
  # Line connects hourly IQR values for each turbine state
  geom_line(size = 1) +
  # Points mark the exact IQR at each hour
  geom_point(size = 2) +
  # Apply the approved orange/blue palette
  scale_color_manual(
    values = c(
      "Background"  = "#E69F00",  # Orange for turbine OFF
      "Operational" = "#0072B2"   # Blue for turbine ON
    ),
    name = "Turbine State"        # Legend title for clarity
  ) +
  # Titles and axis labels
  labs(
    title = "Interquartile Range of RMS by Hour of Day",
    x     = "Hour of Day (UTC)",
    y     = "IQR of RMS (nm)"
  ) +
  # Clean theme for publication quality
  theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom"    # Place legend at bottom for consistency
```

```
  )
```



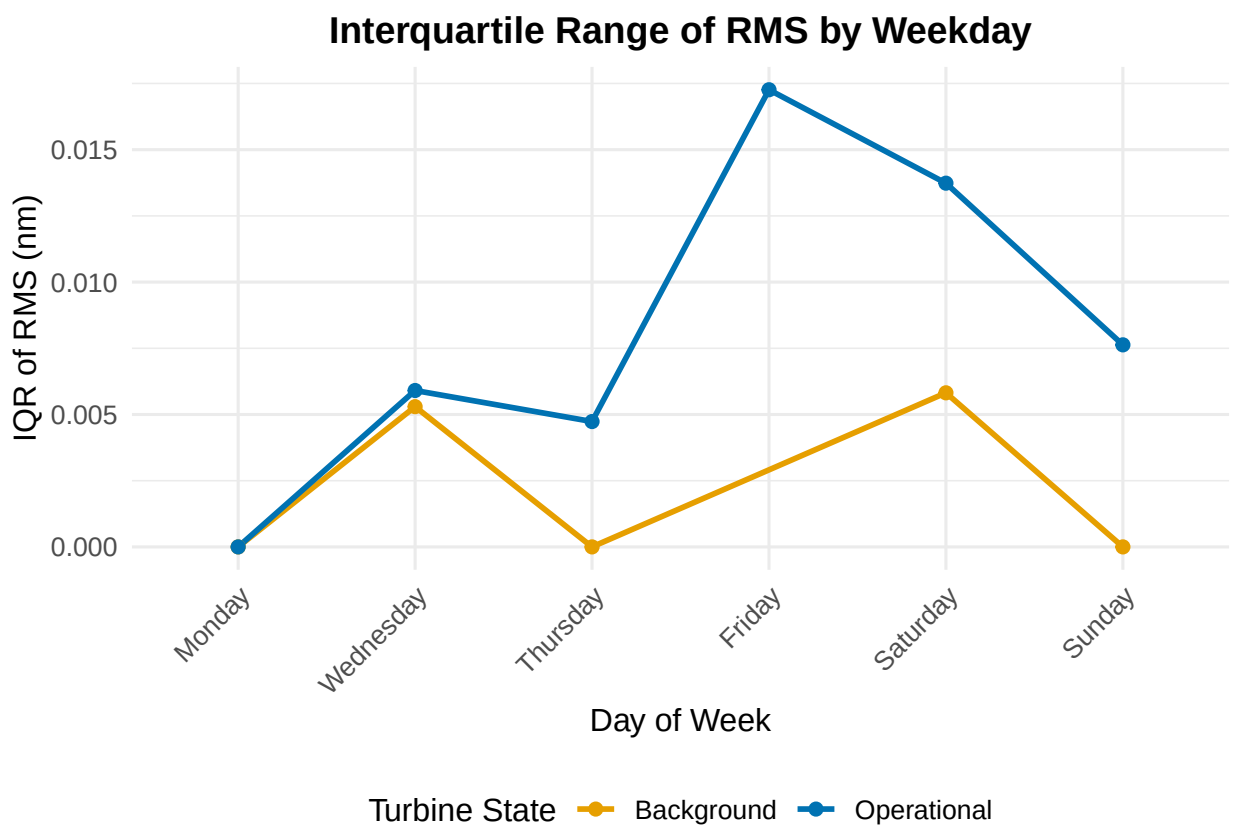

```
##############################################################################
# RMS Distribution by Weekday and Turbine State
# Purpose: Visualise how RMS displacement (nm) varies across days of the week
#          for Background vs Operational periods, using the standard color palette.
# Rationale: Detects any remaining weekday-level patterns after GAM modelling.
#############################################################################

energy_df %>%
  mutate(
    # Convert integrated energy (nm²) to RMS displacement (nm)
    RMS = sqrt(E),
    # Create descriptive factor for turbine state instead of numeric code
    Type = factor(
      Operational,
      levels = c("0", "1"),
      labels = c("Background", "Operational")
    ),
    # Extract full weekday name (Monday-Sunday) from the Day factor
    DOW = wday(
      as.Date(as.character(Day)),
      label = TRUE, abbr = FALSE, week_start = 1
    )
  ) %>%
```

```r
ggplot(aes(x = DOW, y = RMS, fill = Type)) +
  # Boxplots show median, IQR, and outliers of RMS per weekday
  geom_boxplot(
    outlier.size = 0.5,
    alpha        = 0.7
  ) +
  # Apply the approved Orange/Blue palette for turbine state
  scale_fill_manual(
    values = c(
      "Background"  = "#E69F00",  # Orange for OFF
      "Operational" = "#0072B2"   # Blue for ON
    ),
    name = "Turbine State"
  ) +
  # Separate panels for each state for direct comparison
  facet_wrap(~ Type) +
  # Titles and axis labels
  labs(
    title = "RMS Distribution by Weekday",
    x     = "Day of Week",
    y     = "RMS (nm)"
  ) +
  # Minimal theme with formatting tweaks
  theme_minimal(base_size = 12) +
  theme(
    axis.text.x  = element_text(angle = 45, hjust = 1),      # Tilt x-axis labels
    plot.title   = element_text(face = "bold", hjust = 0.5)  # Center and bold the title
  )
```




```
##############################################################################
# Weekly Dispersion of RMS Displacement (Interquartile Range)
# Purpose: Assess the variability of RMS displacement across each day of the week
#          for Background vs Operational periods by plotting the IQR.
# Rationale: Highlights weekdays with greater spread in noise, indicating potential
#            shifts not captured by mean trends or the GAM's fixed effects.
##############################################################################

# Load necessary libraries (skip if already loaded)
library(dplyr)     # Data wrangling
library(ggplot2)   # Plotting
library(lubridate) # Extracting day-of-week

# 1) Compute the IQR of RMS by weekday and turbine state
iqr_weekly <- energy_df %>%
  mutate(
    # Convert integrated energy (nm²) to RMS displacement (nm)
    RMS  = sqrt(E),
    # Human-readable factor for turbine state
    Type = factor(
      Operational,
      levels = c("0", "1"),
      labels = c("Background", "Operational")
    ),
    # Extract full weekday name from the 'Day' factor
    DOW  = wday(
```

```r
            as.Date(as.character(Day)),
            label = TRUE,
            abbr  = FALSE,
            week_start = 1
          )
  ) %>%
  group_by(Type, DOW) %>%
  summarise(
    # Interquartile range of RMS for each turbine state and weekday
    IQR = IQR(RMS, na.rm = TRUE),
    .groups = "drop"
  )

# 2) Plot IQR of RMS by weekday with custom colors
ggplot(iqr_weekly, aes(x = DOW, y = IQR, color = Type, group = Type)) +
  geom_line(size = 1) +  # Connect IQR points across weekdays
  geom_point(size = 2) + # Highlight each weekday's IQR
  scale_color_manual(
    values = c(
      "Background"  = "#E69F00",  # Orange for turbine OFF
      "Operational" = "#0072B2"   # Blue for turbine ON
    ),
    name = "Turbine State"        # Legend title
  ) +
  labs(
    title = "Interquartile Range of RMS by Weekday",
    x     = "Day of Week",
    y     = "IQR of RMS (nm)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom",
    axis.text.x     = element_text(angle = 45, hjust = 1)  # Tilt x-axis labels
  )
```

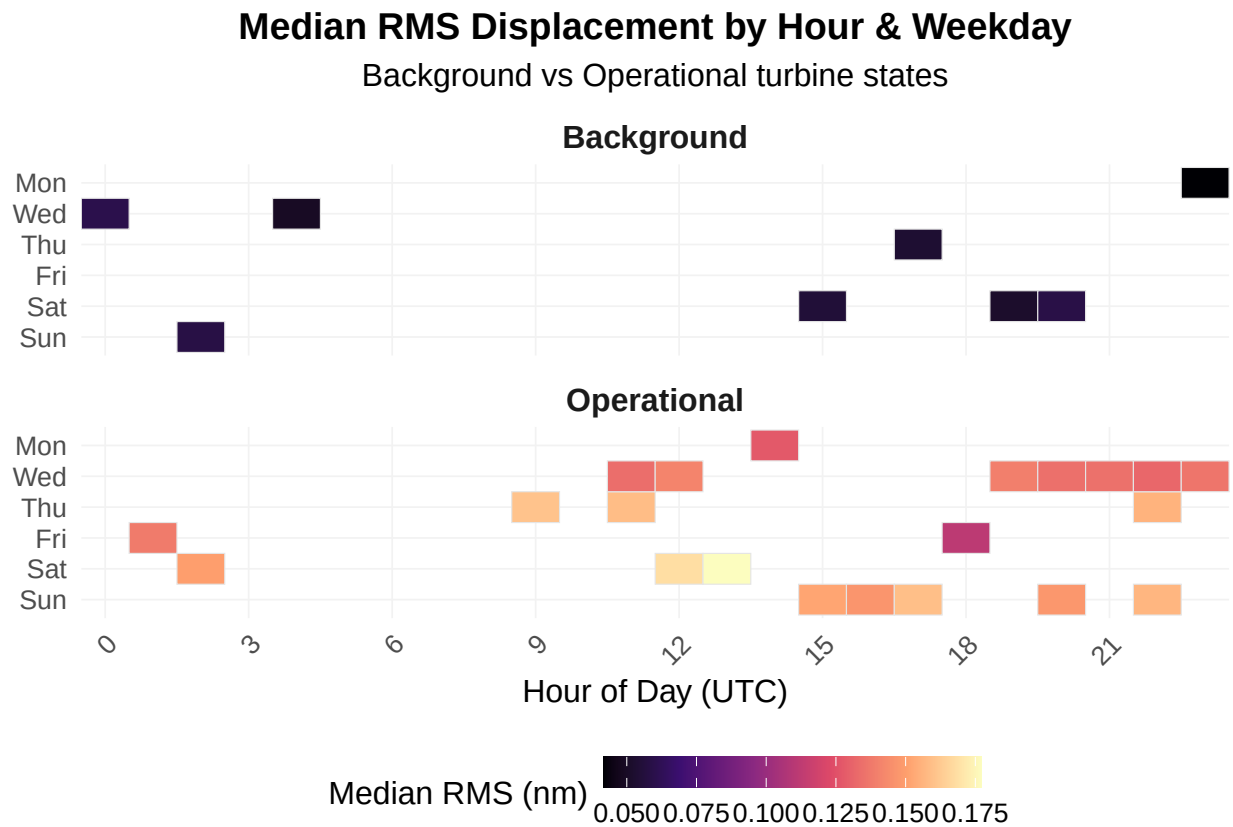



```
##############################################################################
# Heat-Map of Median RMS by Hour and Weekday
# Purpose: Create a publication-ready heat-map showing the median RMS displacement
#          across hours and weekdays, facetted by turbine state, with improved
#          styling for clarity and emphasis.
#############################################################################

library(dplyr)       # Data manipulation
library(ggplot2)     # Plotting
library(lubridate)   # Date-time utilities
library(viridis)     # Viridis color scales
```

```
## Loading required package: viridisLite
```

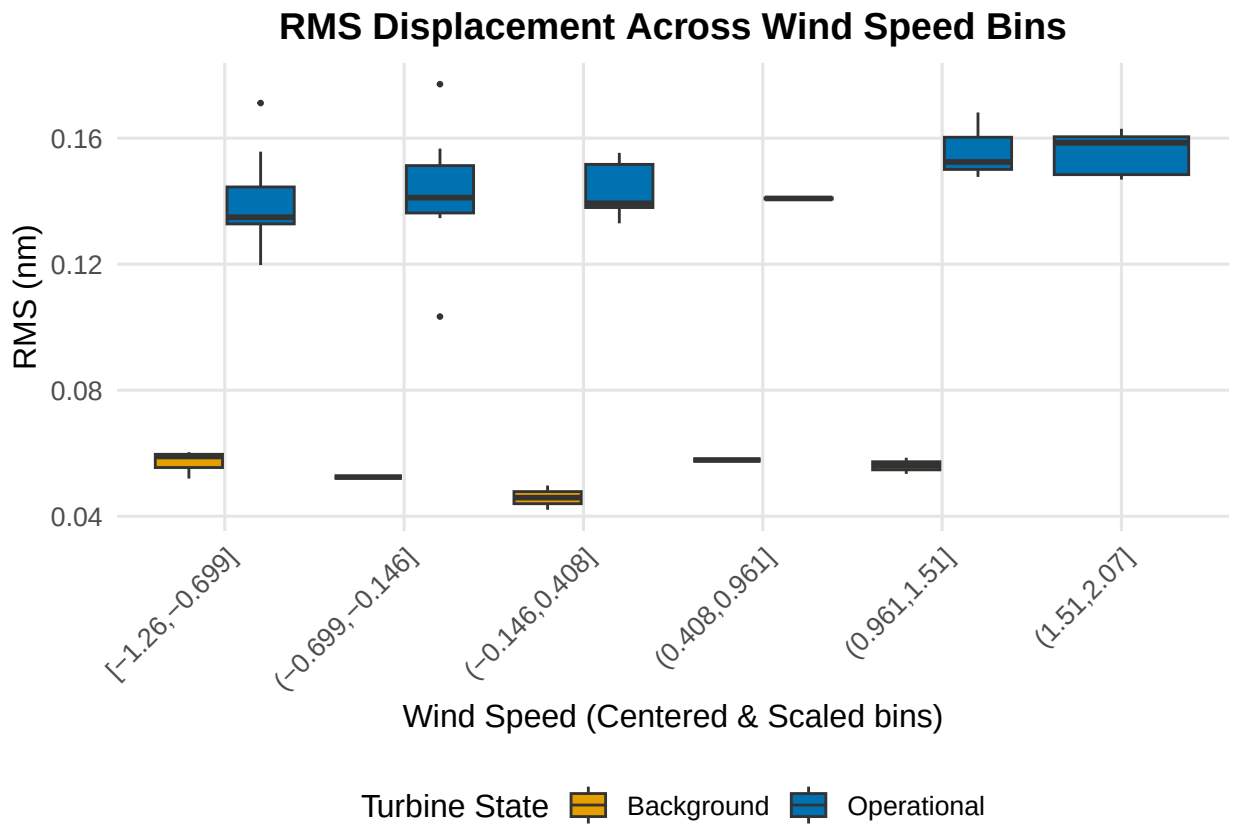
```
# 1) Prepare the summarized data with ordered weekdays (Mon at top)
heat_df <- energy_df %>%
  mutate(
    RMS  = sqrt(E),                                    # nm
    Type = factor(Operational, levels = c(0,1),
                  labels = c("Background","Operational")),
    # Extract abbreviated weekday and reverse levels so Monday is at top
    DOW  = wday(as.Date(as.character(Day)), label = TRUE, abbr = TRUE, week_start = 1) %>%
           factor(levels = rev(levels(.)))
  ) %>%
  group_by(Type, HourOfDay, DOW) %>%
```

```r
  summarise(
    med_RMS = median(RMS, na.rm = TRUE),
    .groups  = "drop"
  )

# 2) Plot the heat-map
ggplot(heat_df, aes(x = HourOfDay, y = DOW, fill = med_RMS)) +
  geom_tile(color = "grey90", size = 0.2) +                    # subtle grid lines
  facet_wrap(~ Type, ncol = 1) +                               # stack panels vertically
  scale_fill_viridis_c(
    option = "magma",                                          # warm-to-cool palette
    name   = "Median RMS (nm)"
  ) +
  scale_x_continuous(
    breaks = seq(0, 24, by = 3),                               # ticks every 3 hours
    expand = c(0, 0)
  ) +
  labs(
    title    = "Median RMS Displacement by Hour & Weekday",
    subtitle = "Background vs Operational turbine states",
    x        = "Hour of Day (UTC)",
    y        = NULL                                            # weekday labels suffice
  ) +
  theme_minimal(base_size = 12) +
  theme(
    # Titles
    plot.title    = element_text(face = "bold", size = 14, hjust = 0.5),
    plot.subtitle = element_text(size = 12, hjust = 0.5),
    # Axis
    axis.text.x    = element_text(angle = 45, vjust = 1, hjust = 1),
    axis.text.y    = element_text(size = 10),
    # Facet labels
    strip.text     = element_text(face = "bold", size = 12),
    # Grid lines
    panel.grid.major = element_line(color = "grey95", size = 0.3),
    panel.grid.minor = element_blank(),
    # Legend
    legend.position  = "bottom",
    legend.key.height = unit(0.4, "cm"),
    legend.key.width  = unit(1.0, "cm")
  )
```



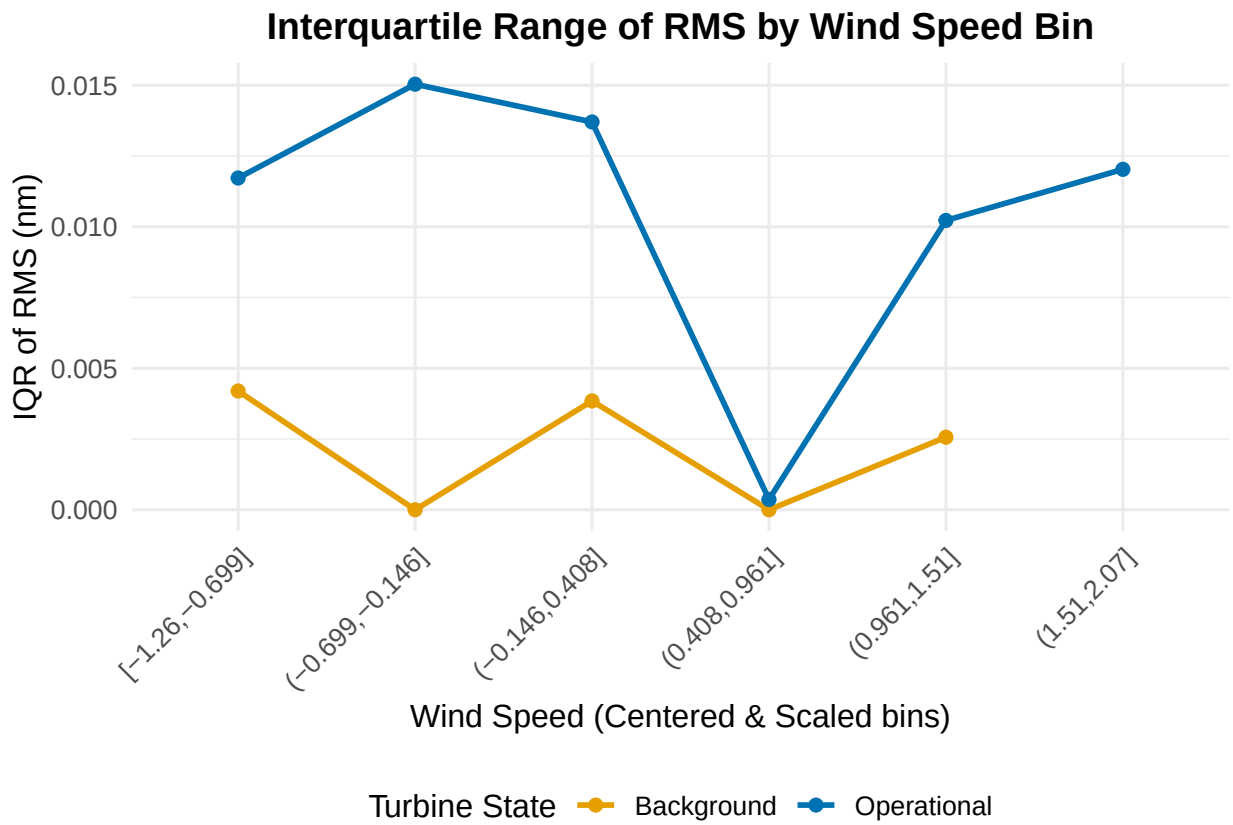

```
##############################################################################
# RMS Variation Across Wind Speed Bins by Turbine State
# Purpose: Examine how the distribution of RMS displacement (nm) varies across
#          discrete bins of centred-&-scaled wind speed, for both Background
#          and Operational periods. Enhances interpretability and presentation.
##############################################################################

# Assumes dplyr and ggplot2 are already loaded.

energy_df %>%
  # 1) Compute RMS and bin wind speed
  mutate(
    RMS    = sqrt(E),                          # Convert energy (nm²) to RMS (nm)
    ws_bin = cut(
               ws_cs,
               breaks = 6,                     # Divide the traffic into 6 equal-width bins
               include.lowest = TRUE           # Include extreme values in first/last bins
             ),
    Type   = factor(
               Operational,
               levels = c(0, 1),
               labels = c("Background", "Operational")
             )                                  # Descriptive turbine state labels
  ) %>%
  # 2) Plot side-by-side boxplots of RMS for each wind-speed bin and state
  ggplot(aes(x = ws_bin, y = RMS, fill = Type)) +
```

```r
  geom_boxplot(
    position = position_dodge(width = 0.8),     # Offset boxes for Background vs Operational
    outlier.size = 0.5                          # Smaller outlier points for cleaner look
  ) +
  # 3) Apply custom orange/blue palette for consistency
  scale_fill_manual(
    values = c(
      "Background"  = "#E69F00",                # Orange for turbine OFF
      "Operational" = "#0072B2"                 # Blue for turbine ON
    ),
    name = "Turbine State"                      # Legend title for clarity
  ) +
  # 4) Labels and titles with clear, publication-quality text
  labs(
    title = "RMS Displacement Across Wind Speed Bins",
    x     = "Wind Speed (Centered & Scaled bins)",
    y     = "RMS (nm)"
  ) +
  # 5) Refined theme settings for presentation
  theme_minimal(base_size = 12) +
  theme(
    plot.title       = element_text(face = "bold", hjust = 0.5, size = 14),
    axis.text.x      = element_text(angle = 45, hjust = 1),   # Tilt x-axis labels for readability
    legend.position  = "bottom",                               # Place legend below plot
    legend.title     = element_text(size = 12),                # Legend title size
    legend.text      = element_text(size = 10),                # Legend text size
    panel.grid.major = element_line(color = "grey90"),         # Subtle grid lines
    panel.grid.minor = element_blank()                         # Remove minor grid lines
  )
```



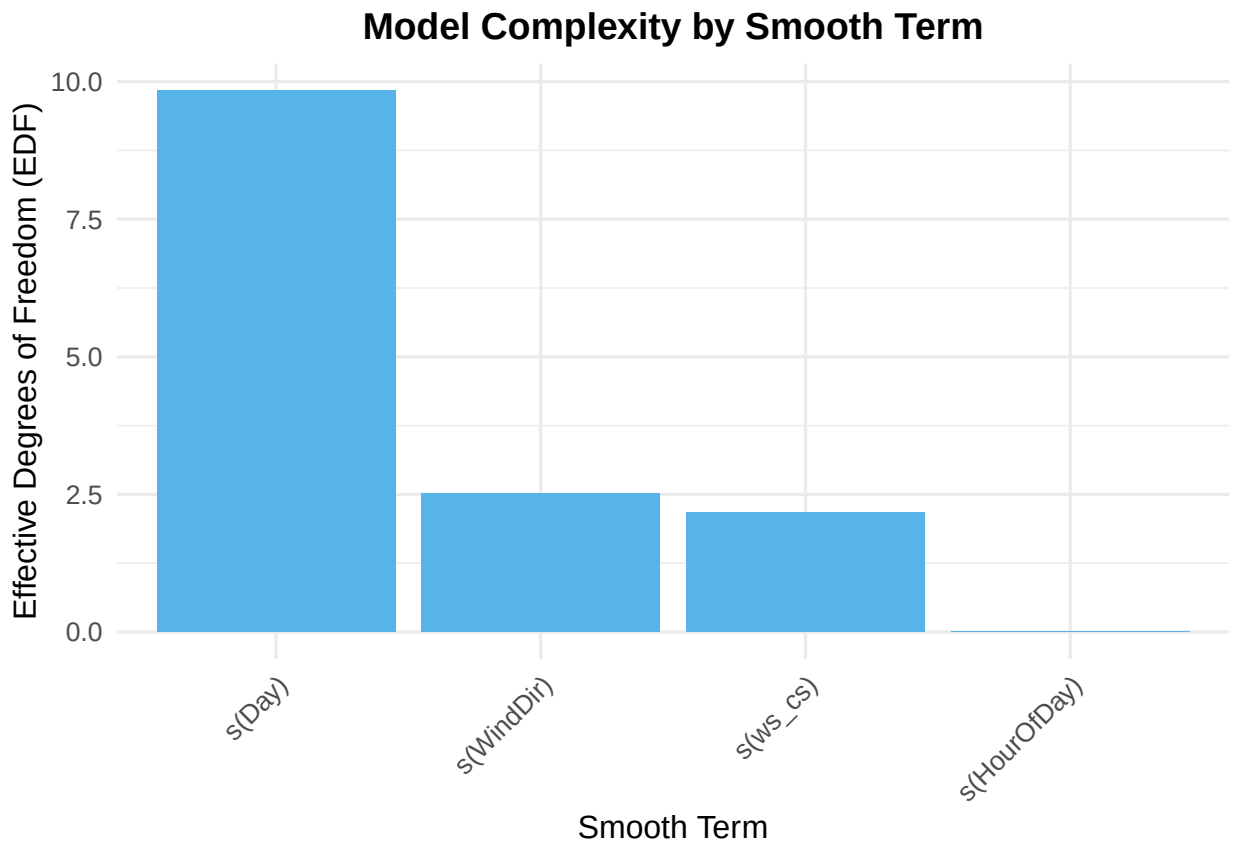

```
##############################################################################
# Interquartile Range of RMS by Wind Speed Bins
# Purpose: Assess how the variability (IQR) of RMS displacement (nm) changes
#          across discrete bins of centered & scaled wind speed for Background
#          vs Operational turbine states.
##############################################################################

# Assumes dplyr and ggplot2 are already loaded.

iqr_ws <- energy_df %>%
  mutate(
    # Convert integrated energy (nm²) to RMS displacement (nm)
    RMS    = sqrt(E),
    # Create descriptive factor for turbine state
    Type   = factor(
               Operational,
               levels = c(0, 1),
               labels = c("Background", "Operational")
             ),
    # Bin centered & scaled wind speed into 6 equal-width categories
    ws_bin = cut(
               ws_cs,
               breaks        = 6,
               include.lowest = TRUE
             )
  ) %>%
```

```r
  group_by(Type, ws_bin) %>%
  summarise(
    # Compute interquartile range of RMS within each wind-speed bin and state
    IQR = IQR(RMS, na.rm = TRUE),
    .groups = "drop"
  )

# Plot IQR vs wind-speed bin for each turbine state
ggplot(iqr_ws, aes(x = ws_bin, y = IQR, color = Type, group = Type)) +
  geom_line(size = 1) +                            # Connect IQR points across bins
  geom_point(size = 2) +                           # Highlight each bin's IQR
  scale_color_manual(
    values = c(
      "Background"  = "#E69F00",                   # Orange for turbine OFF
      "Operational" = "#0072B2"                    # Blue for turbine ON
    ),
    name = "Turbine State"
  ) +
  labs(
    title = "Interquartile Range of RMS by Wind Speed Bin",
    x     = "Wind Speed (Centered & Scaled bins)",
    y     = "IQR of RMS (nm)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    axis.text.x     = element_text(angle = 45, hjust = 1),
    legend.position = "bottom"
  )
```



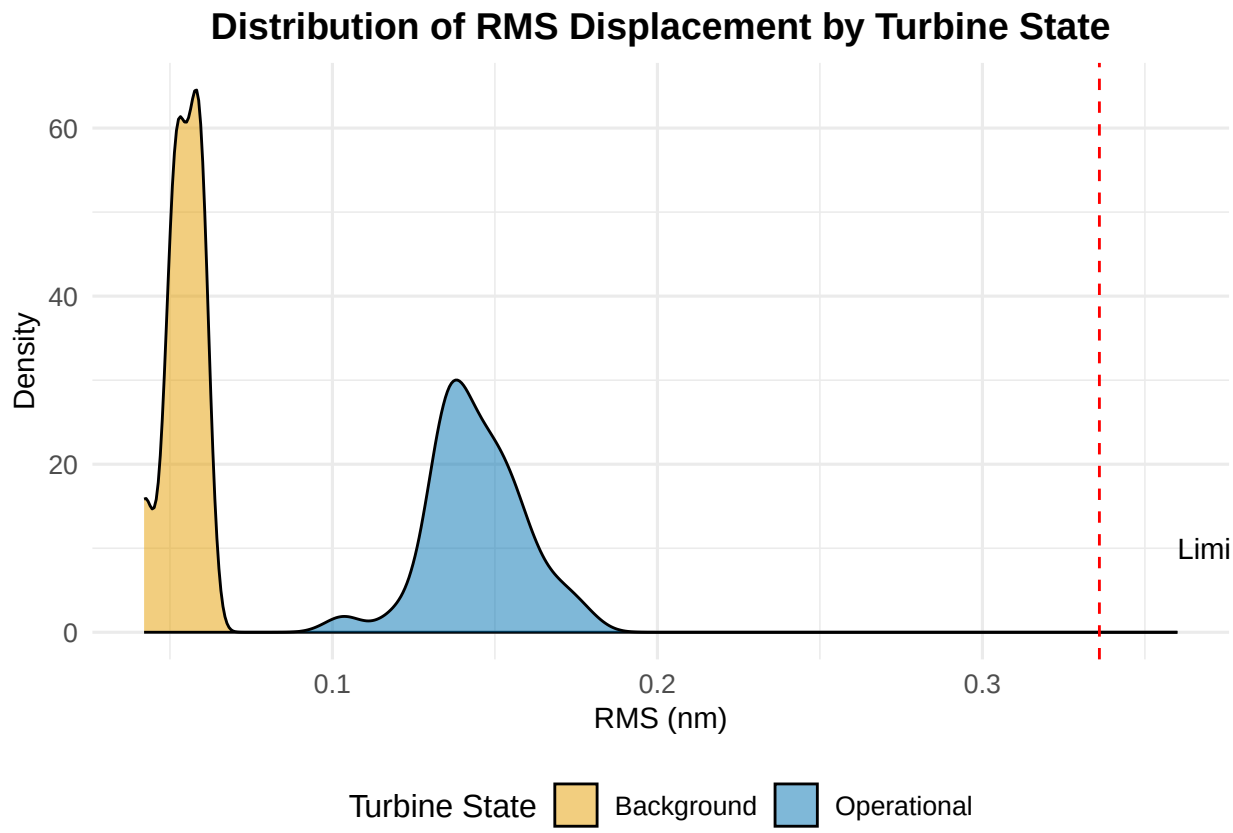

```
##############################################################################
# Visualising Model Complexity via Effective Degrees of Freedom
# Purpose: Extract and plot the EDF for each smooth term in the full GAM,
#          providing a clear summary of the relative complexity (wiggliness)
#          of each smooth component.
##############################################################################

# 1) Extract the smooth term table from the GAM summary
#    - summary(gam_energy)$s.table contains estimated degrees of freedom (edf)
#      and test statistics for each smooth term.
edf_df <- as.data.frame(summary(gam_energy)$s.table)

# 2) Add a 'term' column from the row names to identify each smooth
edf_df$term <- rownames(edf_df)

# 3) Plot EDF by smooth term ordered from most to least complex
#    - geom_col shows a bar for each term's edf
#    - Bar fill "#56B4E9" provides a consistent, publication-quality palette
ggplot(edf_df, aes(x = reorder(term, -edf), y = edf)) +
  geom_col(fill = "#56B4E9") +
  labs(
    title = "Model Complexity by Smooth Term",
    x     = "Smooth Term",                      # Label indicates which smooth
    y     = "Effective Degrees of Freedom (EDF)"# EDF measures wiggliness (higher = more complex)
  ) +
  theme_minimal(base_size = 12) +               # Clean theme for clarity
```

```r
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),   # Tilt x-axis labels for readability
    plot.title  = element_text(face = "bold", hjust = 0.5) # Center and bold the title
  )
```




```r
##############################################################################
# Density Distribution of RMS Displacement by Turbine State
# Purpose: Compare the full distribution of RMS noise for Background vs
#          Operational periods, with the regulatory threshold clearly marked.
##############################################################################

# Load required libraries (skip if already loaded earlier)
library(dplyr)   # Data manipulation: mutate, %>%, etc.
library(ggplot2) # Plotting: ggplot, geom_density, etc.

# 1) Prepare the data frame: compute RMS and create descriptive factor 'Type'
density_df <- energy_df %>%
  mutate(
    RMS  = sqrt(E),                                    # Convert energy (nm²) to RMS (nm)
    Type = factor(
      Operational,
      levels = c(0, 1),
      labels = c("Background", "Operational")
    )                                                  # Human-readable turbine state
  )
```

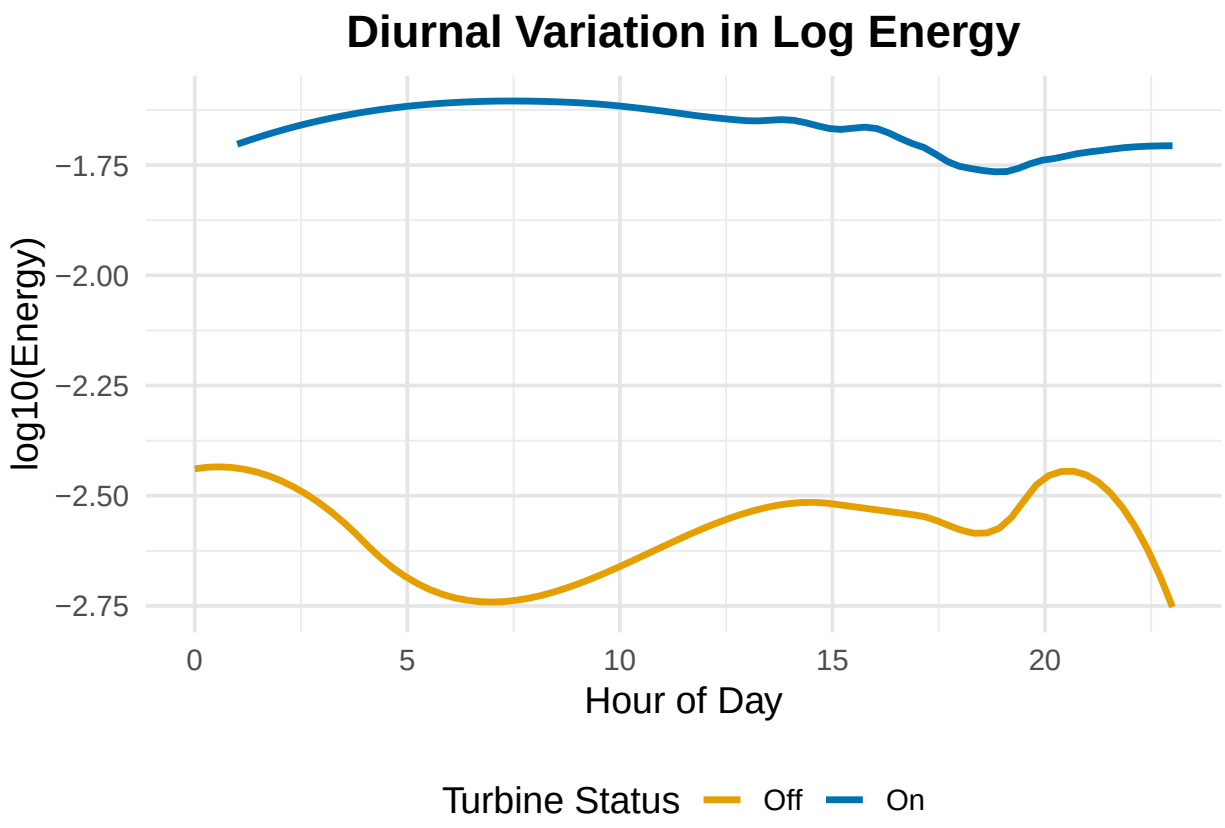
```r
# 2) Plot the density curves with regulatory limit annotation
ggplot(density_df, aes(x = RMS, fill = Type)) +
  geom_density(alpha = 0.5) +                              # Semi-transparent density curves
  geom_vline(
    xintercept = 0.336,                                    # Regulatory RMS limit (nm)
    linetype    = "dashed",
    color       = "red"
  ) +
  annotate(
    "text",
    x     = 0.36,                                           # Place text just above the limit
    y     = 10,                                            # Adjust based on max density
    label = "Limit = 0.336 nm",
    color = "black",
    hjust = 0
  ) +
  scale_fill_manual(
    values = c(
      "Background"  = "#E69F00",                          # Gray for turbine OFF
      "Operational" = "#0072B2"                           # Red for turbine ON
    ),
    name = "Turbine State"                                # Legend title
  ) +
  labs(
    title = "Distribution of RMS Displacement by Turbine State",
    x     = "RMS (nm)",                                   # X-axis label with units
    y     = "Density"                                     # Y-axis label
  ) +
  theme_minimal(base_size = 12) +                         # Clean, legible theme
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom",                           # Place legend below plot
    axis.title      = element_text(size = 11),
    axis.text       = element_text(size = 10)
  )
```




```r
# Diurnal Variation in Log Energy with Legend Placed Below

library(dplyr)
library(ggplot2)

diurnal_df <- energy_df %>%
  group_by(HourOfDay, Operational) %>%
  summarise(
    mean_logE = mean(logE, na.rm = TRUE),
    .groups   = "drop"
  ) %>%
  mutate(
    Status = factor(
      Operational,
      levels = c(0, 1),
      labels = c("Off", "On")
    )
  )

cols <- c(
  "Off" = "#E69F00",
  "On"  = "#0072B2"
)

ggplot(diurnal_df, aes(x = HourOfDay, y = mean_logE, color = Status)) +
  geom_smooth(
```

```r
    method = "loess",
    se     = FALSE,
    span   = 0.6,
    size   = 1.2
  ) +
  scale_color_manual(values = cols, name = "Turbine Status") +
  labs(
    title = "Diurnal Variation in Log Energy",
    x     = "Hour of Day",
    y     = "log10(Energy)"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom",          # move legend below the plot
    panel.grid.major = element_line(color = "grey90")
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'

## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : pseudoinverse used at 20

## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : neighborhood radius 3

## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : reciprocal condition number 0

## Warning in sqrt(sum.squares/one.delta): NaNs produced
```

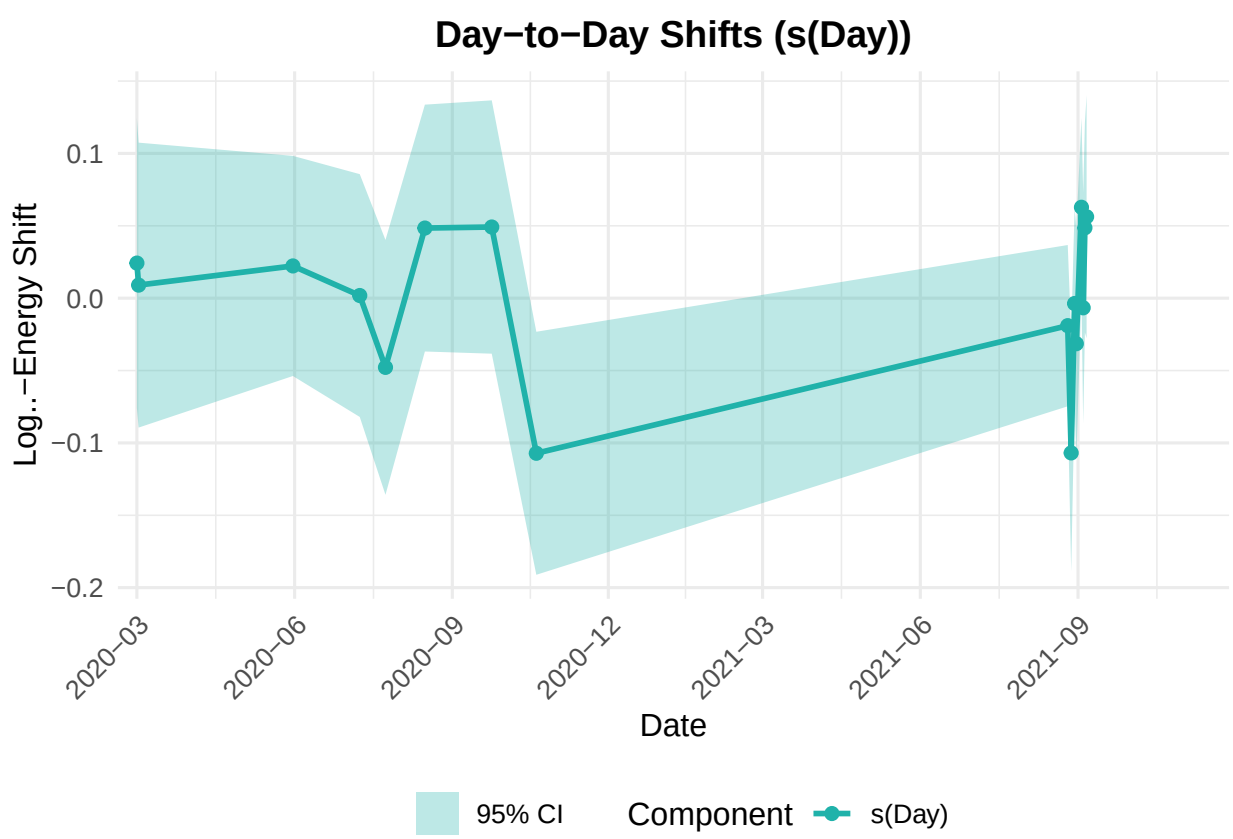



```r
################################################################################
# Day-to-Day Shifts from the s(Day) Term in gam_energy
# Purpose: Extract the estimated random-effect intercepts for each date from
#          the simplified GAM (gam_energy) and plot the daily log-energy shifts
#          with 95% confidence bands over time.
################################################################################

# 0) Load necessary libraries for data manipulation, plotting, and axis formatting
library(dplyr)     # Data wrangling (mutate, transmute, etc.)
library(ggplot2)   # Plotting (ggplot, geom_ribbon, etc.)
library(scales)    # Date-axis formatting helpers (date_labels, breaks)
```

```
##
## Attaching package: 'scales'

## The following object is masked from 'package:viridis':
##
##     viridis_pal

## The following object is masked from 'package:purrr':
##
##     discard

## The following object is masked from 'package:readr':
##
##     col_factor
```

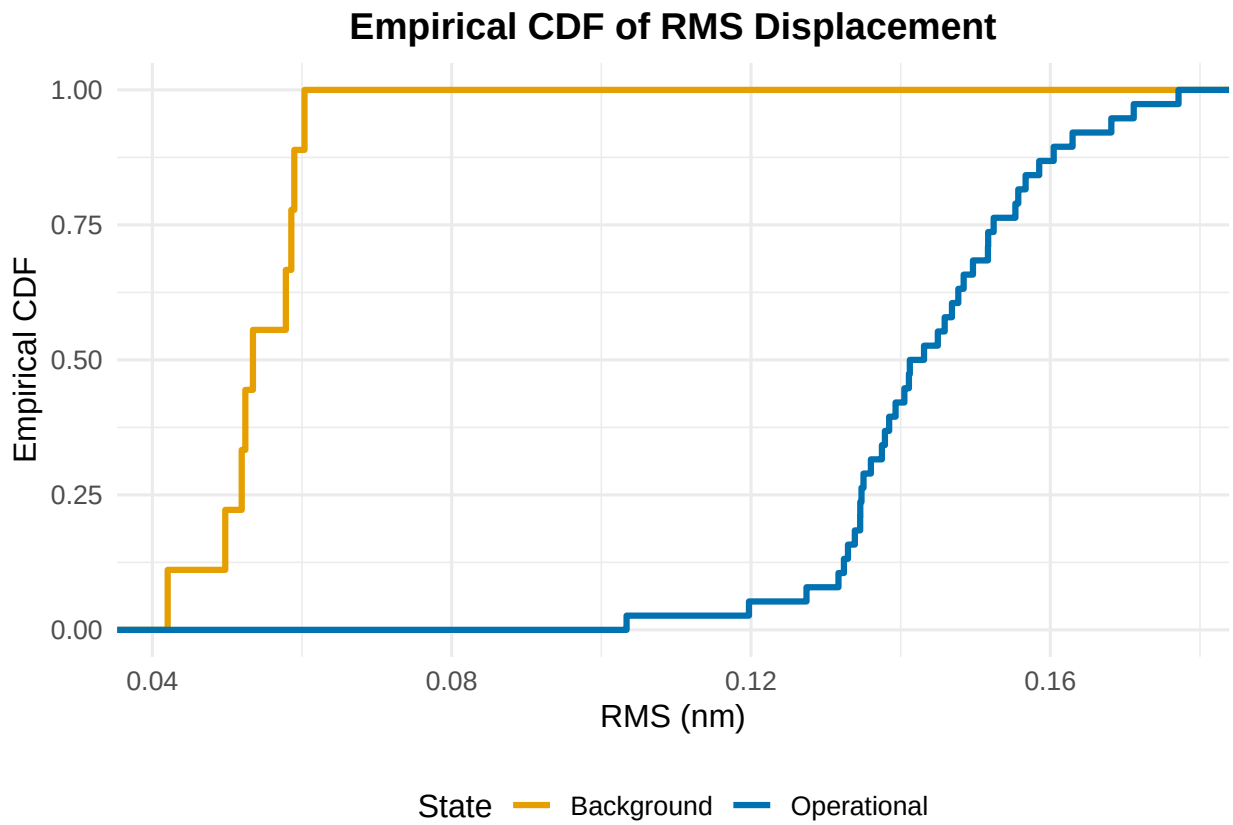
```r
# 1) Build a reference grid containing one row per date
#    - Other predictors fixed at typical values to isolate the day effect
ref_days <- tibble(
  Day         = factor(levels(energy_df$Day), levels = levels(energy_df$Day)),
  ws_cs       = 0,          # Centered & scaled wind-speed at its mean
  WindDir     = 180,        # Wind direction fixed at 180° (southerly)
  HourOfDay   = 12,         # Hour of day fixed at noon UTC
  Operational = 0           # Background (turbine OFF) for baseline
)

# 2) Use predict(..., type="terms") to extract the s(Day) contributions
pred_terms <- predict(
  gam_energy,
  newdata = ref_days,
  type    = "terms",
  se.fit  = TRUE
)
# Identify which column corresponds to the s(Day) smooth
day_col <- grep("^s\\(Day\\)", colnames(pred_terms$fit))

# 3) Assemble a tidy data frame of estimates and 95% confidence intervals
re_df <- ref_days %>%
  transmute(
    Date     = as.Date(as.character(Day)),               # Convert factor to Date class
    Estimate = pred_terms$fit[, day_col],                # Estimated daily intercept shift
    Lower    = Estimate - 2 * pred_terms$se.fit[, day_col],  # Lower 95% CI
    Upper    = Estimate + 2 * pred_terms$se.fit[, day_col]   # Upper 95% CI
  )

# 4) Plot the day-to-day shifts with ribbon and line
p_day <- ggplot(re_df, aes(x = Date)) +
  # Confidence band for the daily effect
  geom_ribbon(
    aes(ymin = Lower, ymax = Upper, fill = "95% CI"),
    alpha = 0.3
  ) +
  # Line and points for the estimated shift
  geom_line(aes(y = Estimate, color = "s(Day)"), size = 1) +
  geom_point(aes(y = Estimate, color = "s(Day)"), size = 2) +

  # Manual scales to label legend entries clearly
  scale_color_manual(
    name   = "Component",
    values = c("s(Day)" = "lightseagreen")
  ) +
  scale_fill_manual(
    name   = "",
    values = c("95% CI" = "lightseagreen")
  ) +

  # Expand x-axis to show every 3 months with YYYY-MM labels
  scale_x_date(
    breaks      = seq(min(re_df$Date), max(re_df$Date), by = "3 months"),
```

```r
    date_labels = "%Y-%m",
    expand      = expansion(mult = c(0.02, 0.15))  # Add 15 days margin at ends
  ) +

  # Titles and axis labels
  labs(
    title = "Day-to-Day Shifts (s(Day))",
    x     = "Date",
    y     = "Log -Energy Shift"
  ) +

  # Clean minimal theme and styling tweaks
  theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom",
    axis.text.x     = element_text(angle = 45, hjust = 1)
  )

# 5) Render the plot
print(p_day)
```



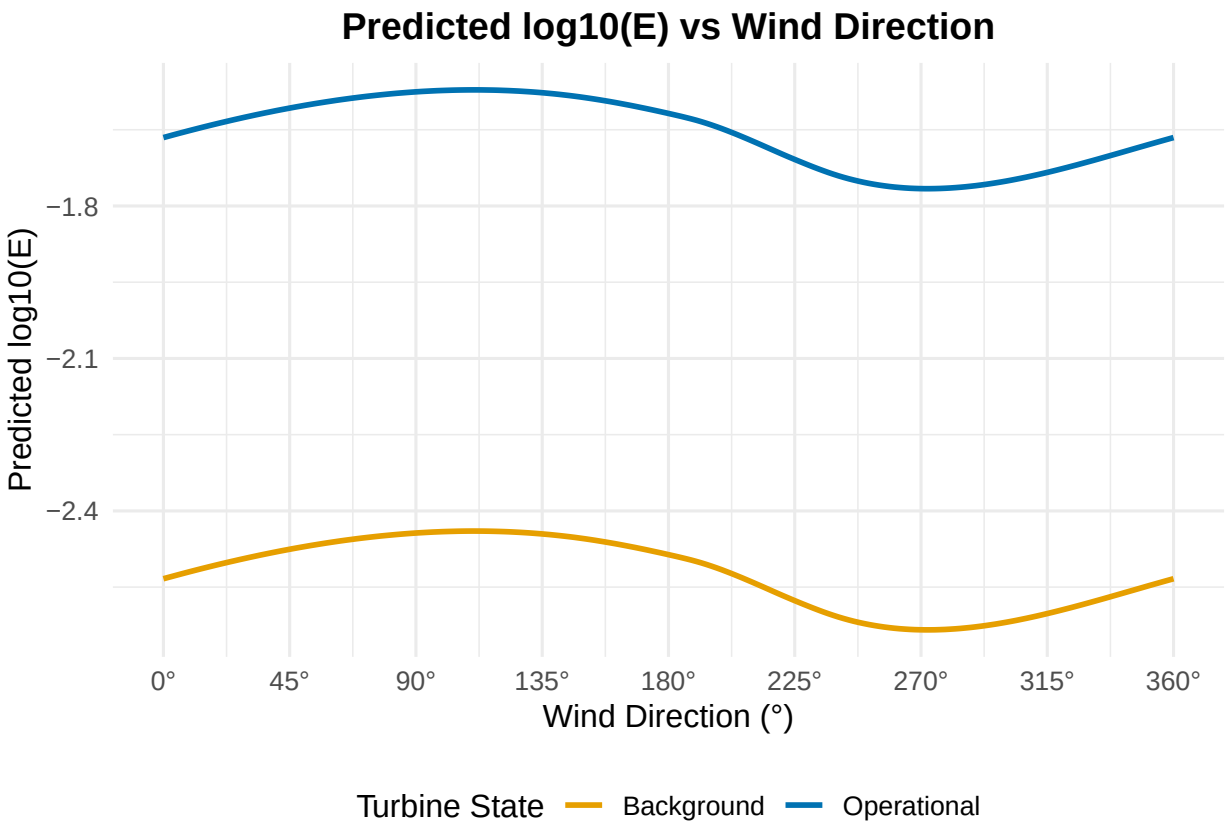

```r
##############################################################################
# Empirical CDF of RMS Displacement by Turbine State
# Purpose: Compare the full distributions of RMS seismic displacement (nm)
```

```r
#          between Background and Operational periods using empirical
#          cumulative distribution functions (ECDFs).
################################################################################

# Load required libraries for data manipulation and plotting
library(dplyr)    # For mutate(), %>%, and data frame operations
library(ggplot2)  # For ggplot(), stat_ecdf(), and theme_minimal()

# 1) Prepare the data frame for plotting
ecdf_df <- energy_df %>%
  mutate(
    # Convert integrated energy (nm²) to RMS displacement (nm)
    RMS   = sqrt(E),
    # Create a descriptive factor for turbine state (Background vs Operational)
    State = factor(
      Operational,
      levels = c(0, 1),                       # Ensure Background comes first
      labels = c("Background", "Operational") # Human-readable labels
    )
  )

# 2) Define custom colors for the two turbine states
cols <- c(
  "Background"  = "#E69F00",
  "Operational" = "#0072B2"
)

# 3) Plot the empirical CDFs
#    - stat_ecdf() computes the ECDF for each group and plots it
#    - Lines are colored by State for direct comparison
ggplot(ecdf_df, aes(x = RMS, color = State)) +
  stat_ecdf(size = 1.1) +                           # Thicker lines improve visibility
  scale_color_manual(values = cols, name = "State") +# Apply custom palette and legend title
  labs(
    title = "Empirical CDF of RMS Displacement",     # Plot title
    x     = "RMS (nm)",                              # X-axis label with units
    y     = "Empirical CDF"                          # Y-axis label
  ) +
  theme_minimal(base_size = 12) +                    # Clean, minimal theme
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom"                       # Place legend below plot
  )
```




```
##############################################################################
# Predicted log (E) vs Wind Direction from the Full GAM
# Purpose: Generate model-predicted log -energy across the full 0°-360° range
#          of wind directions for both Background and Operational states,
#          holding other covariates at typical values to isolate the directional effect.
#############################################################################

# 0) Load required libraries
library(dplyr)   # Data manipulation: expand.grid, mutate, recode_factor
library(ggplot2) # Plotting: ggplot, geom_line, scale_x_continuous, theme_minimal

# 1) Build a prediction grid over WindDir and turbine state
#    - WindDir: 200 equally spaced points from 0° to 360°
#    - ws_cs    fixed at 0 (mean-centered wind speed)
#    - HourOfDay fixed at 12 (noon UTC) to remove diurnal variation
#    - Day      fixed at the first level to isolate only the s(Day) intercept
#    - Operational: factor levels 0 (Background) and 1 (Operational)
dir_grid <- expand.grid(
  WindDir     = seq(0, 360, length = 200),
  ws_cs       = 0,
  HourOfDay   = 12,
  Day         = factor(
                  levels(energy_df$Day)[1],
                  levels = levels(energy_df$Day)
                ),
  Operational = factor(c(0, 1), levels = c(0, 1))
```

```r
)

# 2) Predict log -energy for each grid point using the fitted GAM
#    - type = "response" returns fitted log (E) on the original scale
dir_grid$pred_logE <- predict(
  gam_energy,
  newdata = dir_grid,
  type    = "response"
)

# 3) Create a descriptive 'State' factor for plotting
dir_grid <- dir_grid %>%
  mutate(
    State = recode_factor(
      as.character(Operational),
      `0` = "Background",
      `1` = "Operational"
    )
  )

# 4) Plot the predictions
#    - geom_line() draws smooth curves of predicted log (E) vs WindDir
#    - scale_colour_manual() applies distinct colours for each state
#    - scale_x_continuous() labels the x-axis in degrees at 45° increments
ggplot(dir_grid, aes(x = WindDir, y = pred_logE, colour = State)) +
  geom_line(size = 1) +
  scale_colour_manual(
    name   = "Turbine State",
    values = c(
      "Background"  = "#E69F00",
      "Operational" = "#0072B2"
    )
  ) +
  scale_x_continuous(
    limits = c(0, 360),
    breaks = seq(0, 360, 45),
    labels = paste0(seq(0, 360, 45), "°")
  ) +
  labs(
    title  = "Predicted log10(E) vs Wind Direction",
    x      = "Wind Direction (°)",
    y      = "Predicted log10(E)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", hjust = 0.5),
    legend.position = "bottom"
  )
```


Predicted log10(E) vs Wind Direction
Predicted log10(E)
−1.8
−2.1
−2.4
0°
45°
90°
135°
180°
225°
270°
315°
360°
Wind Direction (°)
Turbine State
Background
Operational